



TRABAJO DE FIN DE GRADO
GRADO EN CIENCIA E INGENIERÍA DE DATOS

Representación y análisis de células mitóticas en histología H&E mediante autoencoders dispersos basados en LISTA

Estudiante: Enrique Pardo García

Dirección: José Rouco Maseda

Jorge Novo Buján

A Coruña, septiembre de 2026.

Amo como ama el amor. No conozco otra razón para amar que amarte. A quienes me acompañaron con amor durante esta etapa, les dedico este trabajo.

Agradecimientos

Nunca pensé que llegaría el momento de escribir esto, y ahora que lo recibo con los brazos abiertos, no encuentro las palabras justas para estar a la altura de este proceso.

Agradezco tanto a Basi como a Ana por ser la luz que me facilita cada paso que doy: desde aguantar mis manías hasta celebrar cada logro a mi lado.

A mis abuelos, tanto los que me acompañan en tierra como los que me guían desde el cielo. Ojalá el paso de los años nunca existiese para vosotros.

A mis tíos, por querer siempre lo mejor para el chico que visteis crecer. Todo es mucho más fácil a vuestro lado.

A mis tutores José Rouco y Jorge Novo, por darme la confianza necesaria para comenzar y terminar este reto, por vuestro constante acompañamiento y orientación.

A Daniela, por permitirme irrumpir en su vida. Por cada palabra de aliento, por cada silencio compartido, y por hacer que este camino mereciera la pena solo por tenerla cerca.

Finalmente, termino agradeciendome a mí mismo. Al canijo que siempre soñó con rozar las nubes con las yemas de los dedos, aunque eso implicase ponerse de puntillas.

Resumen

La detección automática de figuras mitóticas en imágenes histopatológicas H&E constituye una tarea de gran relevancia clínica, pero limitada por el elevado coste de obtener anotaciones de calidad. Este trabajo explora si las representaciones dispersas aprendidas de forma no supervisada mediante la arquitectura LISTA son capaces de capturar información discriminativa entre figuras mitóticas y no mitóticas, sin emplear etiquetas durante el entrenamiento. Para ello se diseña e implementa un autoencoder disperso basado en un encoder LISTA y un decoder de pesos atados, evaluado sobre parches del dataset MIDOG en seis configuraciones experimentales que combinan tres funciones de pérdida de reconstrucción –MSE, MAE y SSIM– con dos dimensiones del diccionario, $m = 128$ y $m = 1024$. Los resultados muestran que, si bien las tres funciones de pérdida se comportan de forma equivalente en el régimen de diccionario reducido, su comportamiento diverge radicalmente al escalar el tamaño del código: MSE y MAE colapsan a una representación trivial en la práctica totalidad de su trayectoria de entrenamiento, mientras que SSIM mantiene la estabilidad e incluso mejora la calidad de reconstrucción. El análisis del potencial discriminativo revela una señal débil pero consistente, presente en átomos individuales del diccionario aunque no traducida en una separación visible del espacio de código. Estos hallazgos sitúan la elección de la función de pérdida y el tamaño del diccionario como los factores más determinantes de la viabilidad del enfoque propuesto.

Abstract

Automatic detection of mitotic figures in H&E histopathology images is a task of high clinical relevance, but one constrained by the high cost of obtaining quality annotations. This work explores whether sparse representations learned in an unsupervised manner through the LISTA architecture are able to capture discriminative information between mitotic and non-mitotic figures, without using labels during training. To this end, a sparse autoencoder is designed and implemented, composed of a LISTA encoder and a tied-weights decoder, and evaluated on patches from the MIDOG dataset across six experimental configurations combining three reconstruction loss functions –MSE, MAE and SSIM– with two dictionary dimensions, $m = 128$ and $m = 1024$. The results show that, while the three loss functions behave equivalently under the smaller dictionary regime, their behaviour diverges sharply when the code size is scaled up: MSE and MAE collapse to a trivial representation over the vast majority of their training trajectory, whereas SSIM remains stable and even improves reconstruction quality. The analysis of discriminative potential reveals a weak but consistent

signal, present in individual dictionary atoms yet not translated into a visible separation of the code space. These findings identify the choice of loss function and dictionary size as the most decisive factors for the viability of the proposed approach.

Palabras clave:

- Autoencoders dispersos
- Arquitectura LISTA
- Aprendizaje no supervisado
- Histología H&E
- Detección de figuras mitóticas
- Funciones de pérdida MSE, MAE y SSIM
- Codificación dispersa

Keywords:

- Sparse autoencoders
- LISTA architecture
- Unsupervised learning
- H&E histology
- Mitotic figure detection
- MSE, MAE and SSIM loss functions
- Sparse coding

Índice general

1	Introducción	1
1.1	Contexto clínico: detección de figuras mitóticas	1
1.2	Motivación del aprendizaje no supervisado	3
1.3	Objetivos del trabajo	4
1.4	Estructura de la memoria	4
2	Estado del arte	6
2.1	Imágenes histopatológicas H&E	7
2.1.1	Imágenes de lámina completa y procesamiento a nivel de parche . . .	7
2.2	Conjuntos de datos para la detección de figuras mitóticas	9
2.3	Métodos de detección mitótica en el estado del arte	10
2.3.1	Métodos clásicos basados en características manuales	10
2.3.2	Métodos basados en aprendizaje profundo supervisado	11
2.3.3	Métodos basados en aprendizaje no supervisado y auto-supervisado .	11
2.4	Codificación dispersa	12
2.4.1	Formulación del problema	12
2.4.2	Normas de dispersión: l0 y l1	13
2.4.3	Problema de inferencia	13
2.4.4	Aprendizaje del diccionario	14
2.5	Algoritmos iterativos de codificación dispersa	14
2.5.1	ISTA	15
2.5.2	FISTA	16
2.6	LISTA: Learned ISTA	16
2.6.1	Arquitectura	17
2.6.2	Entrenamiento	17
2.6.3	Ventajas frente a ISTA	18
2.7	Autoencoders	19

2.8	Discusión	21
3	Metodología y planificación	23
3.1	Modelo de desarrollo	23
3.2	Planificación	23
3.3	Seguimiento y desviaciones de ejecución no previstas	25
3.4	Recursos	27
3.4.1	Recursos materiales	27
3.4.2	Recursos humanos	28
3.5	Estimación de costes	28
3.5.1	Costes de recursos materiales	28
3.5.2	Costes de recursos humanos	28
4	Trabajo desarrollado	30
4.1	Visión general de la metodología	30
4.2	Dataset y preprocesado de parches	31
4.2.1	Estandarización	31
4.2.2	Partición train/test	33
4.2.3	Aumentación de datos	33
4.3	Arquitectura del modelo: encoder y decoder LISTA	34
4.3.1	Encoder LISTA	34
4.3.2	Decoder con pesos atados	35
4.3.3	Configuración del número de iteraciones	36
4.4	Funciones de pérdida de reconstrucción	36
4.4.1	Formulación general: ponderación espacial de la diferencia punto a punto	37
4.4.2	Error cuadrático medio (MSE)	38
4.4.3	Error absoluto medio (MAE)	38
4.4.4	Similitud estructural (SSIM)	38
4.5	Máscara Gaussiana centrada	39
4.6	Detalles de entrenamiento y experimentación	40
4.6.1	Configuraciones experimentales	41
4.6.2	Fases de entrenamiento	41
4.6.3	Calentamiento inicial	42
4.6.4	Reactivación de átomos muertos	42
4.6.5	Ajuste dinámico del peso de dispersión λ_{L1}	42
4.6.6	Gestión de checkpoints	43
4.6.7	Hiperparámetros	44

4.7	Métricas de evaluación	44
4.7.1	Métricas de reconstrucción	44
4.7.2	Métricas de dispersión	46
4.7.3	Métricas de coherencia del diccionario	46
4.7.4	Métricas de discriminabilidad	47
5	Experimentos y resultados	48
5.1	Configuración experimental y protocolo de evaluación	48
5.2	Dinámica del entrenamiento	49
5.2.1	Curvas de pérdida: total, reconstrucción y término L1	49
5.2.2	Controlador automático del peso de dispersión λ_{L1}	50
5.2.3	Evolución de la dispersión durante el entrenamiento	52
5.2.4	Vida y muerte de átomos durante el entrenamiento	52
5.2.5	Detección de saltos y reorganización del diccionario	53
5.2.6	Comparación de la dinámica entre configuraciones	53
5.3	Calidad de reconstrucción	54
5.3.1	Resultados cuantitativos (MAE_g , MSE_g , PSNR, $SSIM_g$)	54
5.3.2	Reconstrucciones cualitativas por clase	55
5.3.3	Evolución del checkpoint a lo largo del entrenamiento	56
5.4	Análisis de la dispersión del código	59
5.4.1	Distribución de átomos activos por muestra	59
5.4.2	Compromiso reconstrucción–dispersión	60
5.5	Análisis del diccionario aprendido	62
5.5.1	Visualización de los átomos más activos	62
5.5.2	Descomposición de un parche en átomos	62
5.5.3	Salud de los átomos sobre el conjunto de test	62
5.5.4	Coherencia del diccionario	64
5.6	Potencial discriminativo de las representaciones	67
5.6.1	Activación media por clase y selección de atributos	67
5.6.2	Visualización del espacio de código	67
5.7	Efecto del tamaño del diccionario	70
5.8	Discusión	71
6	Conclusiones	74
6.1	Situación final del trabajo	74
6.2	Lecciones aprendidas	76
6.3	Relación con las competencias de la titulación	77
6.4	Líneas futuras	77

A Material adicional	80
A.1 Detalle de los episodios de recuperación espontánea en $m = 1024$	80
A.2 Reconstrucciones cualitativas complementarias	81
A.3 Trayectorias complementarias de reconstrucción	85
Lista de acrónimos	87
Bibliografía	89

Índice de figuras

1.1	Comparación entre figura mitótica y no mitótica	2
2.1	Tejido histopatológico H&E con figuras mitótica y no mitótica señaladas	8
2.2	Ejemplo de codificación dispersa LISTA sobre un parche mitótico	14
2.3	Función de umbralización suave de ISTA	16
2.4	Diagrama de la arquitectura LISTA	18
2.5	Esquema general de un autoencoder	19
3.1	Diagrama de Gantt del proyecto	26
4.1	Visión general de la metodología	32
4.2	Efecto de las transformaciones de aumentación de datos	34
4.3	Arquitectura del autoencoder disperso LISTA	36
4.4	Máscara Gaussiana centrada	40
5.1	Evolución del entrenamiento para mse_aug_128	50
5.2	Evolución del entrenamiento para mse_aug_1024: colapso	51
5.3	Evolución del entrenamiento para ssim_aug_1024: estabilidad	51
5.4	Reconstrucción de un parche mitótico, mse_aug_128	56
5.5	Reconstrucción de un parche mitótico, mse_aug_1024: colapso	57
5.6	Reconstrucción de un parche mitótico, ssim_aug_1024: estabilidad	57
5.7	Trayectoria de reconstrucción, mse_aug_1024	58
5.8	Trayectoria de reconstrucción, ssim_aug_1024	59
5.9	Distribución de átomos activos por muestra	60
5.10	Compromiso entre dispersión y reconstrucción cruzada	61
5.11	Átomos de mayor magnitud media de activación	63
5.12	Seis átomos de mayor coeficiente, ssim_aug_128	63
5.13	Seis átomos de mayor coeficiente, ssim_aug_1024	63

5.14	Tasa de activación y salud de átomos, <code>ssim_aug_128</code>	64
5.15	Recuento de átomos vivos/agonizantes/muertos, <code>ssim_aug_1024</code>	65
5.16	Matriz de coherencia, <code>ssim_aug_128</code>	66
5.17	Matriz de coherencia, <code>ssim_aug_1024</code>	66
5.18	Índice de Fisher por átomo, seis configuraciones	68
5.19	Comparativa de criterios de selección, <code>mse_aug</code>	68
5.20	Estructura del espacio de código por clase, <code>mse_aug_128</code>	69
5.21	Proyecciones PCA y t-SNE del espacio de código	70
A.1	Parche mitótico de test, <code>mae_aug_128</code>	82
A.2	Parche no mitótico de test, <code>mae_aug_128</code>	82
A.3	Parche mitótico de test, <code>ssim_aug_128</code>	83
A.4	Parche no mitótico de test, <code>ssim_aug_128</code>	83
A.5	Parche mitótico de test, <code>mae_aug_1024</code>	84
A.6	Parche no mitótico de test, <code>mae_aug_1024</code>	84
A.7	Trayectoria de reconstrucción, <code>mse_aug_128</code>	85
A.8	Trayectoria de reconstrucción para <code>mae_aug_128</code>	86
A.9	Trayectoria de reconstrucción para <code>ssim_aug_128</code>	86
A.10	Trayectoria de reconstrucción para <code>mae_aug_1024</code>	86

Índice de tablas

3.1	Estimación de costes de recursos humanos del proyecto.	29
4.1	Resumen de las seis configuraciones experimentales	41
4.2	Resumen de hiperparámetros de los experimentos	45
5.1	Trayectoria del controlador λ_{L1} para las seis configuraciones.	52
5.2	Dinámica de vida y muerte de átomos para las seis configuraciones.	53
5.3	Comparativa de la dinámica de entrenamiento para las seis configuraciones. .	54
5.4	Métricas de reconstrucción sobre el test compartido	55
5.5	Estadísticos de dispersión y reconstrucción cruzada	61
5.6	Coherencia mutua del diccionario	65
5.7	Átomo de mayor índice de Fisher por configuración	69

Introducción

La detección automática de figuras mitóticas en imágenes histopatológicas constituye uno de los problemas abiertos más relevantes en el ámbito de la visión por computador aplicada a la patología digital. Su resolución tiene un impacto directo en la práctica clínica, donde la evaluación manual de la actividad mitótica es una tarea costosa, tediosa y sujeta a una considerable variabilidad entre observadores.

Este capítulo presenta el contexto y la motivación del trabajo. En primer lugar, se describe la relevancia clínica de la detección de figuras mitóticas y las limitaciones del proceso manual actual. A continuación, se argumenta por qué el aprendizaje no supervisado resulta especialmente adecuado para este dominio, dadas las dificultades inherentes a la obtención de datos etiquetados de calidad. Finalmente, se enuncian los objetivos concretos del trabajo y se describe la organización del resto del documento.

1.1 Contexto clínico: detección de figuras mitóticas

LA detección de figuras mitóticas en imágenes de microscopía histopatológica responde a una tarea de relevancia clínica fundamental en el diagnóstico y pronóstico del cáncer. La mitosis es el proceso mediante el cual una célula se divide para dar lugar a dos células hijas genéticamente idénticas. En tejido tumoral, una elevada tasa de figuras mitóticas es un indicativo de una mayor agresividad del tumor y se correlaciona directamente con un peor pronóstico para el paciente [1].

En la práctica clínica habitual, la evaluación de la actividad mitótica se realiza sobre cortes histológicos teñidos con [Hematoxilina-Eosina \(H&E\)](#), la tinción de referencia en anatomía patológica. El patólogo examina al microscopio regiones de tejido seleccionadas, contando manualmente las figuras mitóticas presentes en un número determinado de campos de gran aumento, pudiéndose obtener así el denominado índice mitótico. Este índice forma parte de sistemas de gradación tumoral ampliamente utilizados, como el sistema de Elston-Ellis pa-

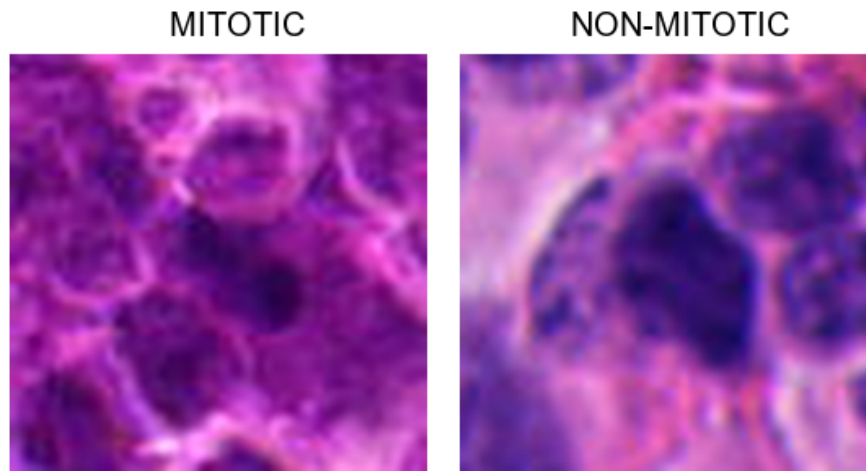


Figura 1.1: Ejemplos de parches de histología H&E. Izquierda: figura mitótica. Derecha: figura no mitótica. La similitud morfológica entre ambas clases ilustra la dificultad inherente a la tarea de detección.

ra el carcinoma de mama [1] o el sistema de gradación de tumores de células del estroma gastrointestinal.

Sin embargo, este proceso presenta importantes limitaciones. En primer lugar, es una tarea tediosa y consumidora de tiempo, especialmente cuando se requiere analizar grandes extensiones de tejido. En segundo lugar, está sujeta a una considerable variabilidad inter e intraobservador, incluso entre patólogos experimentados, derivada tanto de la ambigüedad morfológica de las figuras mitóticas como de las condiciones de tinción y preparación del tejido [2]. La figura 1.1 ilustra esta dificultad: la similitud visual entre figuras mitóticas y no mitóticas hace que incluso un observador experto pueda cometer errores de clasificación.

Estas limitaciones motivan el desarrollo de métodos automáticos de asistencia al patólogo basados en visión por computador y aprendizaje automático. En los últimos años, el avance de las técnicas de aprendizaje profundo ha permitido obtener resultados prometedores en la detección automática de dichas figuras mitóticas, impulsado en parte por la disponibilidad de datasets públicos de referencia como el [Mitosis Domain Generalization Challenge \(MIDOG\)](#) [3]. El dataset [MIDOG](#) proporciona parches de imagen etiquetados procedentes de escáneres y tejidos tumorales de distinta naturaleza, constituyendo un banco de pruebas estándar para la evaluación de modelos en un contexto clínico realista.

No obstante, el etiquetado de este tipo de imágenes a nivel de parche es especialmente costoso, ya que requiere la intervención de patólogos expertos y presenta una considerable variabilidad entre anotadores [2]. Esta dificultad en la obtención de datos etiquetados de cali-

dad hace que los métodos de aprendizaje no supervisado resulten particularmente atractivos para este dominio, tal y como se discutirá en la sección siguiente.

1.2 Motivación del aprendizaje no supervisado

EL desarrollo de métodos automáticos para el análisis de imágenes histopatológicas se ha visto históricamente condicionado por la disponibilidad de datos etiquetados de calidad. A diferencia de otros dominios donde el etiquetado puede realizarse de forma masiva y a bajo coste, la anotación de figuras mitóticas en imágenes H&E requiere la intervención de patólogos expertos, lo que convierte este proceso en un cuello de botella tanto en términos económicos como de tiempo [2].

Esta escasez de datos etiquetados ha despertado un gran interés por los métodos de aprendizaje no supervisado, capaces de extraer representaciones útiles de los datos sin depender de anotaciones exhaustivas [4]. En lugar de aprender a clasificar directamente a partir de etiquetas, estos métodos buscan descubrir la estructura subyacente de los datos, obteniendo representaciones compactas que capturen las características más relevantes de las imágenes.

En este contexto, los autoencoders constituyen una familia de modelos no supervisados ampliamente utilizada para el aprendizaje de representaciones. Un autoencoder aprende a comprimir la entrada en una representación de menor dimensión (código) y a reconstruirla a partir de dicho código, de forma que la representación aprendida pueda capturar la información más relevante de los datos [5].

Entre las distintas variantes de autoencoders, los autoencoders dispersos introducen una restricción adicional sobre el código aprendido: se penaliza que un número elevado de componentes del código estén activos simultáneamente. Esta restricción de dispersión favorece la obtención de representaciones en las que cada muestra se describe mediante la activación de un número reducido de elementos de un diccionario, lo que dota a las representaciones de una mayor interpretabilidad y selectividad [6, 7].

En el dominio de la histología, esta propiedad resulta especialmente interesante: si el modelo aprende un diccionario de patrones visuales relevantes para las imágenes H&E, cada parche quedará representado por la combinación de un pequeño número de dichos patrones. Esta representación compacta e interpretable podría facilitar el análisis posterior de las estructuras celulares presentes en los parches, y en particular, revelar diferencias sistemáticas entre la representación de figuras mitóticas y no mitóticas sin haber usado las etiquetas durante el entrenamiento.

Este trabajo parte precisamente de esta motivación: explorar si las representaciones dispersas aprendidas de forma no supervisada mediante la arquitectura [Learned Iterative Shrinkage-Thresholding Algorithm \(LISTA\)](#) [8] son capaces de capturar información discriminativa entre

ambas clases, y analizar cómo la elección de la función de pérdida de reconstrucción influye en la calidad y el potencial discriminativo de dichas representaciones.

1.3 Objetivos del trabajo

EL presente trabajo tiene como objetivo principal el estudio y desarrollo de autoencoders dispersos basados en la arquitectura [LISTA](#) para la representación de parches de histología [H&E](#), explorando el efecto de distintas funciones de pérdida de reconstrucción sobre la calidad y el potencial discriminativo de las representaciones obtenidas. De forma más concreta, los objetivos específicos del trabajo son los siguientes:

1. **Diseño e implementación del modelo.** Diseñar e implementar un autoencoder disperso basado en la arquitectura [LISTA](#), incluyendo el encoder iterativo de [ISTA](#) y el decoder con pesos atados, junto con la infraestructura completa de entrenamiento, evaluación y visualización.
2. **Exploración de funciones de pérdida de reconstrucción.** Estudiar el efecto de distintas funciones de pérdida — error cuadrático medio ([MSE](#)), error absoluto medio ([MAE](#)) y similitud estructural ([SSIM](#)) — sobre las representaciones aprendidas, analizando su influencia tanto en la calidad de reconstrucción como en las propiedades de dispersión del código obtenido.
3. **Evaluación cuantitativa de los modelos.** Evaluar de forma sistemática los modelos entrenados mediante métricas de reconstrucción y dispersión, comparando el comportamiento de las distintas configuraciones experimentales propuestas.
4. **Análisis del potencial discriminativo.** Estudiar las representaciones aprendidas en relación a las clases de interés, analizando si los códigos dispersos obtenidos de forma no supervisada capturan información discriminativa entre figuras mitóticas y no mitóticas, mediante técnicas de análisis cualitativo.
5. **Análisis del efecto del tamaño del diccionario.** Estudiar la influencia del número de átomos del diccionario sobre la calidad de las representaciones y su capacidad discriminativa, comparando distintas dimensiones del espacio de átomos.

1.4 Estructura de la memoria

EL presente documento se organiza en seis capítulos, incluyendo el presente capítulo introductorio. El contenido del resto de capítulos se resume a continuación:

- **Capítulo 2: Estado del arte.** Revisa los fundamentos teóricos necesarios para situar el trabajo en su contexto científico. Se presentan las imágenes histopatológicas [H&E](#) y el dataset [MIDOG](#), los principios de la codificación dispersa clásica, los algoritmos iterativos de umbralización [ISTA](#) y [FISTA](#), la arquitectura [LISTA](#) y su relación con el aprendizaje de representaciones mediante autoencoders, y una revisión de los métodos de detección mitótica en el estado del arte.
- **Capítulo 3: Metodología y planificación.** Describe el modelo de desarrollo iterativo e incremental seguido en el proyecto, la planificación temporal y el diagrama de Gantt, los recursos materiales y humanos empleados, y la estimación de costes del trabajo.
- **Capítulo 4: Trabajo desarrollado.** Describe el diseño experimental del trabajo. Se detallan el dataset empleado y su preprocesado, la arquitectura del autoencoder disperso basado en [LISTA](#), las tres funciones de pérdida de reconstrucción estudiadas, el uso de la máscara Gaussiana centrada, la estrategia de entrenamiento y las métricas de evaluación utilizadas.
- **Capítulo 5: Experimentos y resultados.** Presenta los resultados obtenidos para las seis configuraciones experimentales definidas, analizando la evolución del entrenamiento, la evaluación cuantitativa mediante métricas de reconstrucción y dispersión, la visualización de las representaciones aprendidas, el análisis del potencial discriminativo y el efecto del tamaño del diccionario.
- **Capítulo 6: Conclusiones.** Recoge las conclusiones del trabajo, su relación con las competencias del grado y las líneas futuras de investigación.

Estado del arte

ESTE capítulo revisa los fundamentos teóricos y el estado del arte en los ámbitos que confluyen en el presente trabajo. Se parte del dominio de aplicación —las imágenes histopatológicas [H&E](#) y los conjuntos de datos disponibles para la detección de figuras mitóticas— para situar el problema en su contexto clínico y establecer los requisitos que deben satisfacer los métodos automáticos. A continuación se revisan los métodos de detección mitótica existentes, desde los enfoques clásicos basados en características manuales hasta los métodos supervisados y auto-supervisados más recientes, lo que permite identificar las limitaciones del estado del arte que motivan el enfoque adoptado en este trabajo.

Los bloques siguientes cubren los fundamentos del modelo propuesto. Se introduce la codificación dispersa como paradigma de aprendizaje de representaciones, con su formulación variacional y las herramientas de optimización asociadas: el algoritmo [ISTA](#), su extensión acelerada [FISTA](#), y la arquitectura [LISTA](#), que sustituye el proceso iterativo de inferencia por una red neuronal entrenable de profundidad fija. Finalmente, se encuadra el modelo propuesto dentro del marco más general de los autoencoders, describiendo las variantes relevantes y su relación con la arquitectura [LISTA](#).

El capítulo se organiza de la siguiente forma:

- La sección [2.1](#) describe las imágenes histopatológicas [H&E](#), las tareas de análisis computacional que se realizan sobre ellas y el procesamiento a nivel de parche mediante escáneres [WSI](#).
- La sección [2.2](#) revisa los principales conjuntos de datos públicos para la detección de figuras mitóticas y justifica la elección del dataset [MIDOG](#).
- La sección [2.3](#) revisa los métodos de detección mitótica en el estado del arte, desde los enfoques clásicos hasta los métodos supervisados y auto-supervisados más recientes.
- La sección [2.4](#) introduce el paradigma de la codificación dispersa, su formulación variacional y el aprendizaje del diccionario.

- La sección 2.5 describe los algoritmos iterativos de inferencia **ISTA** y **FISTA**, con sus operadores proximales asociados.
- La sección 2.6 presenta la arquitectura **LISTA** como aproximación basada en aprendizaje para acelerar la inferencia dispersa, describiendo su estructura, entrenamiento y ventajas frente a los algoritmos iterativos clásicos.
- La sección 2.7 encuadra el modelo propuesto dentro del marco general de los autoencoders, describiendo las variantes más relevantes y su relación con **LISTA**.
- La sección 2.8 cierra el capítulo con una discusión comparativa que destaca las diferencias concretas del enfoque propuesto respecto al estado del arte en cada uno de los ámbitos revisados.

2.1 Imágenes histopatológicas H&E

LA histología es la disciplina biomédica que estudia la composición y estructura microscópica de los tejidos. En el contexto del diagnóstico oncológico, el análisis histopatológico de biopsias teñidas constituye el método de referencia para la caracterización de tumores y la determinación de su grado de malignidad [1]. Entre las distintas técnicas de tinción disponibles, la hematoxilina-eosina (**H&E**) es con diferencia la más empleada en la práctica clínica habitual. La hematoxilina tiñe los núcleos celulares de azul-violeta, mientras que la eosina tiñe el citoplasma y la matriz extracelular de rosa, proporcionando un contraste visual que permite distinguir las distintas estructuras celulares presentes en el tejido.

El análisis computacional de imágenes **H&E** abarca una amplia variedad de tareas con aplicación clínica directa. Entre las más relevantes se encuentran la *segmentación tisular*, que delimita regiones anatómicas como estroma, tumor o necrosis [9]; la *segmentación de instancias nucleares*, que identifica y delinea individualmente cada núcleo celular [10]; la *estimación de celularidad tumoral*, que cuantifica la proporción de células cancerosas en el tejido [11]; y la *detección y clasificación de figuras mitóticas*, orientada a identificar células en proceso de división [2]. Este último problema es el que constituye el objeto del presente trabajo, mientras que las restantes tareas quedan fuera de su ámbito de aplicación.

2.1.1 Imágenes de lámina completa y procesamiento a nivel de parche

Las imágenes histopatológicas se obtienen habitualmente mediante escáneres digitales de **Whole Slide Image (WSI)**. Un escáner **WSI** digitaliza una lámina histológica completa capturando imágenes a múltiples aumentos —típicamente entre $10\times$ y $40\times$ — y las ensambla en una imagen piramidal de alta resolución. El resultado es una imagen de tamaño extremadamente elevado, del orden de varios gigapíxeles, con resoluciones espaciales en torno a

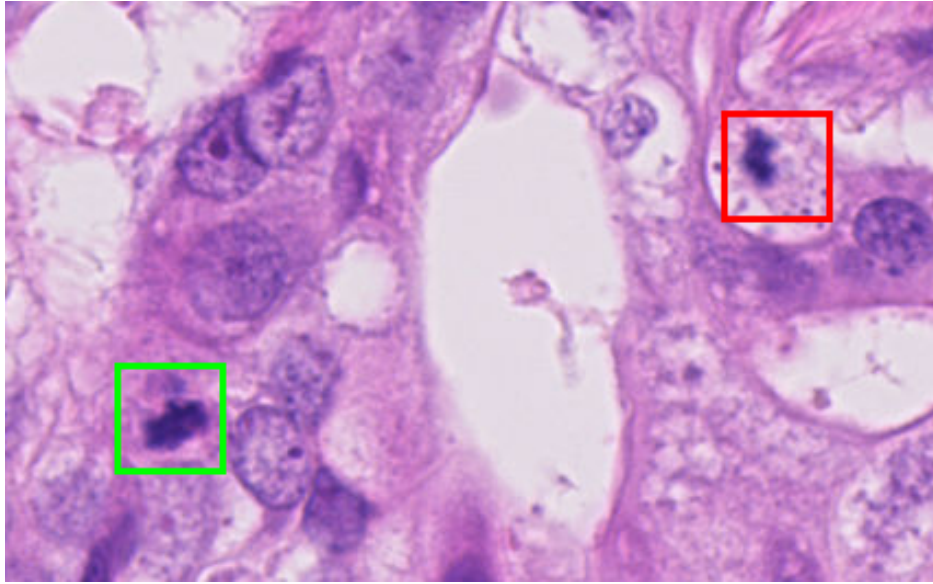


Figura 2.1: Imagen ilustrativa de tejido histopatológico H&E del dataset MIDOG. El recuadro verde señala una figura mitótica y el recuadro rojo una figura no mitótica. A diferencia de los parches empleados en el entrenamiento, esta imagen muestra una región más amplia del tejido con el fin de contextualizar visualmente ambas clases de interés.

0,25–0,50 $\mu\text{m}/\text{píxel}$ a $40\times$ [12]. Las propiedades ópticas del escáner —apertura numérica del objetivo, método de muestreo y esquema de compresión— introducen variaciones de color, contraste y nitidez entre dispositivos y laboratorios que los modelos automáticos deben ser capaces de gestionar [13].

Por este motivo, el procesamiento automático de WSI se realiza habitualmente a nivel de *parche*: se extraen regiones de tamaño fijo que son analizadas de forma independiente, tal y como se ilustra en la figura 2.1 [2]. En el caso de la detección de figuras mitóticas, cada parche contiene típicamente una única célula o un número reducido de éstas, pues los patrones relevantes para la identificación de mitosis no dependen del contexto del tejido circundante. Así, la tarea puede reducirse a una clasificación de parches que indica la presencia o ausencia de una figura mitótica en su interior. Sin embargo, esta clasificación está condicionada a la detección previa de los parches relevantes —aquellos centrados en células en lugar de fondo o tejido conectivo—. Dicho problema de detección queda fuera del ámbito del presente proyecto, y se asumirá que la identificación de parches relevantes viene dada.

Una de las principales dificultades de la tarea de clasificación de figuras mitóticas reside en la elevada variabilidad de su apariencia visual, que depende de la fase del ciclo celular en que se encuentren, del tipo de tejido analizado y de las condiciones de preparación y tinción, así como del dispositivo de adquisición de la imagen WSI y sus parámetros de óptica y muestreo [2]. Además, existen numerosas estructuras celulares que pueden ser confundidas con

figuras mitóticas por su similitud morfológica, como los núcleos hipercromáticos o los artefactos de tinción, lo que dificulta tanto la anotación manual como el desarrollo de métodos automáticos robustos [2].

2.2 Conjuntos de datos para la detección de figuras mitóticas

LA disponibilidad de conjuntos de datos públicos con anotaciones de calidad es un requisito fundamental para el desarrollo y la evaluación de métodos automáticos de detección de figuras mitóticas. Para que un dataset sea adecuado para este problema, debe satisfacer un conjunto de características clave: anotaciones a nivel celular que distingan entre figuras mitóticas y no mitóticas, imágenes procedentes de múltiples escáneres y tipos tumorales para favorecer la generalización entre dominios, y un proceso de anotación con participación de patólogos expertos que garantice la calidad de las etiquetas [2].

A lo largo de los últimos años se han publicado varios conjuntos de datos orientados a esta tarea. A continuación se describen los más relevantes, destacando sus características diferenciales y las razones que justifican la elección realizada en este trabajo.

TUPAC16

El *Tumor Proliferation Assessment Challenge* (TUPAC16) [14] fue organizado en el marco del congreso MICCAI 2016 con el objetivo de evaluar métodos automáticos para la estimación de la proliferación tumoral en cáncer de mama. El dataset proporciona imágenes WSI procedentes de un único tipo tumoral —carcinoma de mama humano— digitalizadas con un único escáner, con anotaciones de figuras mitóticas a nivel de parche. Fue durante varios años una de las referencias principales en el campo, pero la ausencia de diversidad de dominio limita la capacidad de evaluar la robustez de los modelos ante las variaciones habituales en la práctica clínica real.

MITOS_WSI_CMC

El conjunto *MITOS_WSI_CMC* [15] está compuesto por 21 imágenes WSI completas de carcinoma mamario canino, digitalizadas con un único escáner. Contiene más de 13.900 figuras mitóticas anotadas individualmente con coordenadas exactas, junto con más de 36.000 negativos difíciles (*hard negatives*), estructuras de apariencia similar a las mitosis pero que no lo son. El proceso de anotación siguió un protocolo de triple validación entre patólogos con una tasa de desacuerdo del 3%, lo que garantiza una calidad excepcional de las etiquetas. No obstante, al igual que TUPAC16, todas las imágenes provienen de un único escáner y un único tipo tumoral, lo que restringe la variabilidad de dominio presente en el dataset.

MIDOG++

El **Mitosis Domain Generalization Challenge (MIDOG)** [3] constituye el conjunto de datos más grande y diverso disponible para la detección de figuras mitóticas en histología **H&E**. Reúne 11.937 figuras mitóticas anotadas en 503 casos procedentes de **7 tipos tumorales distintos** —3 humanos y 4 caninos—, digitalizados con 5 escáneres diferentes en múltiples laboratorios. Esta diversidad de dominio es precisamente el rasgo diferencial de **MIDOG** frente a los conjuntos anteriores: fue diseñado específicamente para estudiar y mejorar la capacidad de generalización de los modelos ante cambios de escáner, tipo tumoral y condiciones de adquisición. Las anotaciones se realizaron mediante un proceso de tres etapas con consenso entre patólogos, apoyado por detección automática para minimizar omisiones, con una tasa de desacuerdo interobservador del 20%, considerada baja para esta tarea en patología digital.

En el presente trabajo se emplean los parches del dataset **MIDOG** como datos de entrada para el entrenamiento y evaluación de los modelos propuestos. Frente a las alternativas descritas, **MIDOG** ofrece la mayor variabilidad morfológica y de dominio, lo que lo convierte en un contexto más exigente y representativo para evaluar si las representaciones dispersas aprendidas de forma no supervisada son capaces de capturar información discriminativa con independencia del tipo tumoral o del dispositivo de adquisición. Dado el enfoque no supervisado adoptado, las etiquetas de clase no se utilizan durante el entrenamiento, sino únicamente en la fase de análisis del potencial discriminativo de las representaciones aprendidas, tal y como se detalla en el Capítulo 4.

2.3 Métodos de detección mitótica en el estado del arte

LA detección automática de figuras mitóticas en imágenes histopatológicas **H&E** ha sido abordada desde distintas perspectivas a lo largo de los últimos años, evolucionando desde métodos basados en características diseñadas manualmente hasta aproximaciones basadas en aprendizaje profundo y, más recientemente, en aprendizaje no supervisado [2].

2.3.1 Métodos clásicos basados en características manuales

Los primeros métodos automáticos para la detección de figuras mitóticas se basaban en la extracción manual de características morfológicas y de textura: forma del núcleo, intensidad de tinción, descriptores de gradiente o momentos estadísticos de color, combinadas con clasificadores clásicos como máquinas de vectores soporte o bosques aleatorios [2]. Aunque estos métodos mostraron resultados prometedores en condiciones controladas, su rendimiento se degradaba significativamente ante cambios de escáner, protocolo de tinción o tipo tumoral, debido a la escasa capacidad de generalización de las características diseñadas a mano.

2.3.2 Métodos basados en aprendizaje profundo supervisado

Con la irrupción del aprendizaje profundo, la *red neuronal convolucional (CNN)* se convirtió en el paradigma dominante para la detección de figuras mitóticas. El trabajo seminal de Cireşan et al. [16] demostró que una CNN de max-pooling entrenada para clasificar parches centrados en cada píxel era capaz de superar con un margen significativo a todos los métodos competidores en el concurso ICPR 2012, alcanzando un F-score de 0,78. Este resultado estableció el aprendizaje profundo como referencia en el campo y motivó una intensa actividad investigadora en los años siguientes.

Los desarrollos posteriores exploraron distintas formulaciones del problema. Chen et al. [17] propusieron una arquitectura de red neuronal en cascada (*CasNN*) que combina una red de detección de candidatos con una red de verificación, reduciendo drásticamente el coste computacional respecto al enfoque píxel a píxel de Cireşan et al. Li et al. [18] reencuadraron el problema como detección de objetos mediante *DeepMitosis*, un sistema de tres etapas que integra una red de segmentación basada en *Fully Convolutional Network (FCN)* para estimar localizaciones, una red de detección basada en Faster R-CNN y una red de verificación basada en ResNet. Este enfoque multiescala obtuvo resultados destacados tanto en el dataset MITOS 2012 como en el de 2014. Sebai et al. [19] extendieron esta línea con *MaskMitosis*, un framework basado en Mask R-CNN que unifica detección e instanciación en un único modelo entrenado de forma conjunta, abordando además escenarios de supervisión débil y no supervisada.

En la actualidad, los métodos supervisados basados en arquitecturas como ResNet o EfficientNet, preentrenadas en grandes conjuntos de datos y ajustadas sobre anotaciones de patólogos, constituyen el estado del arte en benchmarks de referencia como el challenge *MI-DOG* [3]. Sin embargo, el rendimiento de estos métodos depende fundamentalmente de la disponibilidad de datos etiquetados de calidad: la anotación de figuras mitóticas es un proceso costoso que requiere la intervención de patólogos expertos y presenta una considerable variabilidad entre anotadores, lo que limita la escalabilidad de estos enfoques a nuevos dominios o tipos tumorales [2].

2.3.3 Métodos basados en aprendizaje no supervisado y auto-supervisado

La escasez de datos anotados ha impulsado el interés por los métodos de aprendizaje no supervisado y auto-supervisado, capaces de extraer representaciones útiles sin depender de etiquetas. En el ámbito del aprendizaje contrastivo, métodos como SimCLR [20] o VICReg [21] proponen aprender representaciones invariantes a distintas transformaciones de la imagen mediante aprendizaje contrastivo o de invarianza de varianza-covarianza, sin necesidad de etiquetas. Ciga et al. [22] mostraron que estas representaciones auto-supervisadas, preentre-

nadas sobre grandes colecciones de parches histológicos no etiquetados, pueden ser transferidas eficazmente a tareas de clasificación con un número reducido de muestras anotadas, incluyendo la clasificación de figuras mitóticas.

En este trabajo se adopta un enfoque diferente pero complementario: en lugar de aprender representaciones invariantes a transformaciones, se estudia si las representaciones dispersas aprendidas de forma no supervisada mediante LISTA capturan información discriminativa entre figuras mitóticas y no mitóticas, analizando la estructura del espacio latente en relación con las clases de interés sin haber utilizado las etiquetas durante el entrenamiento.

2.4 Codificación dispersa

LA codificación dispersa (*sparse coding*) es un paradigma de aprendizaje de representaciones que tiene su origen en el estudio del sistema visual biológico. En 1996, Olshausen y Field propusieron que las células simples del córtex visual primario (área V1) pueden interpretarse como un banco de filtros que codifican las imágenes naturales de forma dispersa, es decir, mediante la activación simultánea de un número reducido de neuronas [6]. Esta hipótesis, respaldada posteriormente por experimentos neurofisiológicos, sentó las bases de la codificación dispersa como modelo computacional para el aprendizaje de representaciones [7].

2.4.1 Formulación del problema

Formalmente, dado un vector de entrada $\mathbf{x} \in \mathbb{R}^n$, el objetivo de la codificación dispersa es encontrar un vector de códigos $\mathbf{z} \in \mathbb{R}^m$ tal que $\mathbf{x} \approx \mathbf{W}_d \mathbf{z}$, donde $\mathbf{W}_d \in \mathbb{R}^{n \times m}$ es una matriz de diccionario cuyas columnas son los vectores base o *átomos*, y $\mathbf{z}$ es un vector **disperso**, es decir, con la mayoría de sus componentes iguales a cero y solo un número reducido de ellas con valor no nulo. Esta restricción de dispersión es la que distingue la codificación dispersa de un mero cambio de base: en lugar de representar $\mathbf{x}$ como combinación lineal de todos los átomos del diccionario, se busca una representación que active únicamente los átomos más relevantes para cada entrada concreta.

El caso *sobredimensionado* (*overcomplete*) corresponde a $m > n$, es decir, cuando el diccionario contiene más átomos que la dimensión de la entrada. En un sistema de ecuaciones con más incógnitas que ecuaciones, la solución $\mathbf{z}$ no es única: existen infinitas combinaciones de átomos capaces de reconstruir exactamente $\mathbf{x}$. Es precisamente aquí donde la restricción de dispersión cobra todo su sentido: de entre todas las representaciones posibles, se selecciona la que activa el menor número de átomos, es decir, la más parsimoniosa. Esta elección no es arbitraria —tiene sustento tanto biológico, en la eficiencia del código neural, como computacional, en la obtención de representaciones interpretables donde cada muestra queda descrita por un subconjunto pequeño y significativo de patrones del diccionario [6].

2.4.2 Normas de dispersión: ℓ_0 y ℓ_1

Para formalizar la noción de dispersión se recurre a medidas de la *magnitud* del vector de códigos. La **norma** ℓ_0 de un vector, $\|\mathbf{z}\|_0$, cuenta simplemente el número de componentes no nulas. Minimizarla es, por tanto, la forma más directa de imponer dispersión: se busca la representación que utilice el menor número posible de átomos. Sin embargo, $\|\mathbf{z}\|_0$ es una función discreta y no convexa, lo que hace que el problema de minimización resultante sea NP-difícil en el caso general: no existe ningún algoritmo eficiente que lo resuelva de forma exacta para diccionarios de tamaño arbitrario [7].

Para hacer el problema tratable, se recurre a la **norma** ℓ_1 , $\|\mathbf{z}\|_1 = \sum_i |z_i|$, que es la relajación convexa más ajustada a $\|\mathbf{z}\|_0$. Al ser convexa, admite soluciones eficientes mediante algoritmos de optimización estándar. Y al penalizar la suma de los valores absolutos de los coeficientes, induce dispersión de una forma geoméricamente intuitiva: su región de nivel tiene vértices sobre los ejes coordenados, lo que hace que las soluciones óptimas tiendan a concentrarse en puntos donde la mayoría de las componentes se anulan. Bajo condiciones de coherencia suficientemente baja del diccionario —concretamente, cuando los átomos son suficientemente distintos entre sí—, la solución del problema ℓ_1 coincide con la del problema ℓ_0 , garantizando que la relajación no introduce pérdida de optimalidad [7].

2.4.3 Problema de inferencia

El problema de inferencia, para una entrada $\mathbf{x}$ dada, consiste en encontrar el código disperso óptimo $\mathbf{z}^*$ que minimiza la función de energía que combina el error de reconstrucción cuadrático con una penalización de dispersión mediante la norma ℓ_1 [8]:

$$\mathbf{z}^* = \arg \min_{\mathbf{z}} \frac{1}{2} \|\mathbf{x} - \mathbf{W}_d \mathbf{z}\|_2^2 + \alpha \|\mathbf{z}\|_1 \quad (2.1)$$

donde $\alpha > 0$ es un hiperparámetro que controla el equilibrio entre la calidad de la reconstrucción y el grado de dispersión impuesto sobre el código. Un valor elevado de α penaliza fuertemente los coeficientes no nulos, produciendo representaciones más dispersas pero con mayor error de reconstrucción; un valor reducido permite representaciones más densas y fieles a la entrada.

La figura 2.2 ilustra el esquema general del modelo con un ejemplo concreto aplicado a un parche real de histología H&E del dataset MIDOG. Un parche de célula mitótica (64×64 píxeles) es codificado mediante el modelo entrenado en este trabajo, produciendo un código $\mathbf{z} \in \mathbb{R}^{1024}$ con únicamente 92 componentes activas (9,0% no nulo). En la parte inferior se muestran los seis átomos del diccionario con mayor coeficiente: cada uno captura un patrón visual distinto —textura cromática, gradiente de color, estructura anular— y su combinación ponderada reconstruye fielmente el parche original con un MSE de 0,0571. Este ejemplo ilus-

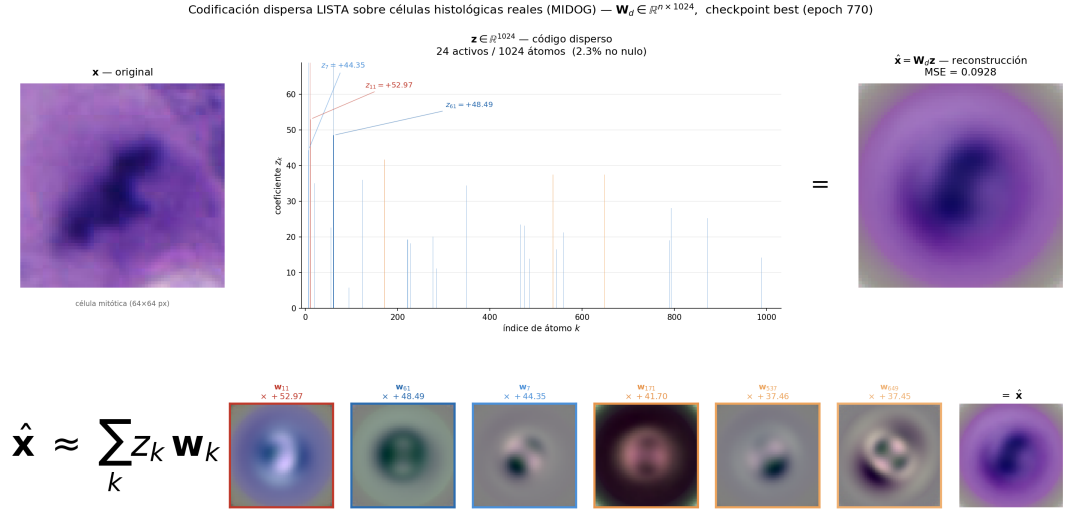


Figura 2.2: Ejemplo de codificación dispersa LISTA sobre un parche de célula mitótica real del dataset MIDOG ($\mathbf{W}_d \in \mathbb{R}^{n \times 1024}$, checkpoint del mejor epoch). Arriba: parche original $\mathbf{x}$ (izquierda), distribución de coeficientes del código disperso $\mathbf{z} \in \mathbb{R}^{1024}$ con 92 activos sobre 1024 átomos (9,0% no nulo) (centro) y reconstrucción $\hat{\mathbf{x}} = \mathbf{W}_d \mathbf{z}$ con MSE = 0,0571 (derecha). Abajo: los seis átomos $\mathbf{w}_k$ del diccionario con coeficiente z_k más alto, mostrando los patrones visuales elementales que combinados reconstruyen el parche.

tra la naturaleza interpretable de las representaciones dispersas: la entrada queda descrita por un pequeño número de patrones elementales del diccionario, cada uno con un rol visual identificable.

2.4.4 Aprendizaje del diccionario

El diccionario $\mathbf{W}_d$ puede ser fijo y analítico —por ejemplo, una base de Gabor o una transformada wavelet— o aprendido a partir de los datos de entrenamiento. En el caso del diccionario aprendido, el proceso de entrenamiento alterna entre dos pasos: la inferencia de los códigos dispersos para cada muestra con el diccionario fijo, y la actualización del diccionario con los códigos fijos. Este proceso iterativo converge a un diccionario adaptado a la estructura estadística de los datos, produciendo típicamente átomos con propiedades similares a los filtros de Gabor cuando se aplica a imágenes naturales [6].

2.5 Algoritmos iterativos de codificación dispersa

LA principal limitación del sparse coding clásico reside en el coste computacional del paso de inferencia: encontrar el código óptimo $\mathbf{z}^*$ para cada nueva muestra requiere re-

solver un problema de optimización iterativo, lo que resulta poco asumible en aplicaciones que requieren procesamiento en tiempo real. Esta limitación motivó el desarrollo de algoritmos iterativos de umbralización más eficientes, entre los que destaca el [Iterative Shrinkage-Thresholding Algorithm \(ISTA\)](#).

2.5.1 ISTA

El algoritmo [ISTA](#) es uno de los métodos iterativos más empleados para resolver el problema de inferencia de la codificación dispersa definido en la ecuación 2.1. La idea fundamental de [ISTA](#) es iterar sucesivamente dos operaciones: un paso de descenso por gradiente sobre el término de reconstrucción, y una operación de umbralización suave sobre el resultado [8].

Esta segunda operación corresponde al *operador proximal* de la penalización ℓ_1 . Dado un funcional convexo g , su operador proximal se define como $\text{prox}_{\lambda g}(\mathbf{v}) = \arg \min_{\mathbf{u}} (g(\mathbf{u}) + \frac{1}{2\lambda} \|\mathbf{u} - \mathbf{v}\|_2^2)$, es decir, el punto que minimiza g sin alejarse demasiado de $\mathbf{v}$. Para $g(\mathbf{z}) = \|\mathbf{z}\|_1$, el operador proximal resulta ser precisamente la umbralización suave componente a componente, lo que dota a [ISTA](#) de una interpretación variacional clara: cada iteración combina un paso de gradiente y una proyección proximal hacia soluciones dispersas.

Formalmente, partiendo de $\mathbf{z}^{(0)} = \mathbf{0}$, [ISTA](#) actualiza el código en cada una de las iteraciones k mediante:

$$\mathbf{z}^{(k+1)} = h_{\theta}(\mathbf{W}_e \mathbf{x} + \mathbf{S} \mathbf{z}^{(k)}) \quad (2.2)$$

donde las matrices $\mathbf{W}_e$ y $\mathbf{S}$ se derivan directamente del diccionario $\mathbf{W}_d$ según:

$$\mathbf{W}_e = \frac{1}{L} \mathbf{W}_d^{\top}, \quad \mathbf{S} = \mathbf{I} - \frac{1}{L} \mathbf{W}_d^{\top} \mathbf{W}_d \quad (2.3)$$

siendo L una cota superior del mayor valor propio de $\mathbf{W}_d^{\top} \mathbf{W}_d$. La función h_{θ} es la umbralización suave componente a componente con umbral $\theta = \alpha/L$, definida como:

$$[h_{\theta}(\mathbf{v})]_i = \text{sign}(v_i) \max(|v_i| - \theta, 0) \quad (2.4)$$

donde v_i es la i -ésima componente de $\mathbf{v}$ y $\theta = \alpha/L$ hereda el hiperparámetro de dispersión α de la ecuación 2.1, escalado por L . Tal y como se ilustra en la figura 2.3, esta función anula las componentes de módulo inferior al umbral θ y contrae las restantes hacia cero en una cantidad θ , actuando así como operador proximal de la norma ℓ_1 e induciendo dispersión en cada iteración.

El algoritmo converge al código óptimo $\mathbf{z}^*$ cuando el número de iteraciones tiende a infinito, aunque en la práctica puede requerir un número elevado de iteraciones para alcanzar una precisión suficiente [8].

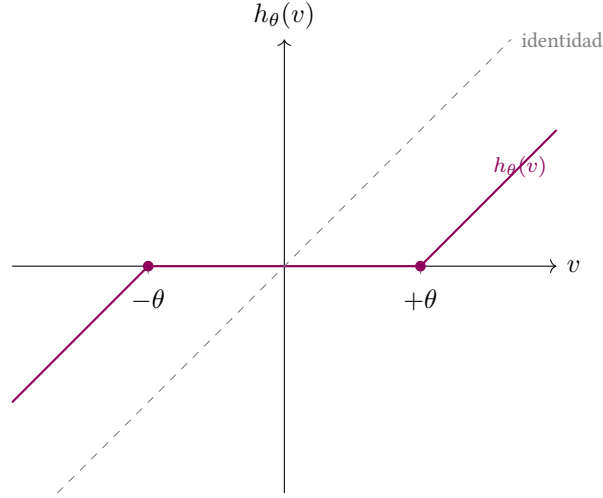


Figura 2.3: Función de umbralización suave h_θ aplicada componente a componente en [ISTA](#). Las componentes de módulo inferior al umbral θ se anulan, mientras que las restantes se contraen hacia cero en una cantidad θ . La línea diagonal punteada representa la función identidad como referencia.

2.5.2 FISTA

[Fast Iterative Shrinkage-Thresholding Algorithm \(FISTA\)](#) es una extensión de [ISTA](#) que mejora su velocidad de convergencia introduciendo un término de momento en la actualización [8]. En lugar de aplicar el paso proximal directamente sobre $\mathbf{z}^{(k)}$, [FISTA](#) mantiene un punto auxiliar $\mathbf{y}^{(k)}$ que combina la iteración actual y la anterior con un peso que crece a lo largo del proceso:

$$\mathbf{z}^{(k+1)} = h_\theta(\mathbf{W}_e \mathbf{x} + \mathbf{S} \mathbf{y}^{(k)}), \quad \mathbf{y}^{(k+1)} = \mathbf{z}^{(k+1)} + \frac{t_k - 1}{t_{k+1}} (\mathbf{z}^{(k+1)} - \mathbf{z}^{(k)}) \quad (2.5)$$

donde la secuencia de pesos t_k sigue la regla $t_1 = 1$, $t_{k+1} = \frac{1 + \sqrt{1 + 4t_k^2}}{2}$. Este mecanismo de momento permite acumular información de iteraciones anteriores y orientar las actualizaciones de forma más eficiente, acelerando la convergencia teórica de $\mathcal{O}(1/k)$ a $\mathcal{O}(1/k^2)$ respecto a [ISTA](#) [8].

2.6 LISTA: Learned ISTA

AUNQUE [FISTA](#) mejora la velocidad de convergencia de [ISTA](#), ambos algoritmos comparten una limitación fundamental: para cada nueva muestra de entrada es necesario ejecutar el proceso iterativo desde cero hasta convergencia, lo que implica un coste computacional

elevado e incompatible con aplicaciones que requieren inferencia en tiempo real. Más aún, el número de iteraciones necesarias para alcanzar una precisión aceptable crece con el tamaño del diccionario, lo que agrava el problema a medida que se aumenta la capacidad del modelo.

Esta limitación motivó el desarrollo de métodos que sustituyen el proceso iterativo por una red neuronal entrenable de profundidad fija, capaz de producir aproximaciones del código disperso en un número reducido y predeterminado de pasos. La ventaja de este enfoque no es únicamente la velocidad de inferencia: al aprender los parámetros específicamente para la distribución de los datos, la red puede explotar la estructura estadística del problema y obtener aproximaciones más precisas que las que [ISTA](#) o [FISTA](#) conseguirían con el mismo número de pasos. El más destacado de estos métodos es [Learned Iterative Shrinkage-Thresholding Algorithm \(LISTA\)](#), descrito a continuación.

2.6.1 Arquitectura

La arquitectura [LISTA](#), propuesta por Gregor y LeCun en 2010 [8], se obtiene desenrollando (*unfolding*) el algoritmo [ISTA](#) un número fijo T de iteraciones y tratando las matrices $\mathbf{W}_e$ y $\mathbf{S}$ como parámetros entrenables en lugar de calcularlas analíticamente a partir del diccionario. Cada iteración del desenrollado tiene la forma:

$$\mathbf{z}^{(k+1)} = h_{\theta}(\mathbf{W}_e \mathbf{x} + \mathbf{S} \mathbf{z}^{(k)}), \quad k = 0, 1, \dots, T-1 \quad (2.6)$$

con $\mathbf{z}^{(0)} = \mathbf{0}$, donde h_{θ} es el operador proximal de [ISTA](#) definido en la ecuación 2.4, o cualquier otra función de activación con efecto equivalente de contracción suave. Desde el punto de vista de la arquitectura de red, cada iteración del desenrollado constituye una capa neuronal: $\mathbf{W}_e \mathbf{x}$ actúa como proyección fija de la entrada compartida por todas las capas, $\mathbf{S} \mathbf{z}^{(k)}$ como conexión recurrente entre capas, y h_{θ} como función de activación no lineal. La arquitectura completa es, por tanto, una red de T capas con activaciones de umbralización, tal y como se ilustra en la figura 2.4.

2.6.2 Entrenamiento

Los parámetros entrenables del modelo son $\mathcal{W} = (\mathbf{W}_e, \mathbf{S}, \theta)$, y se optimizan minimizando una función de pérdida sobre un conjunto de muestras de entrenamiento. En la formulación original de Gregor y LeCun, la función de pérdida es el error cuadrático medio entre el código predicho $\mathbf{z}^{(T)}$ y el código óptimo $\mathbf{z}^*$ obtenido con [ISTA](#) hasta convergencia [8]:

$$\mathcal{L}_{\text{LISTA}} = \frac{1}{N} \sum_{i=1}^N \left\| \mathbf{z}_i^{(T)} - \mathbf{z}_i^* \right\|_2^2 \quad (2.7)$$

El gradiente se propaga hacia atrás a través de las T capas desenrolladas mediante retro-

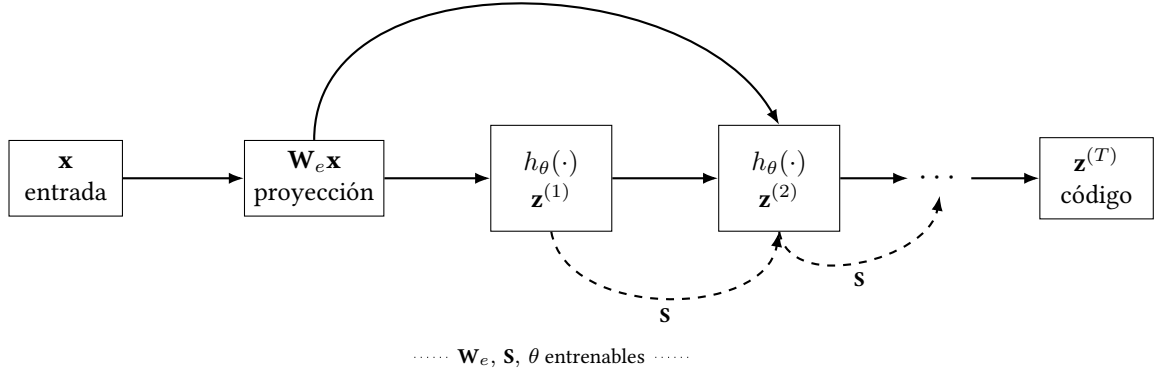


Figura 2.4: Diagrama de la arquitectura **LISTA**. La proyección $W_e x$ es fija para toda la secuencia y se suma en cada capa a la realimentación $Sz^{(k)}$ (flechas discontinuas). Cada bloque $h_\theta(\cdot)$ aplica la umbralización suave y produce el código intermedio $z^{(k)}$. Las matrices W_e , S y el umbral θ se aprenden conjuntamente por retropropagación.

propagación estándar, interpretando la arquitectura como una red recurrente desplegada en el tiempo. Este esquema es posible precisamente porque h_θ es diferenciable en casi todo su dominio, con derivada nula en la región anulada y derivada unitaria fuera de ella.

2.6.3 Ventajas frente a ISTA

La clave de **LISTA** frente a **ISTA** radica en que, al aprender W_e y S específicamente para la distribución de los datos de entrenamiento, el modelo puede explotar correlaciones estadísticas entre muestras que **ISTA** ignora al derivar sus matrices analíticamente del diccionario. Esto permite producir aproximaciones precisas del código disperso con un número de iteraciones muy inferior. Los experimentos originales de Gregor y LeCun demostraron que una sola iteración de **LISTA** produce un error de predicción equivalente al de 18 iteraciones de **FISTA** para diccionarios de 100 átomos, y de 35 iteraciones para diccionarios de 400 átomos [8].

Otra propiedad relevante de **LISTA** es que el encoder resultante es diferenciable tanto con respecto a sus parámetros como con respecto a la entrada, lo que permite integrarlo como componente dentro de sistemas de mayor envergadura entrenados de forma global. Esta propiedad será especialmente relevante en el presente trabajo, donde el encoder **LISTA** se entrena de forma conjunta con un decoder de pesos atados, minimizando directamente el error de reconstrucción sobre los parches de histología **H&E** del dataset **MIDOG**, tal y como se describe en el Capítulo 4.

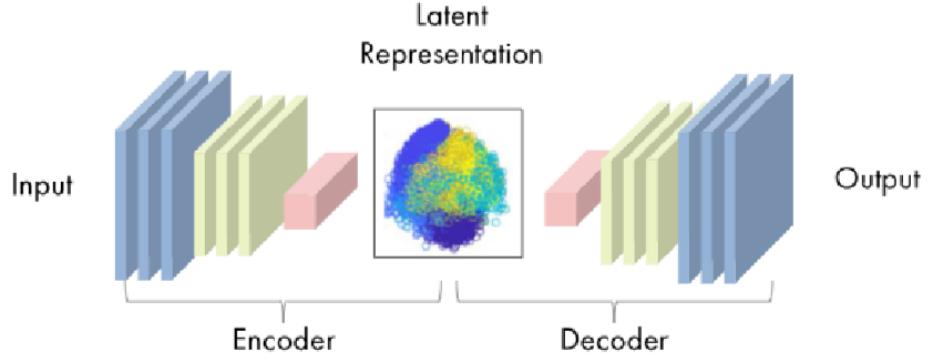


Figura 2.5: Esquema general de un autoencoder. El encoder comprime la entrada en una representación latente de menor dimensión, y el decoder la reconstruye a partir de dicha representación. El entrenamiento se realiza minimizando el error entre la entrada y la salida reconstruida.

2.7 Autoencoders

UN autoencoder es un modelo de aprendizaje auto-supervisado cuyo objetivo es aprender una representación de los datos de entrada en términos de un espacio latente, mediante un proceso de codificación y reconstrucción. El modelo se entrena usando la misma señal (o versiones modificadas de la misma) como entrada y salida objetivo, de forma que debe reconstruirla a partir de la representación latente, sobre la que se ha impuesto algún tipo de restricción adicional para que cumpla determinadas propiedades deseables. La arquitectura consta de dos componentes principales: un encoder $f_\varphi : \mathbb{R}^n \rightarrow \mathbb{R}^m$ que mapea la entrada al espacio latente, y un decoder $g_\psi : \mathbb{R}^m \rightarrow \mathbb{R}^n$ que reconstruye la entrada a partir del código, tal y como se ilustra en la figura 2.5 [5]. El entrenamiento se realiza minimizando el error de reconstrucción sobre un conjunto de muestras:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \ell(\mathbf{x}_i, g_\psi(f_\varphi(\mathbf{x}_i))) \quad (2.8)$$

donde $\ell(\cdot, \cdot)$ es una función de pérdida de reconstrucción, típicamente el error cuadrático medio (MSE) o el error absoluto medio (MAE). Al no requerir etiquetas que requieran intervención humana, los autoencoders pueden entrenarse sobre grandes cantidades de datos no etiquetados, lo que los hace especialmente atractivos en dominios donde la anotación es costosa [4].

La calidad de las representaciones aprendidas depende en gran medida de las restricciones impuestas sobre el espacio latente. Sin ninguna restricción, el modelo podría aprender una función identidad o una transformación trivial del espacio de entrada sin ninguna estructura

útil. Las variantes más relevantes de autoencoders introducen distintos tipos de restricciones para favorecer representaciones con propiedades deseables [5]:

- **Autoencoders de cuello de botella:** reducen la dimensión del espacio latente ($m \ll n$), forzando al modelo a aprender una representación comprimida que capture los factores de variación más relevantes de los datos.
- **Autoencoders dispersos:** penalizan la activación simultánea de muchas componentes del código, favoreciendo representaciones en las que cada muestra se describe mediante un número reducido de elementos activos, permitiendo representar la entrada en términos de combinación de patrones independientes [6].
- **Autoencoders variacionales:** imponen una distribución probabilística sobre el espacio latente (típicamente normal estándar con componentes independientes), lo que además de restringir la dimensionalidad e independencia de variables, permite asegurar que el espacio latente está densamente poblado y regularizado. Esto permite muestrear directamente el espacio latente de acuerdo con su distribución, sin necesidad del encoder, lo que lo hace especialmente útil para la generación de nuevos datos.

Además de las restricciones sobre el espacio latente, existen variantes de autoencoders que explotan las transformaciones entre entradas y salidas como mecanismo complementario. Este es el caso de los **autoencoders denoising** y los **masked autoencoders**: en lugar de reconstruir la entrada a partir de sí misma, el modelo recibe una versión perturbada (con ruido añadido o con regiones enmascaradas) y debe recuperar la señal original. Si la perturbación no es trivial de invertir, el modelo se ve obligado a aprender patrones estructurales de la entrada que le permitan inferir la información perdida, obteniendo representaciones más robustas y generalizables.

Por último, es relevante notar que, en general, las arquitecturas de red neuronal utilizadas para el encoder y el decoder suelen variar libremente. La idea fundamental es que estas redes tienen capacidad para aprender transformaciones no lineales, lo que las dota de una mayor flexibilidad respecto a métodos lineales clásicos como el Análisis de Componentes Principales (PCA) o el Análisis de Componentes Independientes (ICA). Sin embargo, como analogía con dichos algoritmos, es posible encontrar variantes de autoencoders con **pesos atados** (*tied weights*), en los que se comparten los parámetros del encoder y el decoder, de forma que las matrices del decoder son las traspuestas o pseudoinversas de las del encoder. Esta restricción reduce el número de parámetros entrenables y actúa como regularizador implícito, a la vez que establece una correspondencia explícita entre el espacio de entrada y el espacio latente.

Como se habrá observado, la arquitectura LISTA puede interpretarse e implementarse como un autoencoder disperso con pesos atados al que se han añadido restricciones adicionales

sobre la arquitectura del encoder. El encoder se implementa mediante el desenrollado iterativo de [ISTA](#) con parámetros entrenables, produciendo códigos dispersos, mientras que el decoder reconstruye la entrada utilizando la transpuesta de la matriz del encoder como diccionario. Esta restricción adicional dota al modelo de interpretabilidad: el código disperso se expresa en términos de un único diccionario lineal que puede ser explorado y visualizado, permitiendo identificar los patrones visuales que representan cada componente activa. Esto supone una ventaja relevante con respecto a otras arquitecturas de autoencoder de caja negra.

2.8 Discusión

A lo largo de este capítulo se han revisado los fundamentos y el estado del arte en los distintos ámbitos que confluyen en el presente trabajo: el dominio de la histología [H&E](#) y sus conjuntos de datos de referencia, los métodos de codificación dispersa y sus algoritmos de inferencia, la arquitectura [LISTA](#) como aproximación basada en aprendizaje, los autoencoders como marco general de aprendizaje de representaciones, y los métodos de detección mitótica. En esta sección se sintetizan las diferencias concretas del enfoque propuesto respecto al estado del arte en cada uno de estos ámbitos.

Datos. Los métodos supervisados dominantes en la detección de figuras mitóticas se entrenan y evalúan sobre datasets con anotaciones exhaustivas a nivel celular, como TUPAC16 o MITOS_WSI_CMC, que sin embargo presentan escasa variabilidad de dominio. El presente trabajo emplea [MIDOG](#) [3], el conjunto más diverso disponible en términos de tipos tumorales y escáneres, precisamente porque el enfoque no supervisado adoptado no consume las etiquetas durante el entrenamiento. Las etiquetas se reservan únicamente para la evaluación del potencial discriminativo de las representaciones aprendidas, lo que convierte a [MIDOG](#) en el contexto más exigente y representativo para este análisis.

Codificación dispersa e inferencia. El sparse coding clásico [6] y los algoritmos iterativos [ISTA](#) y [FISTA](#) [8] resuelven el problema de inferencia de forma analítica a partir de un diccionario fijo, sin adaptarse a la distribución específica de los datos. Su coste computacional por muestra los hace inviables en escenarios de procesamiento masivo de parches histológicos. El presente trabajo adopta [LISTA](#) [8], que sustituye el proceso iterativo por una red neuronal de profundidad fija cuyos parámetros se aprenden de los datos, obteniendo aproximaciones precisas del código disperso en un número reducido de pasos y con un coste de inferencia constante e independiente del tamaño del diccionario.

Arquitectura del modelo. Los autoencoders empleados habitualmente en histología — autoencoders convolucionales profundos, autoencoders variacionales o modelos basados en

transformers— aprenden representaciones en espacios latentes densos y de alta dimensión, sin restricciones de dispersión explícitas. Su interpretabilidad es limitada: los vectores latentes no tienen una correspondencia directa con patrones visuales identificables. En contraste, el autoencoder disperso con pesos atados basado en [LISTA](#) propuesto en este trabajo produce representaciones en las que cada parche queda descrito por la activación de un número reducido de átomos de un diccionario lineal único y compartido. Esto permite visualizar directamente los patrones que activa cada muestra y cuantificar su relevancia, lo que dota al modelo de una interpretabilidad que los enfoques de caja negra no ofrecen.

Paradigma de aprendizaje. Los métodos supervisados de referencia para la detección de figuras mitóticas [16, 18, 19] optimizan directamente un criterio de clasificación o detección, requiriendo anotaciones de patólogos expertos. Los métodos auto-supervisados más recientes, como SimCLR [20] o VICReg [21], aprenden representaciones invariantes a transformaciones de la imagen sin necesidad de etiquetas, que posteriormente pueden emplearse como punto de partida para tareas de clasificación con supervisión parcial. El presente trabajo adopta un enfoque diferente pero complementario: en lugar de imponer invariancia a transformaciones, se estudia si las representaciones dispersas aprendidas de forma no supervisada mediante [LISTA](#) capturan de forma emergente información discriminativa entre figuras mitóticas y no mitóticas, analizando la estructura del espacio latente en relación a las clases de interés sin haber utilizado las etiquetas durante el entrenamiento.

Función de pérdida de reconstrucción. Ninguno de los trabajos revisados sobre codificación dispersa o autoencoders dispersos en histología aborda de forma sistemática el efecto de la función de pérdida de reconstrucción sobre las propiedades de las representaciones aprendidas. En este trabajo se estudia explícitamente el impacto de tres funciones de pérdida —[MSE](#), [MAE](#) y [SSIM](#), todas ellas aplicadas con una máscara Gaussiana centrada que pondera la región central del parche— sobre la calidad de la reconstrucción y el potencial discriminativo de los códigos dispersos obtenidos. Este análisis constituye una contribución original respecto al estado del arte.

En conjunto, el presente trabajo se distingue del estado del arte por combinar tres elementos que no han sido estudiados de forma conjunta en la literatura: la arquitectura [LISTA](#) como autoencoder disperso interpretable, su aplicación en modo completamente no supervisado sobre parches de histología [H&E](#) con alta variabilidad de dominio, y el análisis sistemático del efecto de la función de pérdida sobre las representaciones aprendidas y su potencial discriminativo para la detección de figuras mitóticas.

Metodología y planificación

EN este capítulo se describe la metodología de desarrollo seguida en el proyecto, junto con la planificación llevada a cabo, los recursos empleados y la estimación de los costes asociados.

3.1 Modelo de desarrollo

Dada la naturaleza exploratoria del proyecto —cuyo objetivo no es únicamente implementar un modelo concreto, sino estudiar el efecto de distintas decisiones de diseño sobre las representaciones aprendidas—, se adoptó un **modelo de desarrollo iterativo e incremental**. Este enfoque permite introducir y evaluar progresivamente nuevas configuraciones experimentales, incorporando los resultados de cada iteración como base para las decisiones de la siguiente.

En la práctica, cada iteración del ciclo de desarrollo comprende las siguientes fases: revisión del estado actual del modelo y los resultados obtenidos, definición de la siguiente configuración o modificación a estudiar, implementación y entrenamiento, evaluación cuantitativa y cualitativa, y extracción de conclusiones que orientan la siguiente iteración. Este ciclo se repitió a lo largo de todo el desarrollo del proyecto, desde las primeras pruebas de concepto hasta la configuración experimental final.

3.2 Planificación

El trabajo se organizó en las siguientes etapas, diseñadas para cubrir de forma progresiva todos los aspectos del desarrollo:

- **Etapla 1: Estudio bibliográfico.** Revisión de la literatura relevante para adquirir los fundamentos necesarios sobre codificación dispersa, los algoritmos [ISTA](#) y [FISTA](#), la arquitectura [LISTA](#), los autoencoders y los métodos de detección de figuras mitóticas en

histología [H&E](#). Esta etapa proporcionó la base teórica necesaria para tomar decisiones de diseño fundamentadas a lo largo del proyecto.

- **Etapa 2: Preparación del dataset.** Descarga, inspección y preprocesado del dataset [MIDOG](#). En esta etapa se realizaron estudios iniciales de distribución de clases, visualización de parches representativos de cada clase y verificación de la integridad de los datos, empleando cuadernos Jupyter para facilitar la exploración interactiva.
- **Etapa 3: Implementación del modelo base.** Diseño e implementación del autoencoder disperso basado en [LISTA](#), incluyendo el encoder con desenrollado iterativo, el decoder con pesos atados y la infraestructura de entrenamiento. En esta fase se realizaron las primeras pruebas de funcionamiento con un subconjunto reducido de datos en el ordenador personal, verificando la correcta propagación de gradientes y la convergencia del modelo.
- **Etapa 4: Implementación de las funciones de pérdida.** Implementación de las tres funciones de pérdida de reconstrucción estudiadas en este trabajo —[MSE](#), [MAE](#) y [SSIM](#)— con máscara Gaussiana centrada, así como de las métricas de evaluación cuantitativa y del sistema de gestión de checkpoints.
- **Etapa 5: Experimentación a escala completa.** Entrenamiento de las seis configuraciones experimentales definidas (tres funciones de pérdida $\times$ dos dimensiones del diccionario) sobre el conjunto completo del dataset [MIDOG](#), empleando la máquina del grupo de investigación [VARPA](#) para los experimentos de mayor coste computacional.
- **Etapa 6: Análisis de resultados.** Evaluación cuantitativa de los modelos entrenados mediante las métricas de reconstrucción, dispersión y coherencia del diccionario. Análisis del potencial discriminativo de las representaciones aprendidas mediante el índice de Fisher, el estadístico F de ANOVA y la información mutua. Visualización de los átomos del diccionario y de las representaciones obtenidas.
- **Etapa 7: Redacción de la memoria.** Documentación de los avances, decisiones de diseño y resultados obtenidos a lo largo del proyecto. La redacción de la memoria comenzó en abril de 2026, una vez consolidados los resultados experimentales, y se llevó a cabo en $\text{\LaTeX}$ mediante Overleaf.

El proyecto se inició en septiembre de 2025 y la entrega tuvo lugar a finales de agosto de 2026, lo que supuso una duración total de 52 semanas. El desarrollo se compaginó con otras actividades académicas, por lo que la dedicación no fue a tiempo completo. Las reuniones de seguimiento con los tutores se celebraron con una periodicidad aproximada de tres semanas hasta mayo de 2026, sin nuevas reuniones a partir de junio, una vez finalizada la fase de

experimentación. Cada reunión tuvo una duración de aproximadamente una hora y media. Adicionalmente, el contacto con los tutores fue continuo a través de mensajes en Microsoft Teams durante ese mismo periodo, lo que permitió resolver dudas y tomar decisiones de forma ágil entre reuniones. La figura 3.1 muestra el diagrama de Gantt con el desglose de tareas y su distribución a lo largo de las 52 semanas del proyecto.

3.3 Seguimiento y desviaciones de ejecución no previstas

Si bien la planificación por etapas descrita en la sección anterior se siguió en líneas generales, el carácter exploratorio del proyecto (sección 3.1) introdujo determinadas desviaciones respecto al calendario inicialmente previsto, derivadas principalmente de hallazgos surgidos durante la propia experimentación.

La desviación más significativa se produjo en la Etapa 5 (Experimentación a escala completa) y su solapamiento con la Etapa 6 (Análisis de resultados): el colapso del diccionario observado en las configuraciones $m = 1024$ bajo MSE y MAE (Capítulo 5) no se detectó hasta avanzado el entrenamiento de dichas configuraciones, lo que obligó a reanalizar los registros de entrenamiento ya generados, replantear el protocolo de comparación de checkpoints (checkpoint óptimo frente a checkpoint final frente a la evolución completa) y ejecutar análisis adicionales no contemplados originalmente, como la caracterización de los episodios de recuperación espontánea recogida en el Anexo A. Esto retrasó el cierre definitivo de la Etapa 6 respecto a lo planificado inicialmente.

De forma similar, durante la Etapa 3 (Implementación del modelo base) se realizaron pruebas preliminares con un número de iteraciones de desenrollado $T = 10$ que resultaron en representaciones de peor calidad y un coste computacional mayor de lo esperado (sección 4.3), lo que obligó a una iteración adicional de diseño no prevista inicialmente antes de fijar $T = 3$ como valor definitivo para el resto del proyecto.

Como consecuencia de estos retrasos acumulados en las etapas de experimentación y análisis, y con el fin de no comprometer la fecha de entrega, la redacción de la memoria (Etapa 7) se inició en abril de 2026 con la fase de experimentación aún en curso, en lugar de tras su finalización completa como estaba previsto originalmente. En conjunto, la duración total del proyecto (52 semanas) se mantuvo dentro del margen previsto, si bien la distribución interna del tiempo entre experimentación, análisis y redacción se ajustó de forma dinámica conforme surgían estos hallazgos.

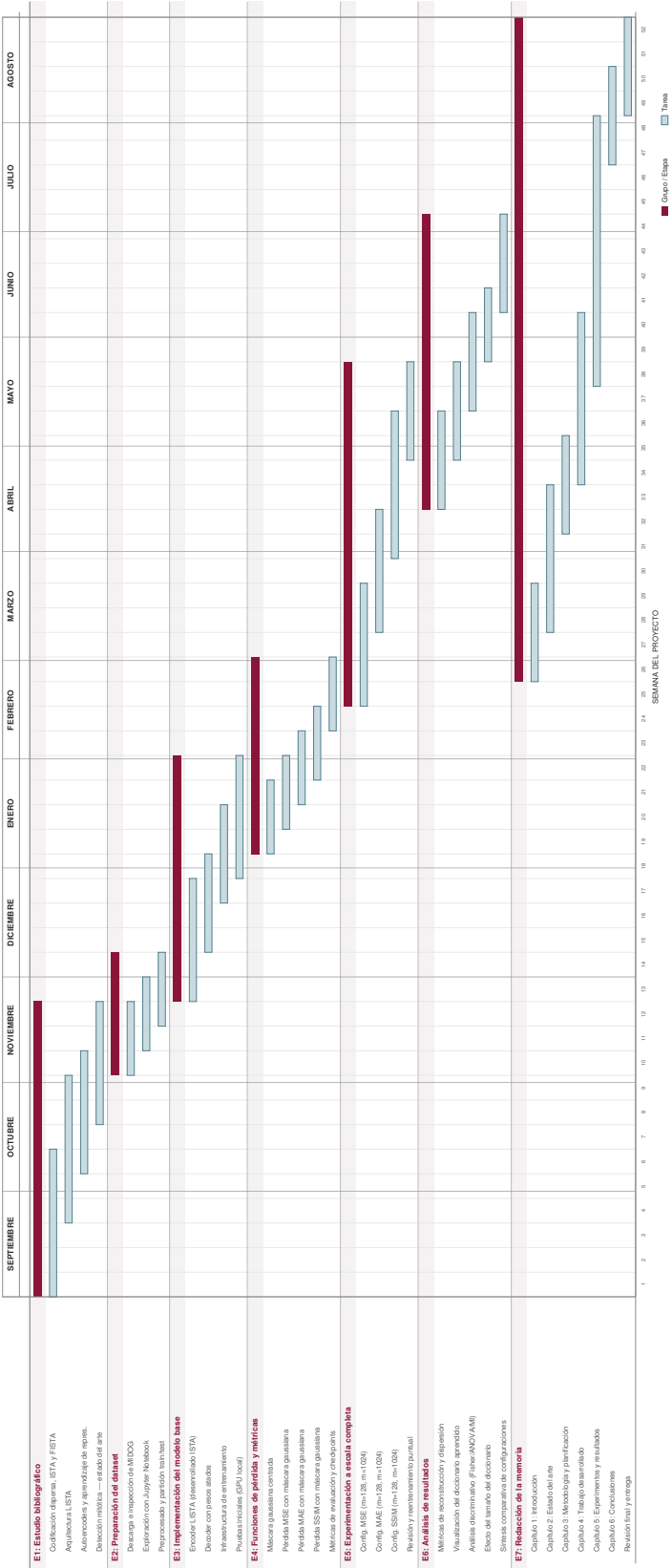


Figura 3.1: Diagrama de Gantt con la planificación del proyecto (semanas 1–52, septiembre 2025 – agosto 2026). Los grupos en rosa corresponden a las etapas principales y las barras en azul a las tareas individuales. La redacción de la memoria (Etapa 7) se inició en abril de 2026, una vez avanzada la fase de experimentación.

3.4 Recursos

3.4.1 Recursos materiales

Software

- **Python 3.10**: lenguaje de programación principal empleado en todas las fases de implementación, entrenamiento y análisis. Las librerías más relevantes utilizadas son:
 - *PyTorch* (`torch`>=2.0.0, `torchvision`>=0.15.0): framework principal para la implementación y entrenamiento del modelo [LISTA](#).
 - *NumPy* (`numpy`>=1.21.0): operaciones matriciales y preprocesado de datos.
 - *Pillow* (`Pillow`>=9.0.0): lectura y manipulación de imágenes.
 - *Matplotlib* (`matplotlib`>=3.5.0): visualización de resultados, métricas de entrenamiento y representaciones aprendidas.
- **Jupyter Notebook**: empleado en las etapas iniciales del proyecto para la exploración interactiva del dataset, la visualización de parches y el estudio de la distribución de clases.
- **Visual Studio Code** con extensión *Remote SSH*: entorno de desarrollo empleado tanto para el desarrollo local como para la conexión y edición remota en el servidor del grupo de investigación.
- **Overleaf**: plataforma en línea para la redacción de la memoria en $\text{\LaTeX}$.
- **Microsoft Teams**: herramienta de comunicación empleada para el seguimiento continuo del proyecto con los tutores entre reuniones.

Hardware

- **Ordenador personal** (MSI Cyborg 15 A13VE): empleado para el desarrollo del código, las pruebas iniciales de funcionamiento con subconjuntos reducidos de datos y la redacción de la memoria.
 - CPU: Intel Core i7-13620H (2,40 GHz)
 - RAM: 16 GB
 - GPU: NVIDIA GeForce RTX 4050 Laptop (6 GB VRAM)
 - Sistema operativo: Windows 11 (64 bits)
 - Almacenamiento: SSD 954 GB

- **Servidor Nidavellir** (grupo [VARPA](#), Universidade da Coruña): máquina empleada para el entrenamiento de todos los experimentos a escala completa, accesible mediante SSH a través de un servidor de paso con VPN de la UDC.
 - GPU: NVIDIA GeForce RTX 2080 Ti (11 GB VRAM)
 - Sistema operativo: Linux (Ubuntu)

3.4.2 Recursos humanos

Los roles del proyecto incluyen los de programador, analista y jefe de proyecto. El alumno, como desarrollador principal y dado el carácter individual del Trabajo de Fin de Grado, asumió los roles de programador y analista. Los **dos directores** asumieron el rol de jefes de proyecto, manteniendo reuniones periódicas de seguimiento y contacto continuo a través de mensajes en Teams para la resolución ágil de dudas y la toma de decisiones entre reuniones, hasta mayo de 2026.

3.5 Estimación de costes

3.5.1 Costes de recursos materiales

Los costes de hardware y software han sido nulos en términos de adquisición específica para el proyecto. El ordenador personal ya estaba disponible con anterioridad al inicio del trabajo. El servidor Nidavellir fue cedido por el grupo de investigación [VARPA](#) sin coste adicional. Todo el software empleado es de código abierto o gratuito (Python, PyTorch, Jupyter, Visual Studio Code, Overleaf en su versión gratuita).

3.5.2 Costes de recursos humanos

Para la estimación de costes humanos se consideran los roles y dedicaciones descritos en la sección anterior. La duración del proyecto fue de 52 semanas. El alumno mantuvo una dedicación parcial de aproximadamente 2–3 horas diarias (2,5 h/día de media) durante 5 días laborales a la semana, a lo largo de todo el proyecto. Los **dos directores**, en cambio, solo participaron activamente hasta mayo de 2026 (semana 39 del proyecto): a partir de junio no se celebró ninguna reunión de seguimiento adicional, al haber concluido la fase de experimentación y estar la redacción de la memoria suficientemente encauzada. Hasta esa fecha, dedicaron, además de las reuniones presenciales de seguimiento (aproximadamente una hora y media cada tres semanas), un tiempo adicional estimado en 2 horas semanales para el seguimiento por mensajes, revisión de avances y preparación de las reuniones.

- **Alumno:** $52 \text{ semanas} \times 5 \text{ días} \times 2,5 \text{ h/día} = 650 \text{ h.}$

Tabla 3.1: Estimación de costes de recursos humanos del proyecto.

Rol	N.º	Coste (€/h)	Horas totales	Coste total (€)
Director de proyecto	2	37	195	7.215
Analista/Programador	1	25	650	16.250
Total:				23.465

- **Directores** (2), solo hasta mayo de 2026 (39 semanas): reuniones presenciales: ≈ 13 reuniones $\times 1,5 \text{ h} \times 2$ directores = 39 h; seguimiento continuo: 39 semanas $\times 2 \text{ h/semana} \times 2$ directores = 156 h. Total: 195 h.

Aplicando las tarifas habituales para perfiles de ingeniería en España e incluyendo una estimación del 24% de coste adicional por cotizaciones a la Seguridad Social:

- Alumno (analista/programador): $20 \text{ €/h} \times 1,24 \approx 25 \text{ €/h}$.
- Directores (jefes de proyecto): $30 \text{ €/h} \times 1,24 \approx 37 \text{ €/h}$.

La tabla 3.1 muestra un resumen de los costes asociados al proyecto.

Trabajo desarrollado

ESTE capítulo describe en detalle el trabajo desarrollado para abordar los objetivos planteados en la Introducción: el diseño de un autoencoder disperso basado en la arquitectura [LISTA](#), capaz de aprender representaciones dispersas de parches de histología [H&E](#) de forma no supervisada, y la evaluación de dichas representaciones tanto en términos de calidad de reconstrucción como de potencial discriminativo entre figuras mitóticas y no mitóticas.

El capítulo se organiza siguiendo el orden natural en que se construyó y ejecutó el trabajo. Tras una visión general de la metodología (sección [4.1](#)), se describe el dataset empleado y su preprocesado (sección [4.2](#)), la arquitectura del modelo (sección [4.3](#)), las funciones de pérdida de reconstrucción comparadas en este trabajo (sección [4.4](#)) y la máscara Gaussiana centrada empleada como ponderación espacial común a varias de las secciones anteriores (sección [4.5](#)). El capítulo concluye con la estrategia de entrenamiento y las configuraciones experimentales consideradas (sección [4.6](#)) y las métricas empleadas para evaluar los modelos resultantes (sección [4.7](#)), cuyos resultados se presentan y discuten en el Capítulo 5.

4.1 Visión general de la metodología

ANTES de entrar en el detalle de cada uno de los componentes del sistema, resulta útil presentar una visión general de la metodología seguida en este trabajo, que se organiza en dos fases claramente diferenciadas: una fase de **entrenamiento**, en la que el autoencoder disperso aprende de forma no supervisada a reconstruir los parches de histología a partir de un código disperso; y una fase de **evaluación y análisis**, en la que dicho código se emplea para caracterizar cuantitativamente el modelo y para estudiar su potencial discriminativo entre figuras mitóticas y no mitóticas.

La figura [4.1](#) resume ambas fases. Durante el entrenamiento (parte superior), los parches del dataset [MIDOG](#) se preprocesan mediante estandarización y aumentación de datos, y se emplean para optimizar conjuntamente el encoder y el decoder [LISTA](#) minimizando una función

de pérdida compuesta por un término de reconstrucción y un término de dispersión (sección 4.4). Este proceso se repite durante un número fijo de épocas, guardando periódicamente los pesos del modelo en forma de checkpoints (sección 4.6).

Una vez entrenado, el modelo se emplea en la fase de evaluación (parte inferior) para codificar los parches del conjunto de test —esta vez sin aumentación— obteniendo para cada uno un código disperso $\mathbf{z}$. A partir de estos códigos se calculan tres tipos de análisis complementarios: (i) métricas cuantitativas de reconstrucción, dispersión y coherencia del diccionario (sección 4.7); (ii) un análisis del potencial discriminativo de los átomos respecto a las clases mitótica y no mitótica; y (iii) visualizaciones cualitativas del diccionario aprendido y de las reconstrucciones obtenidas. El Capítulo 5 desarrolla en detalle los resultados obtenidos en cada uno de estos análisis.

Esta organización refleja el carácter no supervisado del enfoque propuesto: el modelo nunca tiene acceso a las etiquetas de clase durante el entrenamiento; estas se emplean únicamente a posteriori, durante la fase de evaluación, para analizar si la estructura aprendida de forma no supervisada resulta discriminativa.

En las siguientes secciones se describen en detalle cada uno de los componentes introducidos en este esquema, comenzando por el dataset y su preprocesado.

4.2 Dataset y preprocesado de parches

EL dataset empleado en este trabajo es MIDOG 2022 [3], un conjunto de datos público de referencia para la detección de figuras mitóticas en histología H&E. MIDOG proporciona imágenes de lámina completa (WSI) procedentes de escáneres y tejidos tumorales de distinta naturaleza, junto con las posiciones anotadas de las figuras mitóticas presentes en cada imagen, con el objetivo de evaluar la capacidad de generalización de los modelos ante cambios de dominio. A partir de estas anotaciones se extrae, para cada posición etiquetada, un parche centrado en la célula de interés, clasificado como perteneciente a una de dos clases: figura mitótica (*mitotic figure*) o figura no mitótica (*non-mitotic figure*).

Los parches extraídos se almacenan como imágenes RGB de tamaño fijo de 64×64 píxeles, resolución suficiente para capturar los patrones morfológicos relevantes de las figuras mitóticas manteniendo una dimensión de entrada manejable para el modelo LISTA, cuyo vector de entrada tiene dimensión $n = 64 \times 64 \times 3 = 12288$.

4.2.1 Estandarización

Antes de ser procesados por el modelo, los parches se someten a una estandarización Z-score global. Para ello, se calculan una única media μ y una única desviación típica σ (escalares, compartidos por los tres canales RGB) sobre el conjunto completo de píxeles y canales de todos

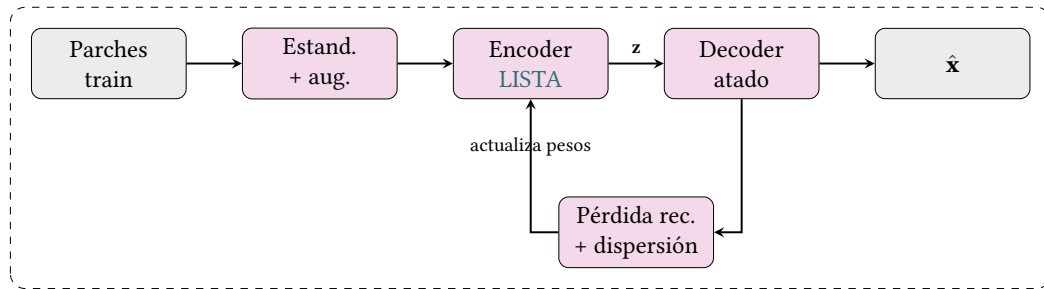
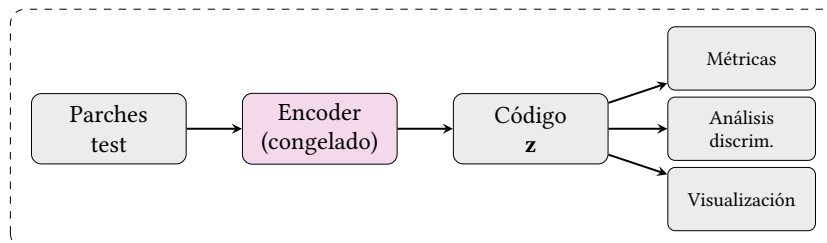
Entrenamiento (no supervisado)**Evaluación / uso**

Figura 4.1: Visión general de la metodología. **Arriba:** fase de entrenamiento no supervisado, en la que el encoder y el decoder **LISTA** se optimizan conjuntamente minimizando la pérdida de reconstrucción y dispersión sobre los parches de entrenamiento aumentados. **Abajo:** fase de evaluación, en la que el encoder ya entrenado se emplea para codificar los parches de test y obtener los códigos dispersos sobre los que se calculan las métricas, el análisis discriminativo y las visualizaciones presentadas en el Capítulo 5.

los parches del dataset:

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu}{\sigma} \quad (4.1)$$

A diferencia de una estandarización por canal, esta estandarización global preserva las relaciones de intensidad relativa entre canales, relevante en histología [H&E](#), donde el balance de color entre canales codifica información sobre la tinción. Este preprocesado centra la distribución de píxel en torno a cero y normaliza su escala, facilitando la convergencia del optimizador y asegurando que las distintas funciones de pérdida operen en un rango comparable. Los valores de μ y σ se calculan una única vez sobre el conjunto de entrenamiento y se reutilizan en la evaluación, evitando que se filtre información del test en el preprocesado.

4.2.2 Partición train/test

El dataset se divide en dos subconjuntos disjuntos: un conjunto de entrenamiento con el 80% de los parches y un conjunto de test con el 20% restante. La partición se realiza de forma aleatoria, sin estratificar por clase, ya que el entrenamiento del autoencoder es no supervisado y no emplea las etiquetas en ningún momento; dichas etiquetas se reservan para la fase de evaluación y análisis discriminativo del Capítulo 5. La partición se almacena en un fichero `split.json` para garantizar la reproducibilidad; todos los modelos comparten la misma partición.

4.2.3 Aumentación de datos

Durante el entrenamiento se aplica augmentación de datos para incrementar la variabilidad de las muestras y mejorar la generalización del modelo. Las transformaciones se aplican sobre el rango de píxel original $[0, 255]$, antes de la estandarización Z-score:

- Volteo horizontal aleatorio con probabilidad $p = 0.5$.
- Volteo vertical aleatorio con probabilidad $p = 0.5$.
- Rotación aleatoria de hasta $\pm 180^\circ$, rellenando las esquinas con un valor de gris uniforme igual a la media global μ del dataset, evitando así píxeles negros o blancos extremos en las esquinas.
- Jitter de brillo y contraste con factor ± 0.4 , simulando variaciones de iluminación y tinción entre escáneres.
- Desenfoque Gaussiano con probabilidad $p = 0.5$ y sigma aleatorio en el rango $[0.1, 2.0]$, simulando variaciones de enfoque del escáner.

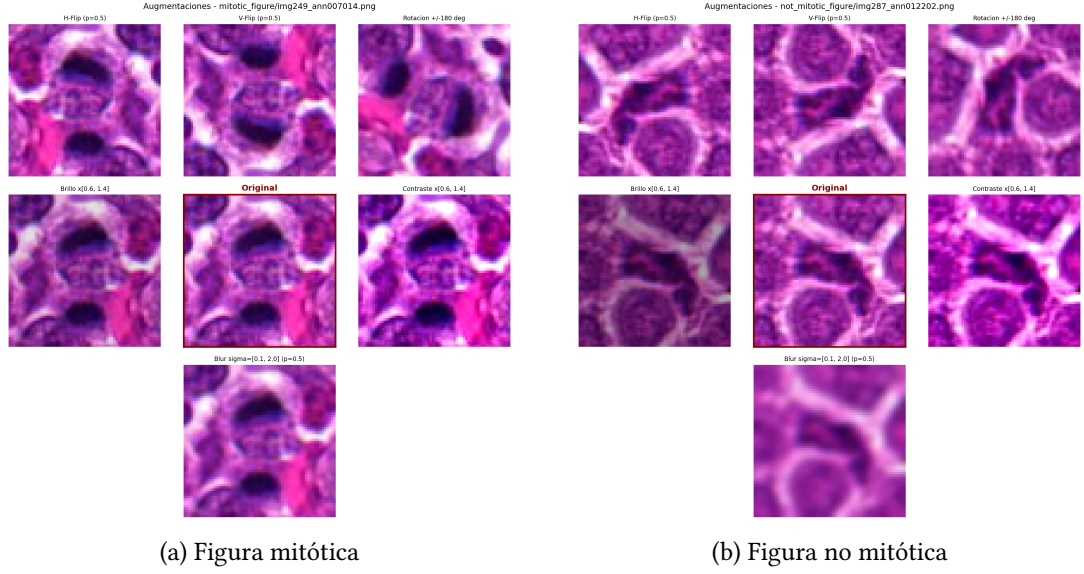


Figura 4.2: Efecto de las transformaciones de aumentación de datos sobre parches del dataset MIDOG. El parche original se muestra en el centro con recuadro rojo. Las restantes subimágenes muestran el resultado de aplicar cada transformación de forma individual.

Estas transformaciones son especialmente adecuadas para histología H&E, ya que las células no presentan orientación preferida y la tinción varía habitualmente entre escáneres y centros clínicos. Las figuras 4.2a y 4.2b muestran el efecto de cada transformación sobre un parche representativo de cada clase. La aumentación se aplica únicamente durante el entrenamiento; en evaluación se usan siempre los parches originales sin transformar.

4.3 Arquitectura del modelo: encoder y decoder LISTA

EL modelo propuesto en este trabajo es un autoencoder disperso basado en la arquitectura LISTA, compuesto por un encoder que produce representaciones dispersas y un decoder con pesos atados que reconstruye la entrada a partir del código obtenido.

4.3.1 Encoder LISTA

El encoder implementa el desenrollado de ISTA con un número fijo de iteraciones T , con parámetros entrenables. Dado un parche de entrada $\mathbf{x} \in \mathbb{R}^n$ con $n = 12288$, el encoder calcula en primer lugar la proyección fija:

$$\mathbf{B} = \mathbf{W}_e \mathbf{x} \quad (4.2)$$

donde $\mathbf{W}_e \in \mathbb{R}^{m \times n}$ es la matriz de proyección, sin término de sesgo propio, fija durante

todas las iteraciones. El código se inicializa directamente con esta proyección, sin no linealidad:

$$\mathbf{z}^{(0)} = \mathbf{B} \quad (4.3)$$

y se refina iterativamente durante $T - 1$ pasos adicionales:

$$\mathbf{z}^{(k+1)} = \text{ReLU}\left(\mathbf{B} + \mathbf{S} \mathbf{z}^{(k)} + \mathbf{b}_S\right), \quad k = 0, 1, \dots, T - 2 \quad (4.4)$$

donde $\mathbf{S} \in \mathbb{R}^{m \times m}$ es la matriz de realimentación que implementa el *explaining away*, y $\mathbf{b}_S \in \mathbb{R}^m$ es su sesgo asociado. Ambas se inicializan a cero, de modo que la primera iteración se reduce a $\mathbf{z}^{(1)} = \text{ReLU}(\mathbf{B})$, lo que facilita la convergencia inicial. Esta inicialización se refuerza mediante un periodo de calentamiento (sección 4.6.3) durante el cual $\mathbf{S}$ y $\mathbf{b}_S$ se mantienen forzados a cero; transcurrido dicho periodo, la competición entre átomos se aprende progresivamente. El código final $\mathbf{z} = \mathbf{z}^{(T-1)} \in \mathbb{R}^m$ constituye la representación dispersa del parche.

Esta formulación reparametriza el umbral entrenable θ del ISTA clásico (ec. 2.2) como el sesgo $\mathbf{b}_S$ de la capa de realimentación. Descomponiendo la ecuación 4.4 componente a componente para cada átomo j :

$$z_j^{(k+1)} = \text{ReLU}\left((\mathbf{B} + \mathbf{S} \mathbf{z}^{(k)})_j + b_{S,j}\right) \quad (4.5)$$

$-b_{S,j}$ actúa como un umbral por átomo: la componente j solo se activa cuando su pre-activación supera dicho umbral. Esto reproduce la función del *soft-thresholding* h_θ del ISTA clásico, con dos diferencias: (i) $b_{S,j}$ se aprende implícitamente como sesgo de $\mathbf{S}$, y (ii) la ReLU, a diferencia de h_θ , no resta el umbral al valor superviviente —se pierde el efecto de *shrinkage* sobre los coeficientes activos, aunque se conserva la umbralización que produce dispersión—. A cambio, se garantiza que todos los coeficientes sean no negativos, facilitando su interpretación como niveles de activación.

Los parámetros entrenables del encoder son $\mathbf{W}_e$, $\mathbf{S}$ y $\mathbf{b}_S$, con un total de $m \cdot n + m^2 + m$ parámetros; para $m = 128$ y $n = 12288$, esto supone aproximadamente 1.59 millones de parámetros.

4.3.2 Decoder con pesos atados

El decoder reconstruye el parche original a partir del código disperso $\mathbf{z}$ utilizando la transpuesta de la matriz del encoder como diccionario (*tied weights*):

$$\hat{\mathbf{x}} = \mathbf{W}_e^\top \mathbf{z} \quad (4.6)$$

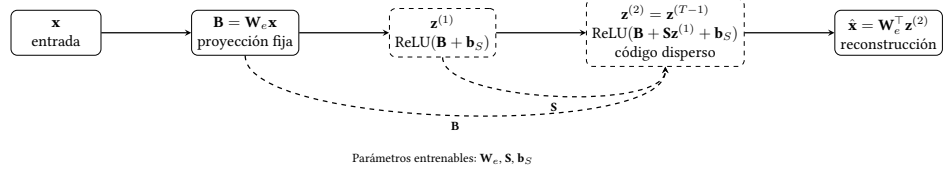


Figura 4.3: Arquitectura del autoencoder disperso basado en LISTA, particularizada para $T = 3$ iteraciones (valor empleado en todos los experimentos de este trabajo). La proyección $\mathbf{B} = \mathbf{W}_e \mathbf{x}$ es fija para las $T - 1 = 2$ iteraciones de refinamiento. Cada bloque aplica ReLU sobre la suma de $\mathbf{B}$, la realimentación $\mathbf{S} \mathbf{z}^{(k)}$ y el sesgo entrenable $\mathbf{b}_S$, que actúa como umbral por átomo. El decoder reconstruye la entrada usando la transpuesta $\mathbf{W}_e^\top$ como diccionario (pesos atados).

Esta elección aporta dos ventajas: reduce a la mitad el número de parámetros del decoder, actuando como regularizador implícito, y fuerza a que las columnas de $\mathbf{W}_e^\top$ actúen simultáneamente como filtros de análisis en el encoder y como átomos del diccionario de síntesis en el decoder. El decoder no incluye sesgo, por lo que la reconstrucción está centrada en torno al origen del espacio de entrada estandarizado.

4.3.3 Configuración del número de iteraciones

El número de iteraciones de desenrollado T controla el equilibrio entre calidad de las representaciones, estabilidad del entrenamiento y coste computacional. En experimentos preliminares se observó que valores altos de T (por ejemplo, $T = 10$) resultaban demasiado agresivos: el código se refinaba en exceso a través de correcciones sucesivas entre átomos antes de evaluar la pérdida, degradando la calidad de las representaciones e incrementando notablemente el coste computacional por paso. En todos los experimentos se emplea en su lugar $T = 3$ iteraciones, valor que ofrece un desenrollado suficiente evitando el sobre-refinamiento observado con valores mayores. La figura 4.3 resume el flujo completo, desde $\mathbf{x}$ hasta $\hat{\mathbf{x}}$, mostrando las $T - 1 = 2$ iteraciones de refinamiento empleadas en este trabajo.

4.4 Funciones de pérdida de reconstrucción

UNA de las contribuciones principales de este trabajo es el estudio del efecto de distintas funciones de pérdida de reconstrucción sobre las representaciones aprendidas por el modelo LISTA. La elección de la función de pérdida no es neutra: determina qué tipo de errores penaliza más el modelo y, en consecuencia, qué propiedades adquiere el diccionario y el código resultante. Una función que penalice fuertemente los errores grandes tenderá a evitar desviaciones puntuales, a costa de tolerar peor un error pequeño pero extendido; una función con estructura local, en cambio, puede favorecer que el diccionario capture patrones

morfológicos coherentes en lugar de minimizar diferencias píxel a píxel. Este trabajo compara tres funciones con estas propiedades distintas —MSE, MAE y SSIM—, tal y como se detalla en el Capítulo 5.

La función de pérdida total combina un término de reconstrucción $\mathcal{L}_{rec}$ con un término de dispersión $\mathcal{L}_{sparse}$, que penaliza la magnitud del código:

$$\mathcal{L} = \lambda_{rec} \cdot \mathcal{L}_{rec} + \lambda_{L1} \cdot \mathcal{L}_{sparse} \quad (4.7)$$

donde λ_{rec} y λ_{L1} controlan el peso relativo de cada término; su elección concreta se describe en la sección 4.6. El entrenamiento optimiza esta pérdida mediante descenso de gradiente estocástico (SGD), promediando sobre mini-batches en cada paso, práctica habitual que se omite de las ecuaciones que siguen.

El término de dispersión penaliza la norma ℓ_1 del código:

$$\mathcal{L}_{sparse} = \|\mathbf{z}\|_1 \quad (4.8)$$

Esta penalización actúa como regularización adicional a la ReLU del encoder: mientras la ReLU solo garantiza coeficientes no negativos, la norma ℓ_1 empuja hacia cero los que no contribuyen a reducir el error. En todos los experimentos se emplea $\lambda_{rec} = 1.0$; λ_{L1} varía según la configuración —en particular, según el tamaño del diccionario—, con el criterio de ajuste automático de la sección 4.6.

4.4.1 Formulación general: ponderación espacial de la diferencia punto a punto

Las tres funciones de pérdida comparten una estructura general en dos etapas. Primero se calcula un **mapa de error** $d(\mathbf{x}, \hat{\mathbf{x}})$ que compara original y reconstrucción de forma local: punto a punto en MSE/MAE, o mediante ventanas locales en SSIM. Después, dicho mapa se agrega mediante una **media espacial ponderada** en lugar de un promedio uniforme, usando la máscara Gaussiana centrada $\tilde{\mathbf{w}}$ (sección 4.5), que asigna mayor peso a la región central del parche.

Formalmente, el operador de media espacial ponderada se define como:

$$\mathbb{E}_{\tilde{\mathbf{w}}}[d(\mathbf{x}, \hat{\mathbf{x}})] = \frac{1}{C} \sum_c \overline{\tilde{\mathbf{w}} \odot d(\mathbf{x}, \hat{\mathbf{x}})}_c \quad (4.9)$$

donde $\odot$ denota el producto elemento a elemento entre máscara y mapa de error, la barra denota el promedio espacial, y el promedio final se realiza sobre los $C = 3$ canales de color. La pérdida de reconstrucción general queda:

$$\mathcal{L}_{rec} = \mathbb{E}_{\tilde{\mathbf{w}}} [d(\mathbf{x}, \hat{\mathbf{x}})] \quad (4.10)$$

Al estar $\tilde{\mathbf{w}}$ normalizada a media 1 (ecuación 4.16), esta media ponderada es directamente comparable en magnitud a una media uniforme sin ponderar, facilitando la comparación entre configuraciones. Las tres subsecciones siguientes particularizan el mapa de error $d(\cdot, \cdot)$ correspondiente.

4.4.2 Error cuadrático medio (MSE)

El primer mapa de error es el cuadrado de la diferencia punto a punto:

$$d_{MSE}(\mathbf{x}, \hat{\mathbf{x}}) = (\hat{\mathbf{x}} - \mathbf{x})^2 \quad (4.11)$$

con $\mathcal{L}_{MSE} = \mathbb{E}_{\tilde{\mathbf{w}}} [d_{MSE}(\mathbf{x}, \hat{\mathbf{x}})]$. Al elevar al cuadrado, penaliza fuertemente los errores grandes y de forma casi despreciable los pequeños, priorizando evitar desviaciones puntuales grandes sobre reducir un error pequeño pero extendido. Su gradiente, proporcional al error ($\partial d_{MSE} / \partial \hat{\mathbf{x}} \propto (\hat{\mathbf{x}} - \mathbf{x})$), produce actualizaciones suaves pero sensibles a valores atípicos: un artefacto puntual con error elevado domina la pérdida y puede sesgar el diccionario. Combinada con la máscara Gaussiana, penaliza con mayor intensidad los errores en la región central del parche.

4.4.3 Error absoluto medio (MAE)

El segundo mapa de error es el valor absoluto de la diferencia:

$$d_{MAE}(\mathbf{x}, \hat{\mathbf{x}}) = |\hat{\mathbf{x}} - \mathbf{x}| \quad (4.12)$$

con $\mathcal{L}_{MAE} = \mathbb{E}_{\tilde{\mathbf{w}}} [d_{MAE}(\mathbf{x}, \hat{\mathbf{x}})]$. Su crecimiento lineal hace que penalice de forma proporcional errores grandes y pequeños, sin que unos pocos píxeles con error elevado dominen la pérdida. Su gradiente es constante en magnitud (salvo en el punto no diferenciable $\hat{\mathbf{x}} = \mathbf{x}$) e indica solo el signo del error, lo que produce actualizaciones más uniformes y menor sensibilidad a artefactos, a costa de convergencia potencialmente más lenta en regiones donde el error ya es pequeño.

4.4.4 Similitud estructural (SSIM)

El tercer mapa de error se basa en el índice de similitud estructural (**SSIM**), que compara ventanas locales en lugar de píxeles aislados, incorporando información estructural del parche. **SSIM** descompone la comparación en luminancia, contraste y estructura, de forma que

dos parches con el mismo error cuadrático medio pueden tener un **SSIM** muy distinto si uno preserva mejor bordes y texturas.

El mapa local de similitud se calcula como:

$$\text{SSIM}(\mathbf{x}, \hat{\mathbf{x}}) = \frac{(2\mu_x\mu_{\hat{x}} + C_1)(2\sigma_{x\hat{x}} + C_2)}{(\mu_x^2 + \mu_{\hat{x}}^2 + C_1)(\sigma_x^2 + \sigma_{\hat{x}}^2 + C_2)} \quad (4.13)$$

donde $\mu_x, \mu_{\hat{x}}$ son las medias locales, $\sigma_x^2, \sigma_{\hat{x}}^2$ las varianzas locales y $\sigma_{x\hat{x}}$ la covarianza local, calculadas mediante convolución con un kernel Gaussiano de tamaño 11×11 y $\sigma = 1.5$ (valores estándar de la formulación original de **SSIM**). Las constantes de estabilidad son $C_1 = (k_1 \cdot d)^2$ y $C_2 = (k_2 \cdot d)^2$, con $k_1 = 0.01, k_2 = 0.03$ y d el rango de los datos; evitan divisiones por valores cercanos a cero en regiones homogéneas del parche.

El mapa de error asociado es el complemento de la similitud:

$$d_{\text{SSIM}}(\mathbf{x}, \hat{\mathbf{x}}) = 1 - \text{SSIM}(\mathbf{x}, \hat{\mathbf{x}}) \quad (4.14)$$

con $\mathcal{L}_{\text{SSIM}} = \mathbb{E}_{\tilde{\mathbf{w}}}[d_{\text{SSIM}}(\mathbf{x}, \hat{\mathbf{x}})]$. A diferencia de **MSE** y **MAE**, cuyo gradiente depende solo del error en cada posición, el gradiente de $\mathcal{L}_{\text{SSIM}}$ en un píxel depende también de sus vecinos dentro de la ventana local, lo que permite a **SSIM** capturar relaciones estructurales que las otras dos ignoran por construcción.

4.5 Máscara Gaussiana centrada

UN aspecto diferencial del diseño experimental de este trabajo es el uso de una máscara Gaussiana centrada como ponderación espacial en las funciones de pérdida y en las métricas de evaluación. Su motivación reside en que cada parche del dataset **MIDOG** está centrado en una célula de interés, por lo que la región central contiene la información morfológica más relevante, mientras que la periferia puede contener células parciales u otras estructuras de menor interés.

La máscara Gaussiana $\mathbf{W} \in \mathbb{R}^{H \times W}$ se define como:

$$w_{h,w} = \exp\left(-\frac{u_h^2 + v_w^2}{2\sigma_g^2}\right) \quad (4.15)$$

donde (u_h, v_w) son coordenadas normalizadas en $[-1, 1]$, y $\sigma_g = 0.35$ controla la anchura de la Gaussiana, elegida para concentrar la mayor parte del peso en el tercio central del parche sin anular por completo la periferia. Valores altos de $w_{h,w}$ en el centro decaen suavemente hacia los bordes, tal y como se aprecia en la figura 4.4. La máscara se normaliza a media 1:

$$\tilde{w}_{h,w} = \frac{w_{h,w}}{\bar{w}} \quad (4.16)$$

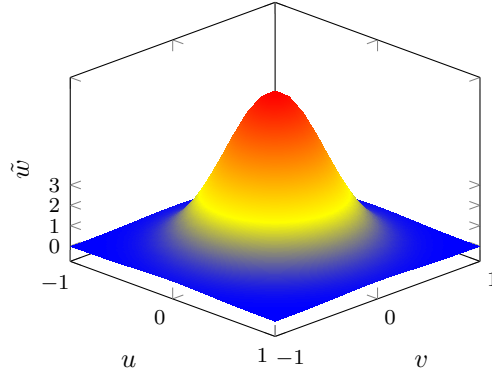


Figura 4.4: Máscara Gaussiana centrada $\tilde{w}_{h,w}$ empleada como ponderación espacial en las funciones de pérdida y métricas de evaluación. La máscara asigna mayor peso a la región central del parche, donde se encuentra la célula de interés, y menor peso a las regiones periféricas.

donde $\bar{w} = \frac{1}{HW} \sum_{h,w} w_{h,w}$.

Aplicación práctica. La máscara normalizada $\tilde{\mathbf{w}}$ es la que aparece en el operador de la ecuación 4.9: se replica sobre los tres canales y se multiplica elemento a elemento con el mapa de error, de forma que un error en el centro (donde $\tilde{w}_{h,w} > 1$) se amplifica antes de promediarse, mientras que uno en una esquina (donde $\tilde{w}_{h,w} < 1$) se atenúa. La misma máscara se emplea tanto en las funciones de pérdida (sección 4.4) como en las métricas de evaluación (sección 4.7).

El uso de esta máscara dirige la atención del modelo hacia la reconstrucción fiel de la región central, favoreciendo que el diccionario capture patrones morfológicos relevantes, y reduce el peso de las regiones periféricas, que pueden contener artefactos o estructuras irrelevantes.

4.6 Detalles de entrenamiento y experimentación

EL entrenamiento de los modelos se realiza mediante el optimizador AdamW [23] con una tasa de aprendizaje inicial de $\eta = 10^{-3}$ y un weight decay de $\lambda_{wd} = 10^{-4}$ sobre $\mathbf{W}_e$. La matriz de realimentación $\mathbf{S}$ emplea un weight decay dos órdenes de magnitud superior, $\lambda_{wd,S} = 10^{-2}$, para evitar una patología de colapso de dicha matriz observada empíricamente con valores menores: al ser $\mathbf{S}$ la responsable de la competición entre átomos, una regularización insuficiente permitía que sus valores crecieran sin control. El tamaño de batch depende de la función de pérdida: $B = 8192$ para MSE y MAE, y $B = 4096$ para SSIM (mediante el argumento `--batch_size`), ya que con $B = 8192$ esta configuración agotaba la memoria de la GPU por el coste adicional de las convoluciones del cálculo de SSIM. El número de épocas es 30 000 en todos los experimentos, valor elegido para garantizar tiempo suficiente

Tabla 4.1: Configuraciones experimentales. Se combinan tres funciones de pérdida de reconstrucción con dos dimensiones del diccionario. En todas las configuraciones se emplea aumentación de datos durante el entrenamiento, tal y como se describe en la sección 4.2. El tamaño de batch se reduce a la mitad para SSIM por motivos de memoria (véase texto). Todas comparten el resto de hiperparámetros de entrenamiento y la misma partición train/test.

Configuración	Función de pérdida	Dimensión del código	Tamaño de batch
mse_aug_128	MSE Gaussiano	128	8192
mse_aug_1024	MSE Gaussiano	1024	8192
mae_aug_128	MAE Gaussiano	128	8192
mae_aug_1024	MAE Gaussiano	1024	8192
ssim_aug_128	SSIM Gaussiano	128	4096
ssim_aug_1024	SSIM Gaussiano	1024	4096

para alcanzar un régimen estable o, en su caso, para que cualquier inestabilidad se manifieste plenamente.

4.6.1 Configuraciones experimentales

Con el objetivo de estudiar el efecto de la función de pérdida y del tamaño del diccionario, se definen seis configuraciones experimentales que combinan tres funciones de pérdida (MSE, MAE y SSIM) con dos dimensiones del código (128 y 1024), empleando augmentación de datos en todos los casos. Estas dos dimensiones se eligen como dos extremos bien diferenciados: $m = 128$ representa un diccionario reducido y esperablemente estable, mientras que $m = 1024$ —ocho veces mayor— somete al modelo a un régimen más exigente, con mayor riesgo de inestabilidad del diccionario y del controlador de dispersión (sección 4.6.5). Esta separación amplia permite exponer con claridad si el tamaño del código condiciona la viabilidad del entrenamiento, en lugar de variaciones marginales cuyo efecto podría quedar enmascarado por el ruido experimental. La tabla 4.1 resume las configuraciones resultantes.

4.6.2 Fases de entrenamiento

Antes de entrar en el detalle de cada mecanismo, esta sección resume de forma conceptual la secuencia completa de fases que atraviesa el entrenamiento. Los detalles concretos de implementación se desarrollan en las tres secciones siguientes.

1. **Calentamiento inicial (duración fija).** La matriz de realimentación $\mathbf{S}$ permanece congelada y no se aplica presión de dispersión, dando tiempo al diccionario $\mathbf{W}_e$ a organizarse antes de que los átomos compitan entre sí (sección 4.6.3).

2. **Fase A: calentamiento extendido (duración variable).** Una vez liberada $\mathbf{S}$, el entrenamiento sigue optimizando solo la reconstrucción, sin presión de dispersión, hasta alcanzar un plateau, determinado dinámicamente (sección 4.6.5).
3. **Fase B: ajuste dinámico de la dispersión.** Concluida la Fase A, un controlador ajusta automáticamente λ_{L1} el resto del entrenamiento, manteniendo una fracción objetivo de activos por muestra (sección 4.6.5).

De forma transversal, un mecanismo de reactivación de átomos muertos actúa tanto durante el calentamiento como en el resto del entrenamiento, con una implementación distinta en cada caso (sección 4.6.4).

4.6.3 Calentamiento inicial

El entrenamiento comienza con un periodo de calentamiento de duración fija, $W_{warmup} = 100$ épocas, común a las seis configuraciones —un valor elegido como compromiso entre dar tiempo suficiente al diccionario $\mathbf{W}_e$ para organizarse y no dilatar innecesariamente el arranque del entrenamiento, ya que en las pruebas preliminares la pérdida de reconstrucción mostraba una tendencia ya estable a esta altura—. Durante este periodo se mantiene $\lambda_{L1} = 0$ y $\mathbf{S}$, $\mathbf{b}_S$ se fuerzan explícitamente a cero al final de cada época, evitando que la matriz de *explaining away* compita entre átomos antes de que $\mathbf{W}_e$ se organice en torno a una proyección razonable; solo transcurrido el calentamiento se permite que $\mathbf{S}$ evolucione libremente (sección 4.3).

4.6.4 Reactivación de átomos muertos

Un mismo mecanismo de fondo —reorientar la fila de $\mathbf{W}_e$ de un átomo inactivo hacia la dirección de entradas peor reconstruidas— se aplica de forma transversal, con dos variantes según la fase.

Durante el calentamiento inicial. Ejecutado cada 10 épocas: los átomos cuya actividad —EMA de su tasa de activación— no supera un umbral mínimo se consideran muertos y se re-muestran reorientando su fila. Durante el calentamiento, $\mathbf{S}$ está congelada, por lo que reorientar $\mathbf{W}_e$ es la única forma de revivir un átomo en esta fase. El número re-muestreado se limita al 5% del diccionario por evento, evitando desestabilizar la reconstrucción.

Tras el calentamiento. Un mecanismo análogo permite reactivar átomos también en las fases A y B (sección 4.6.5); su comportamiento empírico se analiza en el Capítulo 5.

4.6.5 Ajuste dinámico del peso de dispersión λ_{L1}

El peso de dispersión λ_{L1} de la ecuación 4.7 no se fija a un valor constante, sino que se ajusta automáticamente mediante un controlador de dos fases, con el objetivo de mantener

una fracción objetivo de coeficientes activos sin fijar λ_{L1} manualmente por configuración. Entra en juego una vez concluido el calentamiento (sección 4.6.3).

Fase A: calentamiento extendido sin presión de dispersión. λ_{L1} se mantiene en 0 hasta que la reconstrucción alcanza un plateau, detectado comparando el *mínimo* de la pérdida en dos ventanas consecutivas de $W_{plateau}$ épocas: la Fase A concluye cuando el mínimo de la ventana reciente no mejora, respecto a la anterior, en más de un umbral relativo $\varepsilon_{plateau}$. Usar el mínimo en lugar de la media reduce la sensibilidad a picos de ruido, a costa de reaccionar algo más tarde.

Fase B: ajuste multiplicativo hacia el objetivo de dispersión. λ_{L1} se inicializa con una semilla $11_seed = 10^{-7}$ —deliberadamente pequeña, para introducir la presión de dispersión de forma gradual y no desestabilizar con un salto brusco el diccionario recién estabilizado— y se actualiza cada época según una regla multiplicativa que depende de la media móvil exponencial (EMA) de los activos:

$$\bar{a}^{(t)} = \beta \bar{a}^{(t-1)} + (1 - \beta) L0^{(t)} \quad (4.17)$$

donde $L0^{(t)}$ es el número medio de activos en la época t (ecuación 4.23), y $\beta = 0.9$ atenúa las fluctuaciones época a época, favoreciendo una reacción ágil del controlador ante desviaciones sostenidas frente a una memoria más larga de épocas pasadas. El controlador compara $\bar{a}^{(t)}/m$ con el objetivo de diseño $\rho^* = 0.50$ —un punto intermedio entre un código excesivamente denso, que apenas impondría restricción, y uno excesivamente disperso, que limitaría en exceso la capacidad de reconstrucción—, dentro de una banda de tolerancia relativa $\delta = 11_active_tol = 0.05$, y actualiza λ_{L1} :

$$\lambda_{L1}^{(t+1)} = \begin{cases} \max(\lambda_{L1}^{(t)} \cdot (1 + \gamma), 11_seed) & \text{si } \bar{a}^{(t)}/m > \rho^*(1 + \delta) \\ \max(\lambda_{L1}^{(t)}/(1 + \gamma), 11_seed) & \text{si } \bar{a}^{(t)}/m < \rho^*(1 - \delta) \\ \lambda_{L1}^{(t)} & \text{en otro caso} \end{cases} \quad (4.18)$$

donde $\gamma = 11_growth$ es el paso de crecimiento, y el $\max(\cdot, 11_seed)$ impide que λ_{L1} baje de la semilla. La banda muerta δ evita oscilaciones cuando la fracción de activos ya está próxima al objetivo. En la práctica: código demasiado denso aumenta la penalización en la época siguiente; código demasiado disperso la reduce, sin llegar a anularla; dentro de la banda, λ_{L1} no cambia. Su comportamiento empírico, y las limitaciones al escalar el diccionario, se analizan en la sección 5.2.2.

4.6.6 Gestión de checkpoints

Durante el entrenamiento se guardan tres tipos de checkpoints:

- **Checkpoint óptimo:** la época con menor pérdida de reconstrucción sobre el conjunto de entrenamiento.
- **Checkpoint final:** la última época de entrenamiento.
- **Checkpoints de salto:** guardados automáticamente cuando se detecta un cambio brusco (variación relativa $> 25\%$) en la pérdida o en los coeficientes activos entre épocas consecutivas, permitiendo analizar la trayectoria y detectar colapsos o reorganizaciones del diccionario.

Esta estrategia permite comparar el checkpoint óptimo frente al final a lo largo del entrenamiento, tal y como se describe en el Capítulo 5.

4.6.7 Hiperparámetros

La tabla 4.2 resume los hiperparámetros comunes a todos los experimentos, con la justificación de los principales ya detallada a lo largo de esta sección. El peso de dispersión λ_{L1} no se fija a un valor único, sino que se ajusta automáticamente mediante el controlador de dos fases (sección 4.6.5); sus resultados empíricos se analizan en el Capítulo 5.

4.7 Métricas de evaluación

LA evaluación de los modelos entrenados se realiza sobre el conjunto de test, empleando un conjunto de métricas que cubren tres aspectos complementarios de las representaciones aprendidas: la calidad de reconstrucción, la dispersión de los códigos y la coherencia del diccionario. Todas las métricas de reconstrucción se calculan con la máscara Gaussiana centrada descrita en la sección 4.5.

4.7.1 Métricas de reconstrucción

Error cuadrático medio Gaussiano (MSE Gaussiano). Mide el error cuadrático medio entre el parche original y el reconstruido, ponderado con la máscara Gaussiana centrada:

$$\text{MSE}_g = \frac{1}{N} \sum_{i=1}^N \sum_{c,h,w} \tilde{w}_{h,w} \cdot (\hat{x}_{i,c,h,w} - x_{i,c,h,w})^2 \quad (4.19)$$

Error absoluto medio Gaussiano (MAE Gaussiano). Mide el error absoluto medio ponderado con la máscara Gaussiana:

$$\text{MAE}_g = \frac{1}{N} \sum_{i=1}^N \sum_{c,h,w} \tilde{w}_{h,w} \cdot |\hat{x}_{i,c,h,w} - x_{i,c,h,w}| \quad (4.20)$$

Tabla 4.2: Resumen de hiperparámetros empleados en los experimentos.

Hiperparámetro	Valor
Tamaño de parche	64×64 px
Dimensión de entrada n	12 288
Dimensión del código m	128, 1024
Iteraciones LISTA T	3
Épocas	30 000
Épocas de calentamiento W_{warmup}	100
Reactivación de átomos en calentamiento	cada 10 épocas
Fracción máxima re-muestreada por evento	5%
Tamaño de batch B (MSE/MAE)	8192
Tamaño de batch B (SSIM)	4096
Tasa de aprendizaje η	10^{-3}
Weight decay λ_{wd} ($\mathbf{W}_e$)	10^{-4}
Weight decay $\lambda_{wd,S}$ ($\mathbf{S}$)	10^{-2}
Peso reconstrucción λ_{rec}	1.0
Peso dispersión λ_{L1}	autorregulado (sección 4.6.5)
Semilla 11_seed	10^{-7}
Objetivo de activos ρ^*	50%
Banda muerta del regulador δ	5%
Suavizado EMA β	0.9
Sigma máscara Gaussiana σ_g	0.35
Partición train/test	80%/20%

Relación señal-ruido de pico (PSNR). Métrica ampliamente utilizada en procesamiento de imagen que mide la calidad de reconstrucción en decibelios:

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{d^2}{\text{MSE}} \right) \quad (4.21)$$

donde d es el rango de los datos (*data range*) calculado sobre los valores estandarizados del dataset, y MSE es el error cuadrático medio sin ponderar. Valores más altos de PSNR indican mejor calidad de reconstrucción.

SSIM Gaussiano. Índice de similitud estructural ponderado con la máscara Gaussiana, calculado según la ecuación 4.13. Valores más próximos a 1 indican mayor similitud estructural entre el parche original y el reconstruido.

4.7.2 Métricas de dispersión

Fracción de ceros. Proporción de componentes del código que son exactamente cero, promediada sobre todas las muestras del conjunto de test:

$$\text{frac_zeros} = \frac{1}{N} \sum_{i=1}^N \frac{|\{j : z_{ij} = 0\}|}{m} \quad (4.22)$$

Valores próximos a 1 indican códigos muy dispersos. Al emplear ReLU como no linealidad del encoder (sección 4.3), los coeficientes inactivos son ceros exactos, por lo que no es necesario introducir ningún umbral de tolerancia numérica en esta definición.

Norma ℓ_0 media (L0 activos). Número medio de coeficientes activos por muestra, es decir, componentes estrictamente positivas:

$$\text{L0} = \frac{1}{N} \sum_{i=1}^N |\{j : z_{ij} > 0\}| \quad (4.23)$$

4.7.3 Métricas de coherencia del diccionario

Coherencia del diccionario. Mide el grado de correlación entre los átomos del diccionario $\mathbf{W}_e$. Se calcula como la media de los valores absolutos de los productos internos entre pares de átomos normalizados:

$$\mu(\mathbf{W}_e) = \frac{1}{m(m-1)} \sum_{i \neq j} |\langle \tilde{\mathbf{w}}_i, \tilde{\mathbf{w}}_j \rangle| \quad (4.24)$$

donde $\tilde{\mathbf{w}}_i = \mathbf{w}_i / \|\mathbf{w}_i\|_2$ es el i -ésimo átomo normalizado. Valores bajos de coherencia indican que los átomos del diccionario son poco correlados entre sí, lo que favorece representaciones dispersas con mejor capacidad de discriminación.

4.7.4 Métricas de discriminabilidad

Para evaluar el potencial discriminativo de las representaciones aprendidas en relación a las clases de interés, se emplean las siguientes métricas calculadas sobre los códigos dispersos del conjunto de test:

Índice discriminante de Fisher (Fisher index). Para cada átomo j del diccionario, mide la separabilidad entre las distribuciones de activación de las clases mitótica y no mitótica:

$$F_j = \frac{(\mu_{j,+} - \mu_{j,-})^2}{\sigma_{j,+}^2 + \sigma_{j,-}^2} \quad (4.25)$$

donde $\mu_{j,+}$, $\sigma_{j,+}^2$ y $\mu_{j,-}$, $\sigma_{j,-}^2$ son la media y varianza de la activación del átomo j para las clases mitótica y no mitótica respectivamente.

Estadístico F de ANOVA. Para cada átomo, mide la razón entre la varianza inter-clase y la varianza intra-clase, proporcionando una medida complementaria al índice de Fisher.

Información mutua. Para cada átomo, mide la dependencia estadística entre la activación del átomo y la etiqueta de clase, capturando relaciones no lineales que los métodos anteriores podrían no detectar.

Estas tres métricas se calculan sobre todos los átomos del diccionario y se emplean para identificar qué átomos capturan información discriminativa entre figuras mitóticas y no mitóticas, tal y como se describe en el Capítulo 5.

Experimentos y resultados

ESTE capítulo presenta y analiza los resultados obtenidos con el autoencoder disperso basado en [LISTA](#) descrito en el Capítulo 4, evaluando la calidad de las representaciones aprendidas de forma no supervisada y su potencial discriminativo entre figuras mitóticas y no mitóticas, para las seis configuraciones experimentales de la tabla 4.1 —tres funciones de pérdida ([MSE](#), [MAE](#), [SSIM](#)) combinadas con dos dimensiones del diccionario, $m = 128$ y $m = 1024$.

Tras el protocolo de evaluación (sección 5.1), se analiza la dinámica del entrenamiento (sección 5.2), la calidad de reconstrucción y la dispersión de los códigos (secciones 5.3 y 5.4), la estructura del diccionario y su potencial discriminativo (secciones 5.5 y 5.6), y el efecto del tamaño del diccionario (sección 5.7), que sintetiza también la comparación global entre funciones de pérdida. Al ser un capítulo denso, con numerosas subsecciones y resultados, cada sección cierra con una breve síntesis parcial que recoge lo aprendido antes de pasar a la siguiente, de forma que la sección 5.7 y la discusión final (sección 5.8) puedan apoyarse en ideas ya asentadas en lugar de introducirlas de golpe.

5.1 Configuración experimental y protocolo de evaluación

ANTES de presentar los resultados, esta sección resume el protocolo común a todos los experimentos: el entorno de entrenamiento, el dataset de evaluación, y el criterio de comparación entre checkpoints que se aplicará de forma sistemática en el resto del capítulo.

Entorno y reproducibilidad. Todos los experimentos se entrenaron en el servidor Nidavellir del grupo [VARPA](#) (GPU NVIDIA GeForce RTX 2080 Ti, 11 GB VRAM; sección 3.4), fijando `seed = 42` en `random`, `NumPy` y `PyTorch` para garantizar la reproducibilidad.

Dataset de evaluación. El conjunto de parches de [MIDOG](#) consta de 26 286 parches (45.4% mitóticos, 54.6% no mitóticos). La partición fija de `split.json` reserva 21 029 parches para entrenamiento (80%) y 5 257 para test (20%), sin estratificar por clase, ya que las

etiquetas no se emplean en el entrenamiento y se reservan para los análisis de esta sección y las siguientes.

Configuraciones y protocolo. Las métricas se calculan sobre el conjunto de test, con parches originales sin aumentación y ponderando con la máscara Gaussiana centrada (sección 4.5). Para cada configuración se comparan tres puntos de la trayectoria: el checkpoint óptimo (menor pérdida de reconstrucción), el checkpoint final (última época) y la evolución completa entre ambos, a partir de los checkpoints de salto (sección 4.6), en lugar de asumir a priori que el checkpoint de menor pérdida es el más adecuado. Las métricas empleadas se definen en la sección 4.7.

5.2 Dinámica del entrenamiento

ESTA sección analiza cómo evolucionan la pérdida, la dispersión y la salud de los átomos del diccionario a lo largo de las 30 000 épocas, para las seis configuraciones, determinando si el entrenamiento no supervisado llega a producir un diccionario estable y disperso. Se recorren, en orden, las curvas de pérdida (sección 5.2.1), el comportamiento del controlador λ_{L1} (sección 5.2.2), la evolución de la dispersión (sección 5.2.3), la vida y muerte de los átomos (sección 5.2.4), los eventos de salto (sección 5.2.5) y, por último, una comparación cuantitativa de las seis configuraciones (sección 5.2.6).

5.2.1 Curvas de pérdida: total, reconstrucción y término L1

Para cada configuración se registran dos paneles complementarios en escala logarítmica: reconstrucción y término ℓ_1 crudo (superior), y su descomposición en pérdida total, reconstrucción, dispersión, fracción de ceros y activos (inferior). Las figuras 5.1, 5.2 y 5.3 representan la evolución del entrenamiento de las tres configuraciones más representativas de los patrones del capítulo, una por cada función de pérdida; `mae_aug_128`, `ssim_aug_128` y `mae_aug_1024` siguen patrones cualitativamente equivalentes a sus contrapartidas mostradas.

m = 128: **caso base (mse_aug_128).** La figura 5.1 muestra el mínimo de reconstrucción ($\text{rec} = 0.0499$, época 1956) tras una Fase A hasta la época 1773; la Fase B estabiliza $\text{rec} \approx 0.074\text{--}0.077$ y 64–67 de 128 activos ($\approx 49\%$ de ceros), próximo al objetivo del 50%. `mae_aug_128` (mínimo $\text{rec} = 0.157$) y `ssim_aug_128` (mínimo $\text{rec} = 0.266$, transición más tardía por su mapa de error más costoso) reproducen el mismo patrón, estabilizándose en 60–68 de 128 activos en los tres casos: el compromiso reconstrucción-dispersión buscado en el diseño se cumple con independencia de la función de pérdida.

m = 1024: **colapso bajo MSE y MAE.** La figura 5.2 evidencia un colapso progresivo: ya en la época 500 los átomos vivos caen de 972 a 105 de 1024. El mínimo formal ($\text{rec} = 0.1384$, época 770) se produce sobre un diccionario ya deteriorado; a partir de la época 816 aparecen

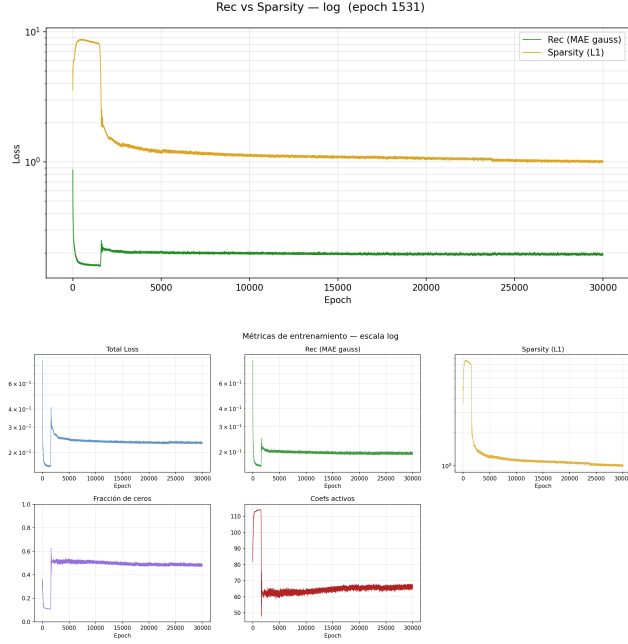


Figura 5.1: Evolución del entrenamiento para `mse_aug_128` (escala logarítmica). Arriba: reconstrucción y término ℓ_1 crudo; el salto hacia la época 2000 marca el arranque de la Fase B. Abajo: pérdida total, reconstrucción, dispersión, fracción de ceros y activos por época.

picos catastróficos recurrentes, y los vivos caen a 0 de forma definitiva desde la época 8509 ($\text{rec} \approx 1.43\text{--}1.44$, salida trivial $\hat{\mathbf{x}} = \mathbf{0}$). `mae_aug_1024` sigue el mismo patrón, con colapso aún más rápido (vivos a 0 desde la época 1953). Ambas configuraciones comprometen el objetivo de aprender un diccionario amplio y útil para $m = 1024$ (se retoma en la sección 5.2.4).

$m = 1024$: **estabilidad bajo SSIM**. La figura 5.3 contrasta claramente: sin colapso, $\text{rec} \approx 0.225\text{--}0.231$ desde ≈ 500 épocas. Los activos se estabilizan en ≈ 170 de 1024 (16.6%, por debajo del 50% objetivo): el número de activos EMA parte ya bajo la banda objetivo, por lo que λ_{L1} queda anclado en su semilla (10^{-7}) sin ejercer presión efectiva. Es la única configuración de $m = 1024$ que conserva átomos vivos de forma no trivial, indicando que **SSIM** es la única pérdida que permite escalar el diccionario sin degeneración (sección 5.2.6).

5.2.2 Controlador automático del peso de dispersión λ_{L1}

Esta subsección examina la trayectoria del controlador λ_{L1} (mecanismo de dos fases, sección 4.6.5, ecuaciones 4.17 y 4.18). Su comportamiento determina si el objetivo de dispersión llega a cumplirse. La tabla 5.1 resume, para las seis configuraciones, la época de fin de la Fase A, el pico de λ_{L1} y su época, el valor de equilibrio o final, y los átomos activos resultantes.

$m = 128$. Tras el fin de la Fase A (épocas 1773, 1333 y 1786 para **MSE**, **MAE**, **SSIM**), λ_{L1} crece desde la semilla, sobrepasa transitoriamente su equilibrio y se relaja. Pico y equilibrio

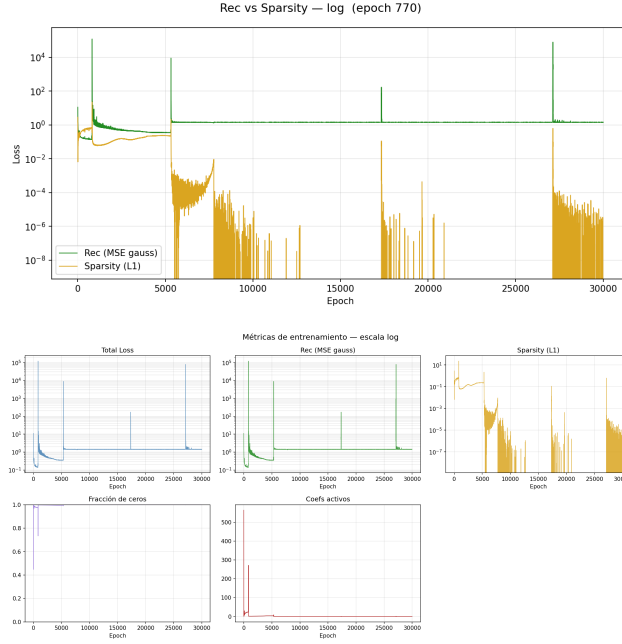


Figura 5.2: Evolución del entrenamiento para `mse_aug_1024` (escala logarítmica). Picos catastróficos recurrentes; colapso irreversible de los átomos vivos a 0 desde la época 8509.

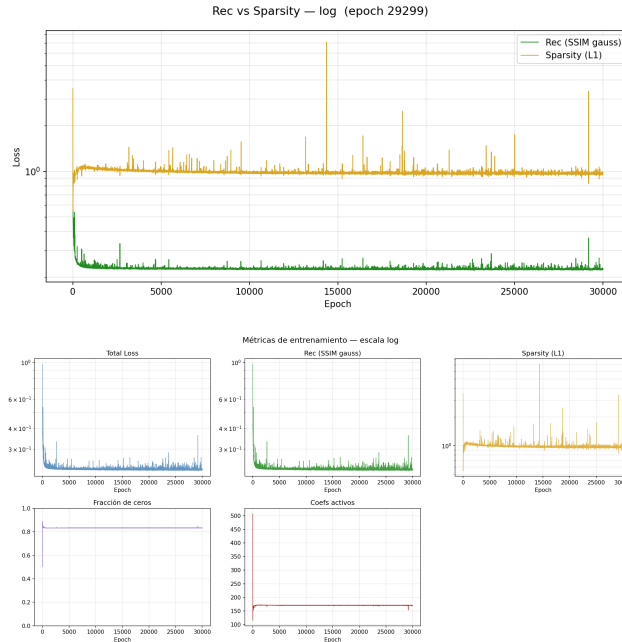


Figura 5.3: Evolución del entrenamiento para `ssim_aug_1024` (escala logarítmica). Sin colapso; activos estables en ≈ 170 de 1024, con λ_{L1} anclado en su valor semilla.

Tabla 5.1: Trayectoria del controlador λ_{L1} para las seis configuraciones.

Configuración	Fin Fase A	Pico λ_{L1}	Época pico	Equilibrio/final	Activos
mse_aug_128	1773	0.0501	2041	0.0162	$\approx 65/128$
mae_aug_128	1333	0.0816	1611	0.0379	$\approx 62-67/128$
ssim_aug_128	1786	0.1539	2077	0.0730	$\approx 65-67/128$
mse_aug_1024	1172	10^{-7}	—	10^{-7}	0/1024
mae_aug_1024	1927	10^{-7}	—	10^{-7}	0/1024
ssim_aug_1024	1643	10^{-7}	—	10^{-7}	$\approx 170/1024$

crecen en el orden $MSE < MAE < SSIM$, consistente con el coste de optimización de cada pérdida. En los tres casos los activos convergen a $\approx 60-67$ de 128: el controlador cumple su función con independencia de la pérdida.

$m = 1024$. λ_{L1} nunca supera la semilla 10^{-7} en ninguna de las tres configuraciones: el número de activos EMA se mantiene siempre muy por debajo del objetivo de 512, de modo que el regulador nunca recibe la señal para subir λ_{L1} . Esto sugiere que la semilla y el paso de crecimiento —calibrados para $m = 128$ — resultan demasiado conservadores al escalar el diccionario un orden de magnitud (se retoma en la discusión, sección 5.8).

5.2.3 Evolución de la dispersión durante el entrenamiento

ESTA subsección compara la dispersión por muestra (fracción de ceros y ℓ_0), distinguiendo Fase A y Fase B.

Fase A. Sin penalización, el único límite es la ReLU del encoder. En $m = 128$ esto produce códigos casi densos ($\approx 88-92\%$ activos). En $m = 1024$ el número de activos ya decae espontáneamente durante la propia Fase A, sin mediar regulación alguna: la dimensión del diccionario afecta a la estabilidad incluso antes de que actúe el controlador.

Fase B. Las tres de $m = 128$ convergen de forma consistente al entorno del 50% ($\approx 60-68$ activos), pese a requerir λ_{L1} muy distintos. En $m = 1024$, ninguna alcanza el objetivo de 512 activos, pero por vías opuestas: mse/mae llegan al 100% de ceros por colapso degenerado, mientras ssim_aug_1024 se estabiliza en ≈ 170 activos (83% de ceros) por falta de presión, no por extinción.

Síntesis parcial. A $m = 128$ el sistema converge de forma consistente al objetivo de diseño con independencia de la función de pérdida; a $m = 1024$ se bifurca entre la extinción total del código en MSE/MAE y una estabilización sub-óptima pero no degenerada en SSIM, único caso que conserva una representación potencialmente útil (se retoma en la sección 5.2.4).

5.2.4 Vida y muerte de átomos durante el entrenamiento

ESTA subsección examina los átomos vivos/muertos/agonizantes (ecuación 4.17). El mecanismo de revival está desactivado tras el calentamiento en las seis configuraciones:

Tabla 5.2: Dinámica de vida y muerte de átomos para las seis configuraciones.

Configuración	Vivos ép. 1	Máx. muertos	Época máx. muertos	Vivos final	Eventos de salto	Revivals post-warmup
mse_aug_128	127/128	18/128	25533	115/128	6	0
mae_aug_128	128/128	6/128	14370	126/128	0	0
ssim_aug_128	128/128	12/128	21264	121/128	0	0
mse_aug_1024	972/1024	1024/1024	— (sostenido)	0/1024	493	0
mae_aug_1024	995/1024	1024/1024	— (sostenido)	0/1024	116	0
ssim_aug_1024	1016/1024	832/1024	9363	293/1024	8	0

cualquier recuperación posterior es espontánea, no deliberada.

$m = 128$: **erosión lenta, sin colapso**. El diccionario permanece mayoritariamente vivo (mse: 18 muertos máx. de 128; mae: 6; ssim: 12). Los vivos se mantienen siempre por encima del 86%.

$m = 1024$: **dos colapsos totales, una estabilización sub-óptima**. mse/mae caen a 0 vivos de forma definitiva (épocas 8509 y 1953), con episodios recurrentes de recuperación espontánea seguidos de recolapso (detalle en el Anexo A). ssim_aug_1024 decae con fuerza en Fase A pero se estabiliza, oscilando entre 192 y 293 de 1024 sin volver a acercarse a 0.

La tabla 5.2 muestra, para las seis configuraciones, los vivos en la primera época, el máximo de muertos, los vivos al final del entrenamiento, y el número de saltos y revivals. Confirma que, para $m = 1024$, es la dimensión del diccionario —y no el controlador, que nunca se activa— la que determina la estabilidad. Hallazgo central, retomado en la discusión.

5.2.5 Detección de saltos y reorganización del diccionario

ESTA subsección revisa los eventos de salto detectados durante el entrenamiento (sección 4.6), que registran pares de épocas consecutivas con variación $> 25\%$ en reconstrucción o activos, como confirmación adicional —desde una fuente de datos distinta a la dinámica de vida y muerte de átomos— de los patrones ya observados. El total por configuración va de 0–6 eventos en $m = 128$ a 8–493 en $m = 1024$, reflejando directamente los colapsos ya documentados.

En $m = 128$, mae y ssim no registran ningún salto; los 6 de mse son reorganizaciones locales menores. En $m = 1024$, mse (493) y mae (116) muestran una dinámica oscilante en torno a un punto degenerado (detalle en el Anexo A); ssim_aug_1024, en cambio, concentra sus 8 eventos en las primeras 500 épocas sin ningún colapso sostenido, confirmando la mayor estabilidad de SSIM a esta dimensión.

5.2.6 Comparación de la dinámica entre configuraciones

LA tabla 5.3 reúne las magnitudes clave de la sección para las seis configuraciones. Es la dimensión del diccionario, no la función de pérdida, la que determina si el controlador

Tabla 5.3: Comparativa de la dinámica de entrenamiento para las seis configuraciones.

Configuración	Fin Fase A	rec min. (óptimo)	rec final	λ_{L1} pico/final	Activos/Vivos final	Salto	Modo de fallo
mse_aug_128	1773	0.0499	$\approx 0.074\text{--}0.077$	0.0501 / 0.0162	$\approx 65 / 115$ de 128	6	Ninguno
mae_aug_128	1333	0.1574	$\approx 0.195\text{--}0.199$	0.0816 / 0.0379	$\approx 62\text{--}67 / 126$ de 128	0	Ninguno
ssim_aug_128	1786	0.2661	$\approx 0.345\text{--}0.355$	0.1539 / 0.0730	$\approx 65\text{--}68 / 121$ de 128	0	Ninguno
mse_aug_1024	1172	0.1384	$\approx 1.43\text{--}1.44$	$10^{-7} / 10^{-7}$	0 / 0 de 1024	493	Colapso progresivo
mae_aug_1024	1927	0.2163	$\approx 1.00\text{--}1.01$	$10^{-7} / 10^{-7}$	0 / 0 de 1024	116	Colapso súbito
ssim_aug_1024	1643	0.2217	$\approx 0.225\text{--}0.231$	$10^{-7} / 10^{-7}$	$\approx 170 / 293$ de 1024	8	Sub-sparsificación crónica

opera como fue diseñado: a $m = 128$ las tres pérdidas alcanzan el objetivo del 50%; a $m = 1024$ ninguna lo alcanza, y el resultado se bifurca entre el colapso irreversible de **MSE/MAE** y la sub-sparsificación crónica pero estable de **SSIM**. La causa raíz es la misma en los tres casos: λ_{L1} nunca supera la semilla 10^{-7} , apuntando a una calibración del controlador implícitamente ajustada para $m = 128$.

Síntesis parcial. La dinámica de entrenamiento revela un patrón consistente: a $m = 128$ las tres funciones de pérdida producen trayectorias estables, con el controlador cumpliendo su objetivo. A $m = 1024$ el comportamiento se bifurca según la pérdida, y en los tres casos coincide la misma causa raíz: el controlador λ_{L1} nunca despegas de su semilla al escalar el diccionario. Esta idea del colapso del diccionario a $m = 1024$ como fenómeno determinado por la calibración del controlador —y no únicamente por la función de pérdida en sí— es uno de los hilos conductores que se retoma a lo largo de las secciones siguientes y se sintetiza en la sección 5.7.

5.3 Calidad de reconstrucción

ESTA sección evalúa la calidad de reconstrucción de las seis configuraciones sobre el conjunto de test, complementando la dinámica de entrenamiento con métricas sobre datos nunca vistos: primero los resultados cuantitativos agregados (sección 5.3.1), después una inspección cualitativa de reconstrucciones concretas (sección 5.3.2), y por último la evolución del checkpoint a lo largo de toda la trayectoria (sección 5.3.3). Para mse_aug_1024 y mae_aug_1024, el checkpoint final corresponde a un diccionario colapsado: sus métricas deben leerse como el resultado degenerado de predecir $\hat{\mathbf{x}} = \mathbf{0}$, no como una reconstrucción genuina.

5.3.1 Resultados cuantitativos (MAE_g , MSE_g , PSNR , SSIM_g)

La tabla 5.4 recoge las cuatro métricas sobre el test ($n = 5\,257$ parches) para ambos checkpoints. El PSNR se calcula sin ponderar (a diferencia de las otras tres, que sí usan la máscara Gaussiana; sección 4.7).

Checkpoint óptimo. Para $m = 128$, cada pérdida domina en la métrica que optimiza. Este patrón se acentúa en $m = 1024$: ssim_aug_1024 logra el mejor $\text{SSIM}_g/\text{MSE}_g/\text{MAE}_g$ de las seis —mejores incluso que $m = 128$ — pero el PSNR más bajo (12.24 dB), porque al

Tabla 5.4: Métricas de reconstrucción sobre el test compartido para las seis configuraciones. Negrita: mejor resultado del bloque. `mse_aug_1024` y `mae_aug_1024` son idénticos en el checkpoint final por colapso a $\mathbf{z} = \mathbf{0}$.

Configuración	Época	MAE _g (↓)	MSE _g (↓)	PSNR dB (↑)	SSIM _g (↑)
Checkpoint óptimo					
<code>mse_aug_128</code>	1956	0.1935	0.0713	19.43	0.6492
<code>mae_aug_128</code>	1531	0.1953	0.0751	18.57	0.6477
<code>ssim_aug_128</code>	1997	0.2005	0.0814	16.65	0.6694
<code>mse_aug_1024</code>	770	0.3117	0.1736	17.31	0.4190
<code>mae_aug_1024</code>	1563	0.2577	0.1255	17.65	0.5135
<code>ssim_aug_1024</code>	29299	0.1758	0.0670	12.24	0.7201
Checkpoint final (época 30 000)					
<code>mse_aug_128</code>	30000	0.2335	0.1020	18.34	0.5655
<code>mae_aug_128</code>	30000	0.2291	0.1045	15.46	0.5792
<code>ssim_aug_128</code>	30000	0.2529	0.1329	11.93	0.5776
<code>mse_aug_1024</code>	30000	0.9375	1.2691	12.54	0.0070
<code>mae_aug_1024</code>	30000	0.9375	1.2691	12.54	0.0070
<code>ssim_aug_1024</code>	30000	0.1763	0.0657	12.17	0.7179

entrenar solo sobre la región central ponderada, el modelo concentra ahí su capacidad, a costa de la periferia.

Checkpoint final. Para $m = 128$, las tres se degradan moderadamente respecto al checkpoint óptimo, coherente con que este se alcanza antes de que el controlador termine de imponer su presión. Para $m = 1024$, `mse_aug_1024` y `mae_aug_1024` obtienen valores *idénticos* en las cuatro métricas —confirmación directa del colapso—, mientras que `ssim_aug_1024` apenas se degrada y sigue siendo la mejor configuración. Primera evidencia cuantitativa sobre test de que la viabilidad de $m = 1024$ depende críticamente de la función de pérdida.

5.3.2 Reconstrucciones cualitativas por clase

Se ilustran los resultados cuantitativos sobre una muestra mitótica de test fija (idx 6132), comparando ambos checkpoints, para las tres configuraciones más representativas. El patrón en la muestra no mitótica (idx 26139) es análogo, con MSE_g sistemáticamente menor: la muestra mitótica es consistentemente más difícil de reconstruir, apuntando a una dificultad intrínseca de la morfología mitótica más que a un artefacto de la función de pérdida (se retoma en la sección 5.6).

Caso base. La figura 5.4 muestra que la reconstrucción renuncia a la textura fina, conservando solo la paleta dominante y la posición del núcleo —contrapartida de que el código se explique con ≈ 65 átomos de 128.

Colapso. La figura 5.5 ilustra directamente el colapso: en el checkpoint óptimo (época

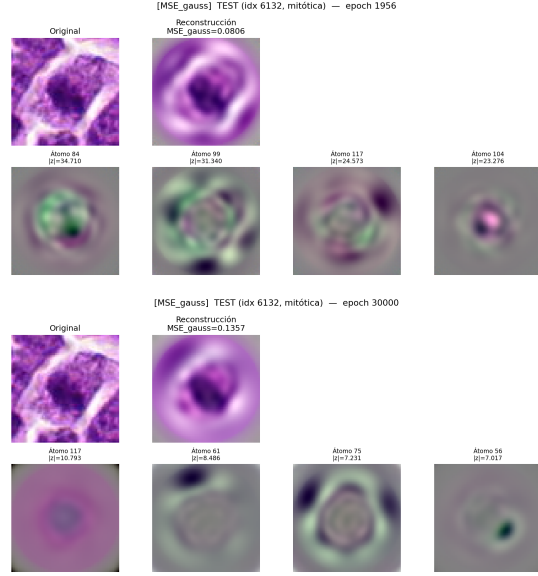


Figura 5.4: Reconstrucción de un parche mitótico de test (`mse_aug_128`, idx 6132) en el checkpoint óptimo (arriba, $MSE_g = 0.0806$) y el checkpoint final (abajo, $MSE_g = 0.1357$).

770) la reconstrucción aún conserva estructura; en el checkpoint final es un color uniforme, idéntico con independencia de la entrada —la representación visual del colapso a $\mathbf{z} = \mathbf{0}$. `mae_aug_1024` reproduce el mismo patrón. Demuestra que `MSE/MAE` pierden por completo su capacidad representacional a $m = 1024$ sin ajustar el controlador de dispersión.

Estabilidad sin colapso. La figura 5.6 confirma la trayectoria estacionaria de `ssim_aug_1024`: la reconstrucción apenas varía entre checkpoints. Reproduce el patrón de `ssim_aug_128` (pérdida de textura fina, artefactos en las esquinas), pero reparte la codificación de fondo entre *dos* átomos redundantes. Única configuración de $m = 1024$ que conserva utilidad representacional real, y por tanto la candidata más relevante para el análisis discriminativo siguiente.

5.3.3 Evolución del checkpoint a lo largo del entrenamiento

Se reconstruye la trayectoria completa de la muestra mitótica de test a través de los checkpoints de salto, evaluados sobre el test, para comprobar si la comparación aislada entre checkpoint óptimo y final resulta suficiente. `mse_aug_128` revela un patrón más rico que la simple comparación entre ambos checkpoints: mejora hasta un plateau ya hacia la época 416, sostenido hasta el checkpoint óptimo (época 1956); tras el salto de la época 4241, el error empeora a 0.1373 y no recupera el nivel previo, confirmando que el checkpoint óptimo es un óptimo genuino pero se pierde relativamente pronto.

Colapso confirmado sobre test. La figura 5.7 proyecta el colapso con más contundencia

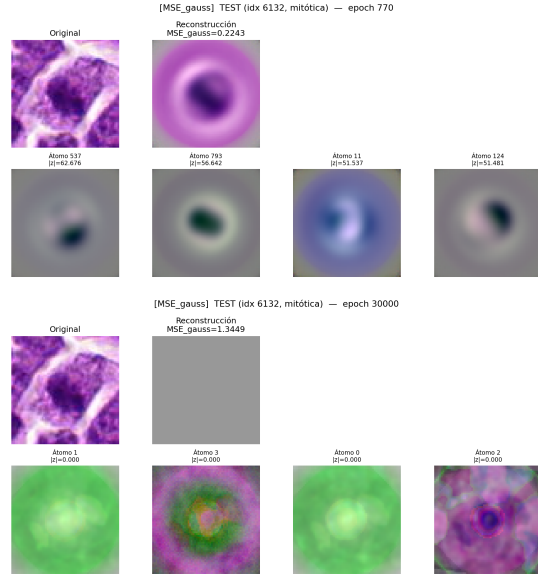


Figura 5.5: Parche mitótico de test, mse_aug_1024 , en el checkpoint óptimo (arriba, época 770, $MSE_g = 0.2243$) y el checkpoint final (abajo, $MSE_g = 1.3449$, color uniforme: código colapsado a $\mathbf{z} = \mathbf{0}$).

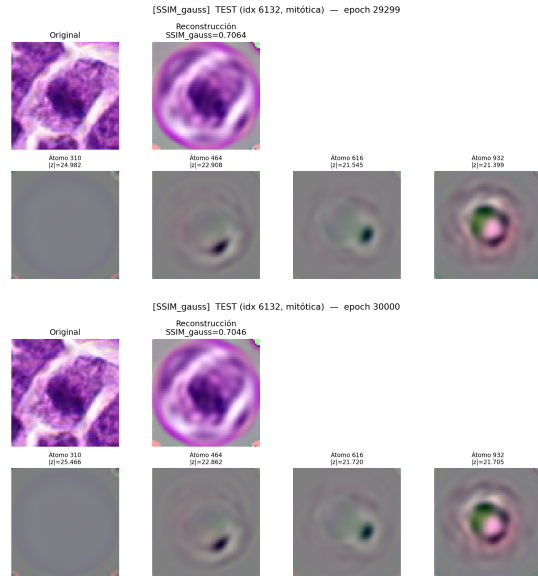


Figura 5.6: Parche mitótico de test, $ssim_aug_1024$, en el checkpoint óptimo (arriba, época 29299, $SSIM_g = 0.7064$) y el checkpoint final (abajo, $SSIM_g = 0.7046$): sin colapso, a diferencia de la figura 5.5.

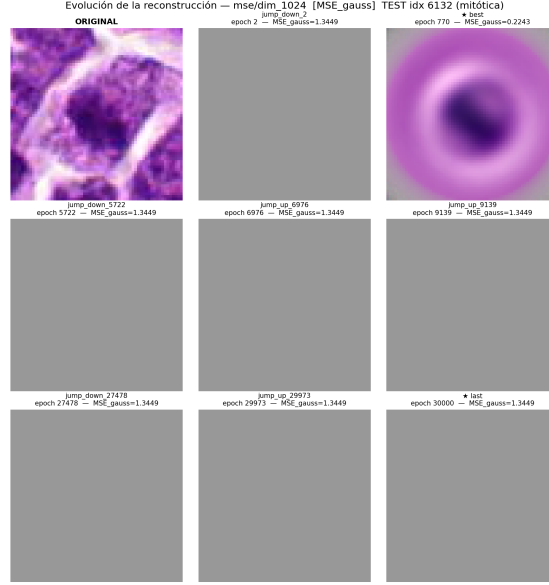


Figura 5.7: Trayectoria de reconstrucción del parche mitótico de test para `mse_aug_1024`. Salvo la breve ventana del checkpoint óptimo (época 770), todos los checkpoints muestran la reconstrucción trivial.

que las curvas de entrenamiento: de ocho checkpoints, solo el óptimo exhibe estructura reconocible; los demás son indistinguibles de la reconstrucción trivial. `mae_aug_1024` sigue el mismo patrón (Anexo A). Evidencia más directa de que, para $m = 1024$ con `MSE/MAE`, el modelo es inutilizable en casi toda su trayectoria.

Estabilidad temprana y sostenida. La figura 5.8 confirma que ningún checkpoint produce la reconstrucción trivial: ya hacia la época 2663 la métrica está a menos de dos puntos porcentuales del valor final, lo que explica por qué el checkpoint óptimo se alcanza tan tarde (época 29 299): no por mejora sostenida, sino por fluctuaciones pequeñas una vez alcanzado el régimen estable. El argumento visual más claro de por qué `SSIM` es la única función de pérdida viable para diccionarios grandes.

Síntesis parcial. Sobre el test, la evaluación cuantitativa confirma lo observado durante el entrenamiento: a $m = 128$ las tres pérdidas ofrecen reconstrucciones razonables y estables; a $m = 1024$ el comportamiento se bifurca —`SSIM` logra la mejor reconstrucción cruzada de las seis, mientras `MSE/MAE` colapsan en casi toda su trayectoria—. El análisis por trayectoria completa resulta especialmente revelador: comparar únicamente checkpoint óptimo y final, como es habitual en la literatura, puede ocultar tanto plateaus perdidos tras un salto como colapsos casi totales que solo se manifiestan al reconstruir la trayectoria entera.

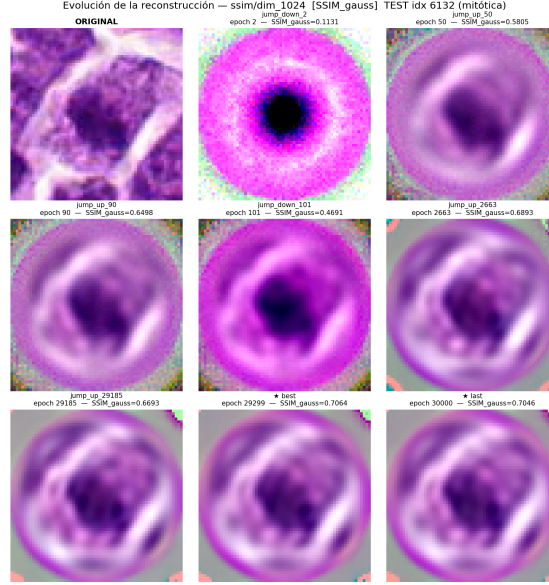


Figura 5.8: Trayectoria de reconstrucción del parche mitótico de test para `ssim_aug_1024`. Ningún checkpoint produce la reconstrucción trivial; el régimen estable se alcanza ya hacia la época 2663.

5.4 Análisis de la dispersión del código

ESTA sección profundiza en la dispersión de los códigos aprendidos sobre el conjunto de test, más allá de la evolución agregada ya vista en la sección 5.2.3: primero examina la distribución completa de átomos activos por muestra (sección 5.4.1) y después el compromiso que esa dispersión establece con la calidad de reconstrucción (sección 5.4.2), determinando qué configuraciones producen representaciones aptas para el análisis discriminativo posterior.

5.4.1 Distribución de átomos activos por muestra

Esta subsección examina, para las seis configuraciones y ambos checkpoints, la distribución completa de ℓ_0 sobre el test. La tabla 5.5 recoge los estadísticos y la figura 5.9 los histogramas: el nivel real de dispersión condiciona si las representaciones son lo bastante selectivas para las tareas discriminativas posteriores.

Para $m = 128$, las tres pérdidas muestran un patrón consistente: la distribución de ℓ_0 es marcadamente bimodal entre checkpoints, aunque cada checkpoint es en realidad unimodal por separado —una campana estrecha en ≈ 65 –70 para el checkpoint final y otra en ≈ 113 –118 para el óptimo—; es su superposición la que produce la apariencia bimodal. El checkpoint final se sitúa cerca del objetivo del 50%, confirmando que el óptimo se alcanza antes de que el controlador termine de converger, con un coste moderado en reconstrucción (sección 5.4.2).

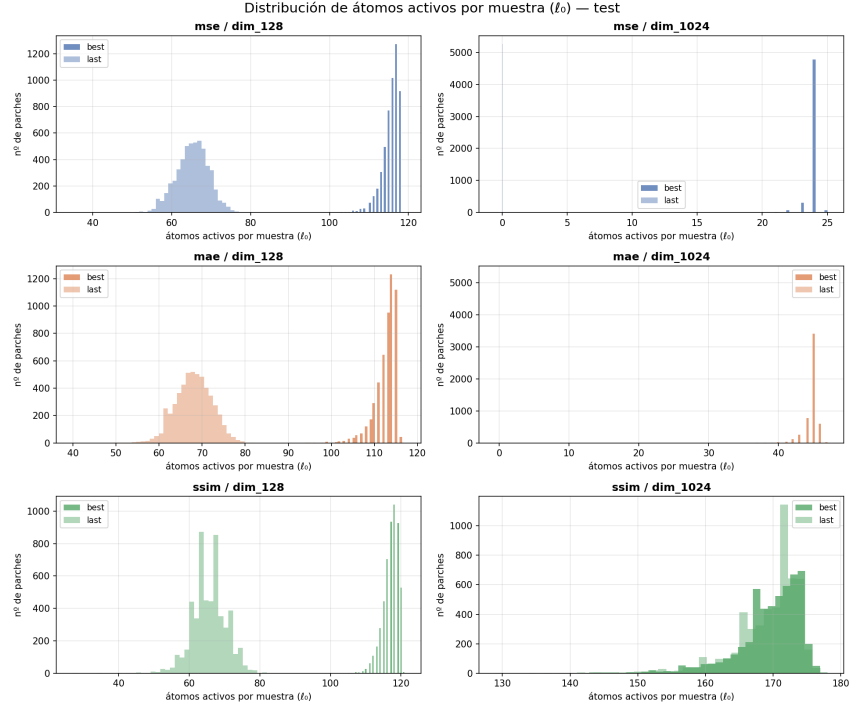


Figura 5.9: Distribución de átomos activos por muestra (ℓ_0) sobre el test, para las seis configuraciones y ambos checkpoints (óptimo en oscuro, final en claro). Filas: MSE, MAE, SSIM; columnas: $m = 128$ / $m = 1024$ (escalas distintas).

Para $m = 1024$, ninguna se acerca al objetivo ($\ell_0 \approx 512$). `mse_aug_1024` y `mae_aug_1024` muestran en el óptimo picos estrechos y aislados, y en el final colapsan a $\ell_0 = 0$ para el 100% de las muestras —el contraste más directo entre checkpoint funcional y degenerado—. `ssim_aug_1024` es la excepción: distribución alta, ancha y asimétrica en $\ell_0 \approx 167$ – 173 (16.5% del diccionario), con ambos checkpoints casi superpuestos, confirmando su estabilidad.

Como resultado adicional, los checkpoints finales de `mse_aug_1024` y `mae_aug_1024` no solo colapsan ambos a $\ell_0 = 0$, sino que producen métricas cruzadas *idénticas* hasta el quinto decimal: al no incorporar sesgo, el decoder produce necesariamente $\hat{\mathbf{x}} = \mathbf{0}$ con independencia de los pesos aprendidos cuando el código colapsa.

5.4.2 Compromiso reconstrucción–dispersión

La figura 5.10 sintetiza la relación entre dispersión y reconstrucción cruzada, en MSE_g (log) y SSIM_g ; las flechas conectan el checkpoint óptimo (oscuro) con el final (claro).

Para $m = 128$, las tres se comportan de forma similar: `frac_zeros` pasa de ≈ 0.09 – 0.12 a ≈ 0.47 – 0.49 (se quintuplica), con un descenso moderado de SSIM_g . El coste de quintuplicar la dispersión es, por tanto, moderado y consistente entre pérdidas.

Tabla 5.5: Estadísticos de dispersión del código y métricas de reconstrucción cruzada sobre el test, para las seis configuraciones y ambos checkpoints. Negrita: mejor resultado del bloque.

Configuración	ℓ_0 medio	std	frac_zeros	MSE_g ($\downarrow$)	MAE_g ($\downarrow$)	$SSIM_g$ ($\uparrow$)
Checkpoint óptimo						
mse_aug_128	115.58	2.24	0.097	0.0737	0.1973	0.6446
mae_aug_128	112.61	2.52	0.120	0.0776	0.1990	0.6432
ssim_aug_128	117.01	2.23	0.086	0.0837	0.2038	0.6658
mse_aug_1024	23.91	0.46	0.977	0.1783	0.3165	0.4133
mae_aug_1024	44.75	0.99	0.956	0.1292	0.2621	0.5080
ssim_aug_1024	169.47	4.95	0.835	0.0690	0.1789	0.7171
Checkpoint final (época 30 000)						
mse_aug_128	65.27	4.02	0.490	0.1052	0.2377	0.5604
mae_aug_128	68.12	4.15	0.468	0.1077	0.2332	0.5743
ssim_aug_128	65.59	4.80	0.487	0.1362	0.2566	0.5733
mse_aug_1024	0.00	0.00	1.000	1.2768	0.9416	0.0066
mae_aug_1024	0.00	0.00	1.000	1.2768	0.9416	0.0066
ssim_aug_1024	169.20	5.04	0.835	0.0676	0.1794	0.7148

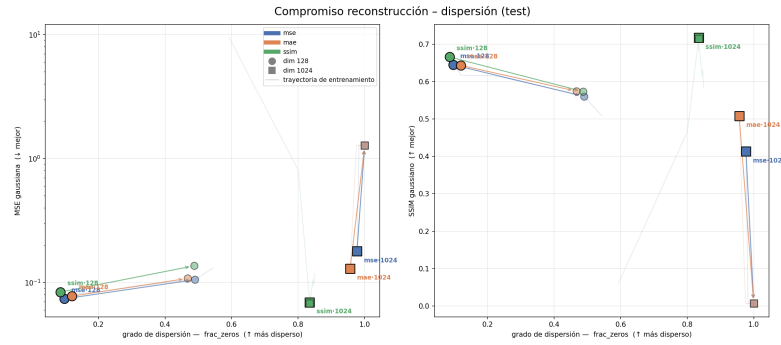


Figura 5.10: Compromiso entre dispersión (frac_zeros) y reconstrucción cruzada, en MSE_g (izq., escala log) y $SSIM_g$ (der.), para las seis configuraciones. Marcadores oscuros: checkpoint óptimo; claros: checkpoint final; flechas: sentido de la transición.

Para $m = 1024$, el compromiso se rompe de forma muy distinta según la pérdida. En $\text{mse}/\text{mae_aug_1024}$, el óptimo ya parte de dispersión muy alta con reconstrucción moderada, y el final empuja la dispersión al máximo mientras $SSIM_g$ se desploma a 0.0066 —la firma numérica exacta del colapso—. En ssim_aug_1024 , en cambio, frac_zeros permanece casi inalterado, con la mejor reconstrucción cruzada de las seis en ambos casos. Solo las tres de $m = 128$ y esta configuración resultan aptas para los análisis discriminativos siguientes.

Síntesis parcial. A $m = 128$, las seis configuraciones convergen a un compromiso reconstrucción-dispersión razonable en torno al 50%. A $m = 1024$, la dispersión deja de ser informativa por sí sola: MSE/MAE alcanzan dispersión total pero por colapso degenerado, mientras que $SSIM$ se estabiliza en una dispersión alta pero genuina. Esta distinción entre

dispersión "sana" y "por extinción" es clave para interpretar los resultados de las secciones siguientes.

5.5 Análisis del diccionario aprendido

ESTA sección examina directamente el diccionario aprendido —los átomos que lo componen, su descomposición en la reconstrucción de un parche concreto, su estado de salud sobre el test y su coherencia interna— para valorar si captura patrones morfológicos interpretables más allá de lo que indican las métricas agregadas de las secciones anteriores.

5.5.1 Visualización de los átomos más activos

Esta subsección examina el diccionario de forma directa. La figura 5.11 muestra, para `ssim_aug` en sus dos dimensiones, los 32 átomos con mayor magnitud media de activación $\text{act}_k = \frac{1}{N} \sum_n z_{nk}$ sobre 512 parches.¹

Para $m = 128$, el átomo dominante (#81) casi duplica al segundo y corresponde a un "átomo de fondo" casi uniforme; el resto combina manchas oscuras en anillos claros (coherentes con el núcleo). Para $m = 1024$, el dominante es #310 —el mismo ya identificado en la sección 5.3.2—, con mayor proporción de átomos casi uniformes entre los principales, coherente con la baja utilización del diccionario ya vista (sección 5.4.1).

5.5.2 Descomposición de un parche en átomos

Esta subsección ilustra, a nivel de una única muestra, cómo el código disperso se traduce en una combinación concreta de átomos del diccionario. Las figuras 5.12 y 5.13 muestran los seis átomos de mayor coeficiente en la reconstrucción de un mismo parche. En ambos casos el átomo dominante es casi uniforme, confirmando a nivel de muestra el papel de componente de fondo. El histograma central confirma la utilización del código: 119/128 activos (93.0%) frente a 155/1024 (15.1%).

5.5.3 Salud de los átomos sobre el conjunto de test

Esta subsección evalúa la salud del diccionario a nivel de átomo individual sobre el conjunto de test. Para cada átomo se mide su tasa de activación, clasificado como vivo (≥ 0.25), agonizante ($0.10-0.25$) o muerto (< 0.10).

Para `ssim_aug_128` (figura 5.14), la inmensa mayoría permanece viva (123 de 128, 96.1%) en el checkpoint óptimo; en el final los muertos suben a 28, consistente con la caída

¹ Muestreado sin semilla fija, combinando entrenamiento y test; la composición exacta puede variar, aunque los átomos dominantes son estables.

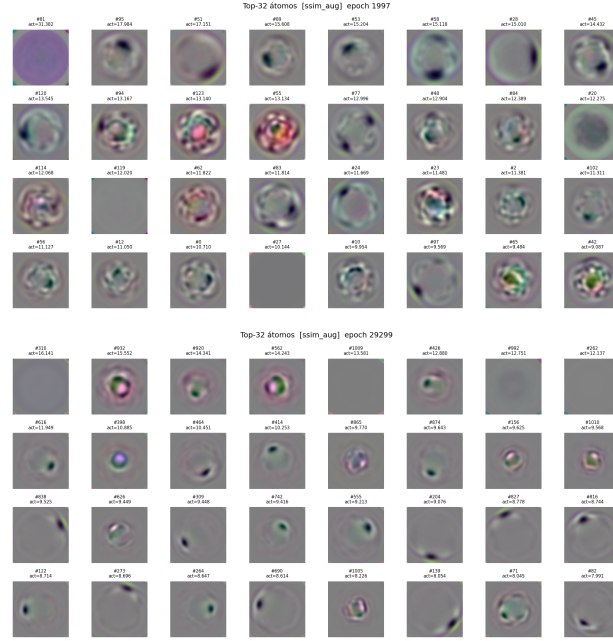


Figura 5.11: Los 32 átomos de mayor magnitud media de activación en `ssim_aug_128` (arriba) y `ssim_aug_1024` (abajo), ambas en el checkpoint óptimo. El átomo dominante a $m = 1024$ (#310) coincide con el ya identificado en las reconstrucciones individuales.

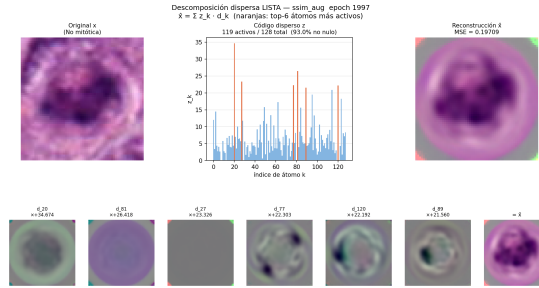


Figura 5.12: Seis átomos de mayor coeficiente en la reconstrucción de un parche, `ssim_aug_128` (checkpoint óptimo).

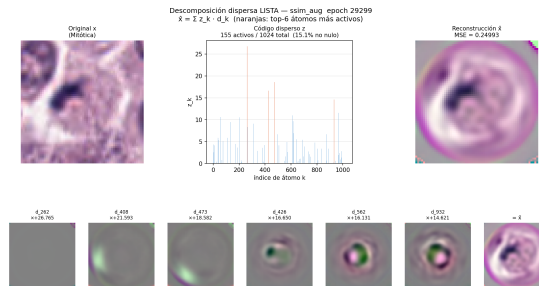


Figura 5.13: Seis átomos de mayor coeficiente en la reconstrucción de un parche, `ssim_aug_1024` (checkpoint óptimo).

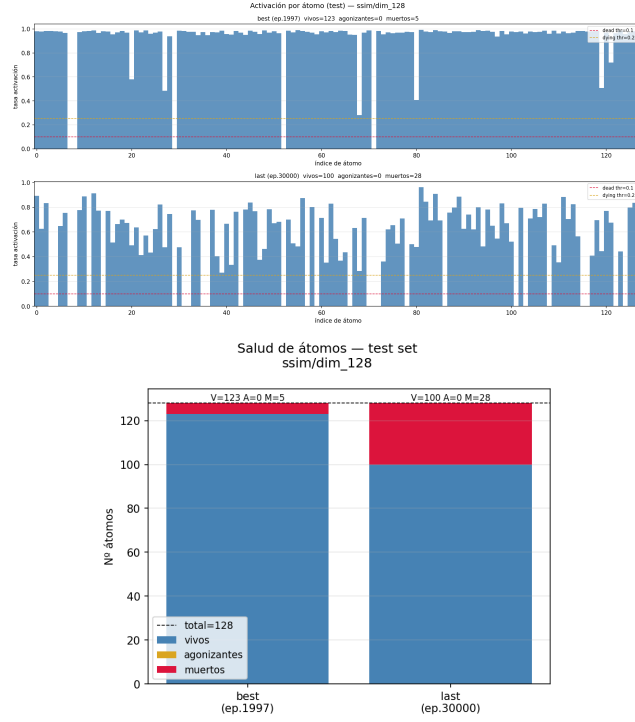


Figura 5.14: Tasa de activación por átomo (arriba) y recuento de átomos vivos/agonizantes/muertos (abajo) para `ssim_aug_128`.

de ℓ_0 medio ya vista. Para `ssim_aug_1024` (figura 5.15), solo 192 de 1024 están vivos (18.75%), con cifras **idénticas** entre ambos checkpoints. En términos absolutos, sin embargo, 192 vivos en $m = 1024$ supera a los ≤ 123 de $m = 128$: la escala mayor sostiene más capacidad funcional pese a la menor proporción utilizada.

5.5.4 Coherencia del diccionario

Esta subsección cierra el análisis del diccionario examinando su coherencia mutua, que mide el solapamiento entre átomos; valores bajos indican diccionario casi ortogonal. La tabla 5.6 recoge media, percentiles y máximo; las figuras 5.16 y 5.17 muestran la matriz para `ssim_aug`.

Para $m = 128$, las tres pérdidas producen diccionarios casi idénticos en el óptimo (coherencia 0.011–0.012), que se multiplica 4–6 $\times$ en el final al concentrar la energía en menos activos. Para $m = 1024$, mse/mae muestran diccionarios masivamente redundantes (hasta 0.60), mientras `ssim` se mantiene sustancialmente más ortogonal y estable (≈ 0.16), reproduciendo el patrón de estabilidad ya visto en dispersión y salud de átomos. Ambas configuraciones comparten el mismo número de muertos pero difieren en coherencia por un factor > 3 : la mayor redundancia de mse/mae es un efecto real, no un artefacto del recuento de muertos.

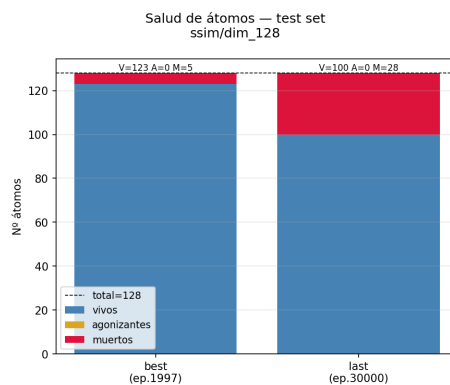


Figura 5.15: Recuento de átomos vivos/agonizantes/muertos para `ssim_aug_1024` (cifras idénticas entre ambos checkpoints).

Tabla 5.6: Coherencia mutua del diccionario para las seis configuraciones y ambos checkpoints. Negrita: mínimo del bloque (más ortogonal, más deseable).

Configuración	Ckpt.	media	p25	p50	p75	max
<i>m = 128</i>						
mse_aug	Óptimo	0.0110	0.0034	0.0073	0.0129	0.9872
mse_aug	Final	0.0601	0.0188	0.0412	0.0788	0.9948
mae_aug	Óptimo	0.0120	0.0039	0.0084	0.0147	0.9900
mae_aug	Final	0.0531	0.0162	0.0354	0.0658	0.9943
ssim_aug	Óptimo	0.0116	0.0034	0.0072	0.0132	0.9908
ssim_aug	Final	0.0687	0.0219	0.0486	0.0904	0.9873
<i>m = 1024</i>						
mse_aug	Óptimo	0.4993	0.4001	0.4819	0.6085	0.9951
mse_aug	Final	0.6025	0.3770	0.6531	0.8628	0.9995
mae_aug	Óptimo	0.3148	0.1322	0.2726	0.4049	0.9937
mae_aug	Final	0.5688	0.3261	0.6123	0.8319	0.9992
ssim_aug	Óptimo	0.1606	0.0288	0.1391	0.2466	0.9942
ssim_aug	Final	0.1646	0.0291	0.1459	0.2552	0.9938

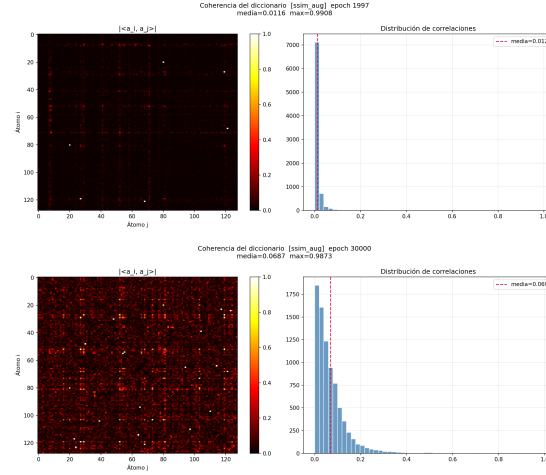


Figura 5.16: Matriz de coherencia para $ssim_aug_128$, en el checkpoint óptimo (arriba) y el checkpoint final (abajo).

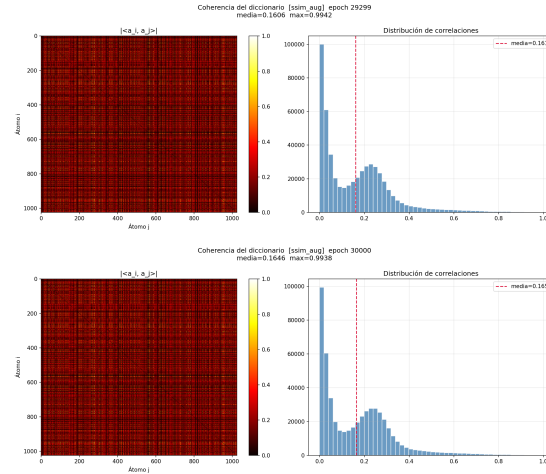


Figura 5.17: Matriz de coherencia para $ssim_aug_1024$, en el checkpoint óptimo (arriba) y el checkpoint final (abajo). Escala notablemente más alta que a $m = 128$ y práctica invariancia entre checkpoints.

Síntesis parcial. El examen del diccionario refuerza el patrón de dispersión: los diccionarios de $m = 128$ son compactos, mayoritariamente vivos y casi ortogonales con independencia de la pérdida. A $m = 1024$, el diccionario aprendido bajo SSIM conserva más átomos vivos en términos absolutos que a $m = 128$, con una coherencia notablemente más baja que MSE/MAE, lo que indica que aprovecha mejor el espacio disponible sin caer en la redundancia masiva de las otras dos pérdidas. Esto sitúa a SSIM como la única configuración de $m = 1024$ que llega en condiciones sólidas al análisis discriminativo.

5.6 Potencial discriminativo de las representaciones

ESTA sección analiza si los códigos dispersos aprendidos de forma no supervisada capturan información discriminativa entre figuras mitóticas y no mitóticas, combinando primero la activación media por átomo y tres criterios de selección de atributos (sección 5.6.1), y después una visualización directa de la estructura del espacio de código (sección 5.6.2) para comprobar si esa señal estadística se traduce en una separación visible entre clases.

5.6.1 Activación media por clase y selección de atributos

Esta subsección examina qué átomos separan mejor las clases mitótica y no mitótica, combinando el *índice de Fisher*, el *F-score de ANOVA* y la *información mutua* (MI). Miden la misma pregunta desde ángulos distintos, por lo que su grado de acuerdo es en sí mismo informativo.

La figura 5.18 muestra el índice de Fisher por átomo y el ranking de los 16 más discriminativos; la tabla 5.7 recoge el átomo top de cada configuración —sin solapamiento de identidad entre configuraciones, como cabe esperar de diccionarios entrenados de forma independiente.

El resultado más relevante es el rango normalizado de cada criterio. A $m = 128$, el rango de ANOVA-F del átomo top de Fisher es casi máximo: Fisher y ANOVA-F identifican esencialmente el mismo átomo. A $m = 1024$, el rango de ANOVA-F se desploma pese a mantener un valor *absoluto* similar: al escalar el diccionario, Fisher/ANOVA-F pasan a señalar zonas distintas. La MI, en cambio, mantiene rangos altos y estables en ambas dimensiones, consolidándose como el criterio de mayor concordancia relativa.

5.6.2 Visualización del espacio de código

Esta subsección comprueba si la señal discriminativa es visible en la estructura del espacio de código, mediante correlación entre átomos por clase, boxplots, dispersión conjunta por pares, y proyecciones PCA/t-SNE.

La figura 5.20 (mse_aug_128) muestra que las matrices de correlación entre átomos por clase son prácticamente indistinguibles: la coactivación entre pares no cambia apreciablemente entre clases, pese a existir átomos individuales con activación media distinta. Los

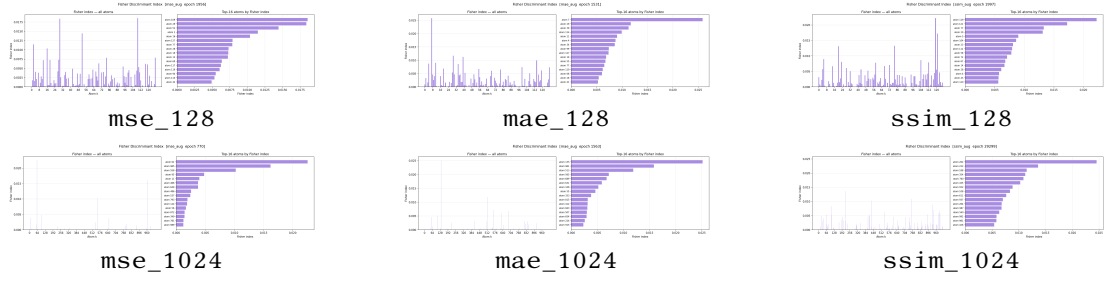


Figura 5.18: Índice de Fisher por átomo y ranking de los 16 más discriminativos, para las seis configuraciones (checkpoint óptimo).

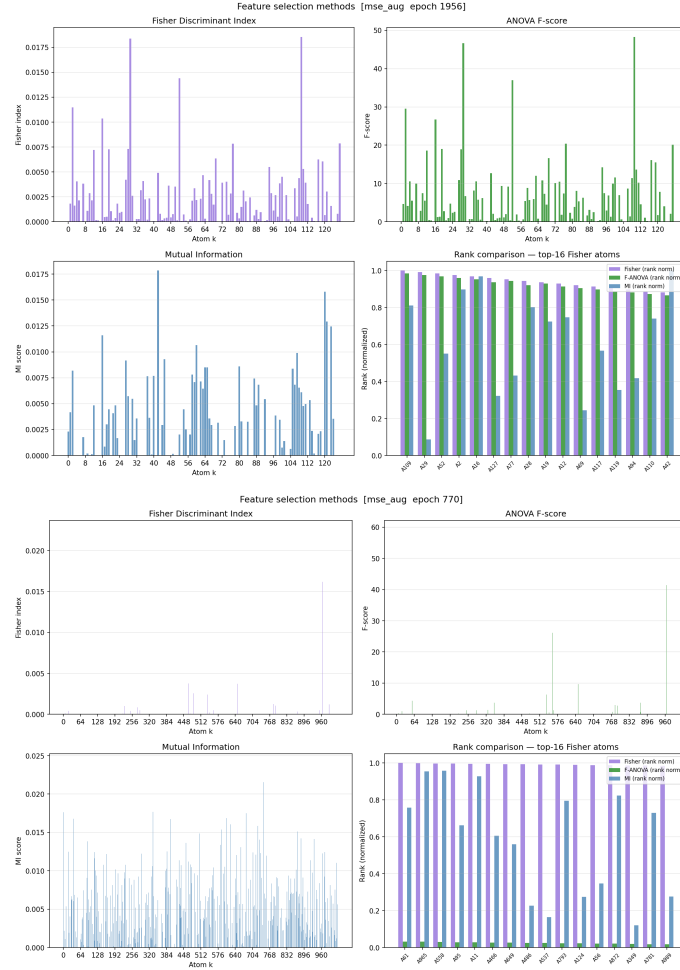


Figura 5.19: Comparativa de los tres criterios de selección para mse_aug , a $m = 128$ (arriba, alto acuerdo Fisher/ANOVA-F) y $m = 1024$ (abajo, acuerdo casi nulo).

Tabla 5.7: Átomo de mayor índice de Fisher en cada configuración (checkpoint óptimo), con su valor absoluto y rango normalizado en los tres criterios. El rango de ANOVA-F se desploma al pasar de $m = 128$ a $m = 1024$, pese a un valor absoluto de F-ANOVA similar.

Config.	Átomo	Fisher	F-ANOVA	MI	rank_fisher	rank_anova	rank_mi
$m = 128$							
mse_aug	109	0.01856	48.30	0.00610	1.000	0.984	0.811
mae_aug	7	0.02582	65.79	0.00754	1.000	1.000	0.898
ssim_aug	119	0.02221	56.67	0.00000	1.000	0.961	0.339
$m = 1024$							
mse_aug	61	0.02255	59.19	0.00494	1.000	0.032	0.759
mae_aug	135	0.02514	65.34	0.01487	1.000	0.070	0.978
ssim_aug	262	0.02453	62.47	0.01134	1.000	0.188	0.942

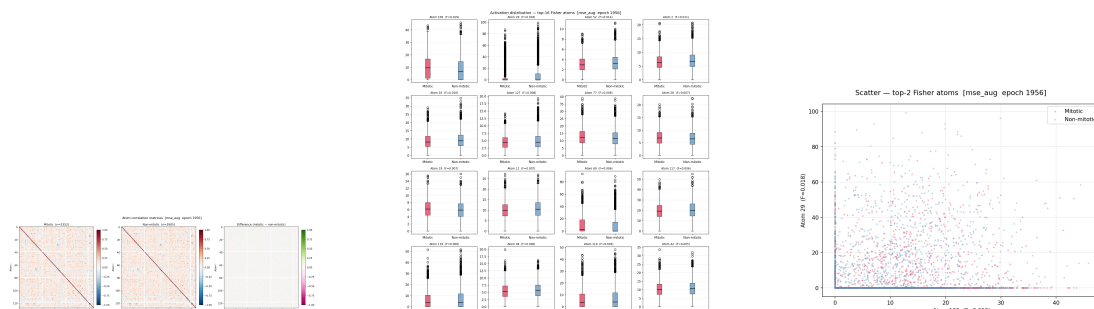


Figura 5.20: Estructura del espacio de código por clase para mse_aug_128 (checkpoint óptimo): correlación entre átomos por clase (izq.), boxplots de los 16 átomos de mayor Fisher (centro), dispersión de los dos más discriminativos (der.).

boxplots y el diagrama de dispersión confirman el solapamiento: ninguna agrupación separable.

Esta falta de separación se confirma a nivel agregado en la figura 5.21 (PCA y t-SNE): las clases aparecen densamente entremezcladas en las cuatro proyecciones, incluso en la configuración más favorable, sin clústeres correspondientes a la etiqueta de clase.

En conjunto, aunque existen átomos con diferencias de activación estadísticamente detectables entre clases, esa señal es débil frente a la varianza intra-clase y no produce separación visualmente reconocible: cualquier capacidad discriminativa presente tendría que emerger de la combinación no lineal y distribuida de muchos átomos.

Síntesis parcial. El resultado más cauteloso del capítulo: existen átomos individuales con diferencias de activación estadísticamente detectables entre clases, identificados de forma consistente por los tres criterios, pero esa señal es débil frente a la varianza intra-clase y no se traduce en ninguna separación visible en el espacio de código, ni siquiera en la configuración más favorable. El desacople entre Fisher/ANOVA-F al escalar el diccionario, frente a la estabilidad de la información mutua, sugiere además que la elección del criterio de selección no es neutra en espacios de alta dimensión.

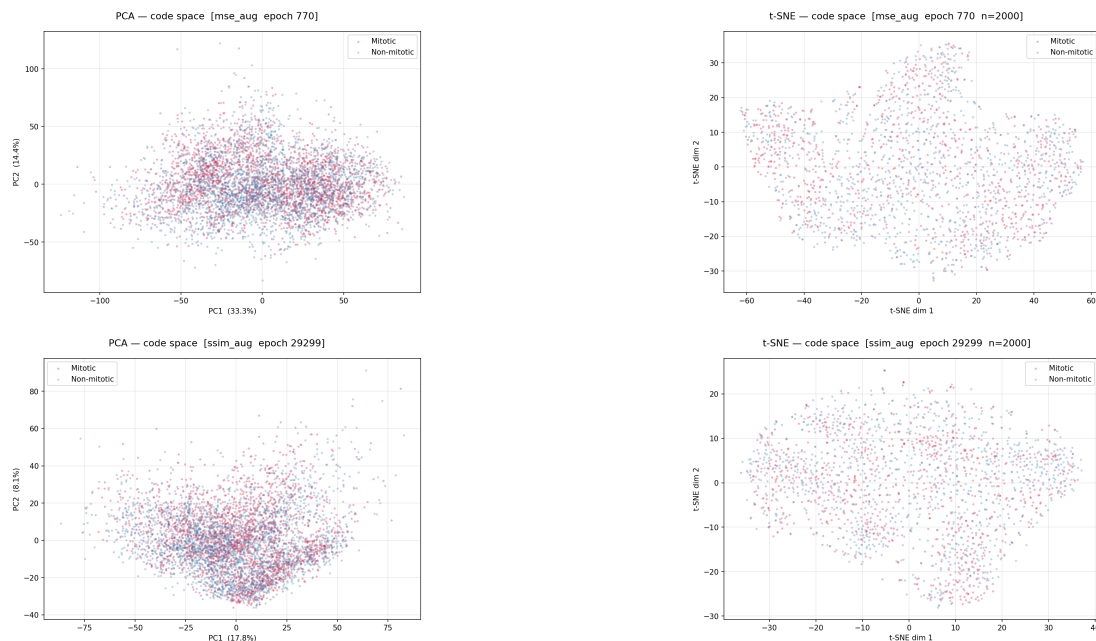


Figura 5.21: Proyecciones PCA (izq.) y t-SNE (der.) del espacio de código completo, para `mse_aug_1024` (arriba) y `ssim_aug_1024` (abajo), ambas en el checkpoint óptimo. Ninguna muestra separación reconocible entre clases.

5.7 Efecto del tamaño del diccionario

ESTA sección recopila y sintetiza los hallazgos transversales de las secciones anteriores para responder directamente a qué efecto tiene multiplicar por ocho la dimensión del diccionario, revisando en orden la reconstrucción, la dispersión, la coherencia y la selección de atributos, antes de cerrar con una síntesis global que blindas las ideas ya adelantadas a lo largo del capítulo.

Reconstrucción y estabilidad de la trayectoria. A $m = 128$, las tres funciones de pérdida convergen a checkpoints óptimos razonablemente pronto y con trayectorias mayormente suaves (figuras A.7–A.9), con la excepción de un plateau subóptimo persistente en `mse_aug_128` tras un salto. A $m = 1024$, la respuesta se bifurca de forma tajante: `mse/mae_aug_1024` colapsan a una reconstrucción trivial en la práctica totalidad de su trayectoria (figuras 5.7 y A.10), mientras que `ssim_aug_1024` no solo evita el colapso sino que alcanza la mejor reconstrucción cruzada de las seis configuraciones ($SSIM_g \approx 0.72$).

Dispersión del código. El controlador alcanza su objetivo del 50% únicamente a $m = 128$, y solo en el checkpoint final. A $m = 1024$, `mse` y `mae` terminan con dispersión total (colapso), mientras `ssim` se estabiliza en torno al 83% de átomos inactivos sin degenerar. En términos absolutos, el número de átomos vivos en $m = 1024$ es superior al de $m = 128$ para

la única configuración sana (192 vivos en `ssim_aug_1024` frente a un máximo de 123 en `ssim_aug_128`): la escala mayor sostiene más capacidad funcional en términos absolutos, aunque la proporción utilizada sea menor.

Coherencia del diccionario. La coherencia mutua media es baja y prácticamente uniforme entre pérdidas a $m = 128$ en el checkpoint óptimo, y aumenta de forma moderada hacia el final. A $m = 1024$, la coherencia se dispara para mse y mae, mientras ssim se mantiene en un régimen sustancialmente más ortogonal —el mismo patrón de bifurcación por pérdida que en reconstrucción y dispersión.

Selección de atributos discriminativos. A $m = 128$, Fisher y ANOVA-F identifican esencialmente el mismo átomo en las tres pérdidas. A $m = 1024$, ambos criterios se desacoplan drásticamente pese a mantener valores absolutos similares. La información mutua es el único criterio que mantiene un acuerdo relativo alto en ambas dimensiones. Consistentemente, ninguna configuración muestra separación por clase visible ni a $m = 128$ ni a $m = 1024$, lo que sugiere que este desacople de criterios no se traduce en una mejora ni un empeoramiento neto de la discriminabilidad global entre dimensiones.

Síntesis. El efecto dominante de multiplicar por ocho la dimensión del diccionario no es gradual, sino que introduce un punto de bifurcación cualitativo según la función de pérdida empleada: MSE y MAE, robustas a $m = 128$, colapsan de forma casi total a $m = 1024$ en reconstrucción, dispersión y coherencia por igual, mientras que SSIM se mantiene funcional —e incluso mejora en reconstrucción cruzada— en ambas dimensiones. Este patrón, consistente a lo largo de cinco análisis independientes, constituye uno de los hallazgos centrales del capítulo: la elección de la función de pérdida condiciona de forma determinante si el modelo escala favorablemente, antes incluso de considerar su potencial discriminativo.

5.8 Discusión

Los resultados de este capítulo sitúan la elección de la función de pérdida y el tamaño del diccionario como los dos factores que más condicionan la viabilidad del sistema propuesto, muy por encima del potencial discriminativo de las representaciones, que resulta positivo pero modesto. Esta sección discute cuatro implicaciones que se derivan de los resultados anteriores, antes de cerrar el capítulo y dar paso a las conclusiones del trabajo.

¿Un fallo de MSE/MAE, o un fallo de calibración del controlador? A lo largo del capítulo se ha repetido una misma observación: el colapso del diccionario a $m = 1024$ bajo MSE y MAE coincide siempre con que λ_{L1} nunca logra despegar de su semilla 10^{-7} . Esto plantea una pregunta que los datos no permiten cerrar del todo: ¿es el colapso una consecuencia intrínseca de optimizar estas pérdidas sobre un diccionario grande, o simplemente el resultado de que la semilla y el paso de crecimiento —calibrados sobre $m = 128$ — resultan

demasiado conservadores al escalar un orden de magnitud? La segunda hipótesis es la más verosímil: el propio mecanismo por el que λ_{L1} debería crecer queda inutilizado en cuanto el código colapsa a cero, generando un punto fijo degenerado del que el controlador no puede escapar. De ser así, la superioridad observada de **SSIM** a $m = 1024$ no reflejaría necesariamente una propiedad intrínseca de esta función de pérdida, sino que **SSIM** —al alcanzar de partida una fracción de activos más próxima al objetivo— nunca llega a caer en ese punto fijo. Esto limita la conclusión de que **SSIM** escala mejor a las condiciones concretas de este trabajo, y no permite descartar que una recalibración del controlador habilite también a **MSE** y **MAE** a $m = 1024$.

La selección del checkpoint no es un detalle menor. El contraste sistemático entre ambos checkpoints ha mostrado que pueden diferir sustancialmente en dispersión, calidad de reconstrucción, e incluso en si el diccionario está vivo o colapsado. El caso más claro es `mse_aug_128`, donde el checkpoint óptimo corresponde a un plateau temprano que nunca se recupera tras un salto posterior: una comparación limitada al checkpoint final habría subestimado la capacidad real del modelo. Esto sugiere que cualquier criterio de selección basado en un único punto de la trayectoria —incluida la práctica habitual de reportar solo el mejor checkpoint— puede dar una imagen incompleta o engañosa del comportamiento del sistema.

El potencial discriminativo es real pero débil. Existen átomos con diferencias de activación estadísticamente detectables entre clases, identificados mediante Fisher, ANOVA-F e información mutua. Sin embargo, esa señal es débil frente a la varianza intra-clase y no produce una separación visible en el espacio de código, ni siquiera en la configuración más favorable. Los tres criterios coinciden en identificar señal estadísticamente significativa en átomos individuales, lo que indica que la información discriminativa presente, aunque débil, es consistente entre distintas formas de medirla.

Limitaciones del alcance experimental. Todos los resultados proceden de un único dataset (**MIDOG**) y de dos dimensiones del diccionario, elegidas precisamente por representar dos extremos bien diferenciados con el fin de exponer con claridad el efecto del tamaño del diccionario. Esta elección deja sin explorar el rango intermedio, por lo que no es posible determinar con precisión en qué punto se sitúa la bifurcación observada, ni si la transición es abrupta o gradual; estudiar dimensiones intermedias en trabajos futuros permitiría acotar mejor ese punto. Esta limitación no invalida los hallazgos —el contraste entre configuraciones es interno y consistente—, pero sí acota su alcance a las condiciones concretas evaluadas en este trabajo.

En conjunto, los resultados de este capítulo confirman que el autoencoder disperso basado en **LISTA** propuesto es capaz de aprender representaciones dispersas razonablemente fieles y con indicios de contenido discriminativo, pero que su viabilidad a mayor escala depende de forma crítica de decisiones de diseño —función de pérdida, calibración del controlador de

dispersión— que este trabajo ha caracterizado empíricamente sin llegar a resolver por completo. Estas cuestiones abiertas, junto con las líneas de trabajo futuro que se derivan de ellas, se retoman en el Capítulo 6.

Conclusiones

ESTE último capítulo recoge la situación final del trabajo, las lecciones aprendidas durante su desarrollo, la relación con las competencias de la titulación –Grado en Ciencia e Ingeniería de Datos, Universidade da Coruña– y de la mención cursada, y las posibles líneas futuras que se derivan de los resultados obtenidos.

6.1 Situación final del trabajo

El objetivo principal del trabajo –el estudio y desarrollo de autoencoders dispersos basados en la arquitectura [LISTA](#) para la representación de parches de histología [H&E](#), explorando el efecto de distintas funciones de pérdida de reconstrucción sobre la calidad y el potencial discriminativo de las representaciones obtenidas– se ha alcanzado de forma sustancial, con resultados que en varios puntos han resultado más matizados, y en ocasiones más informativos, de lo que una lectura superficial de las métricas agregadas habría sugerido.

Respecto a los cinco objetivos específicos planteados en el capítulo 1:

1. **Diseño e implementación del modelo.** Se ha implementado un autoencoder [LISTA](#) completo –encoder iterativo con umbralización aprendida, decoder de pesos atados, máscara Gaussiana centrada, controlador automático de dispersión de dos fases, mecanismo de reactivación de átomos muertos y gestión de checkpoints por eventos de salto– junto con la infraestructura de entrenamiento, evaluación y visualización necesaria para producir los resultados del capítulo 5. El sistema ha demostrado ser robusto en la configuración de referencia ($m = 128$) y ha revelado, precisamente por su robustez en ese régimen, los límites del enfoque cuando se le exige escalar.
2. **Exploración de funciones de pérdida de reconstrucción.** La comparación sistemática de [MSE](#), [MAE](#) y [SSIM](#) ha sido, junto con el efecto del tamaño del diccionario, el hallazgo más determinante del trabajo. A $m = 128$ las tres funciones se comportan de

forma muy similar entre sí en prácticamente todos los ejes analizados –reconstrucción, dispersión, coherencia y selección de atributos–, lo que podría llevar a concluir que la elección de la función de pérdida es poco relevante. El experimento a $m = 1024$ desmiente esa conclusión: **MSE** y **MAE** colapsan el código a una salida trivial en la práctica totalidad de su trayectoria de entrenamiento, mientras que **SSIM** no solo evita el colapso sino que mejora su calidad de reconstrucción respecto a $m = 128$. Este contraste coincide sistemáticamente con que el controlador de dispersión nunca llega a activarse para **MSE/MAE** a $m = 1024$ (sección 5.2.2), por lo que, como se discute en la sección 5.8, no puede descartarse que la ventaja observada de **SSIM** responda, al menos en parte, a una calibración del controlador ajustada para $m = 128$, y no a una propiedad intrínsecamente superior de su estructura perceptual.

3. **Evaluación cuantitativa de los modelos.** Las seis configuraciones se han evaluado de forma sistemática y comparable sobre un conjunto de test compartido, mediante métricas de reconstrucción (MSE_g , MAE_g , PSNR, $SSIM_g$), de dispersión (ℓ_0 , `frac_zeros`) y de coherencia del diccionario, tanto en valores agregados como en su evolución completa a lo largo del entrenamiento (sección 5.3.3). Esta evaluación temporal, más allá de la comparación habitual entre el checkpoint óptimo y el checkpoint final, ha sido decisiva para detectar que el colapso a $m = 1024$ no es un artefacto puntual del final del entrenamiento, sino el estado que domina casi toda la trayectoria salvo una ventana estrecha.
4. **Análisis del potencial discriminativo.** Los resultados en este eje son los más cautelosos del trabajo. Existen átomos individuales con diferencias de activación estadísticamente detectables entre parches mitóticos y no mitóticos (sección 5.6.1), pero esa señal es débil frente a la varianza intra-clase y no se traduce en una separación visualmente reconocible del espacio de código, ni en diagramas de dispersión por pares de átomos ni en proyecciones agregadas mediante PCA o t-SNE (sección 5.6.2), incluso en la configuración con mejor comportamiento global (`ssim_aug_1024`). Esto no permite descartar que el código disperso capture información discriminativa útil mediante la combinación no lineal de muchos átomos débilmente informativos, pero sí obliga a matizar cualquier afirmación de que las representaciones aprendidas de forma no supervisada separan las clases de interés de forma directa o trivial.
5. **Análisis del efecto del tamaño del diccionario.** Este objetivo ha producido el resultado más claro y quizá menos esperado del trabajo: el efecto de escalar el diccionario de $m = 128$ a $m = 1024$ no es gradual, sino un punto de bifurcación cualitativo que depende por completo de la función de pérdida empleada (sección 5.7). Un mismo cambio de hiperparámetro –multiplicar por ocho el tamaño del código– resulta prácticamente

irrelevante para *SSIM* y catastrófico para *MSE* y *MAE*, un resultado que subraya que la elección de la función de pérdida y la capacidad del modelo no son decisiones independientes entre sí.

6.2 Lecciones aprendidas

Más allá de los resultados concretos, el desarrollo del trabajo ha dejado varias lecciones de carácter más general.

La evaluación de un único checkpoint es insuficiente. La comparación aislada entre el checkpoint óptimo y el checkpoint final, habitual en la literatura, habría ocultado fenómenos centrales de este trabajo: la pérdida de un plateau de reconstrucción tras un evento de salto en $m = 128$, y el colapso casi total del código en $m = 1024$ (Capítulo 5), ninguno de los dos visible si solo se hubiese registrado el extremo final del entrenamiento. Reconstruir la trayectoria completa, y no solo sus dos extremos, ha resultado imprescindible para interpretar correctamente los resultados.

Los criterios de selección de atributos no son intercambiables. La comparación de Fisher, ANOVA-F e información mutua ha mostrado que, aunque estos criterios coinciden en gran medida en el régimen de baja dimensión, se desacoplan sustancialmente al escalar el diccionario. Esto advierte contra el uso de un único criterio de discriminabilidad sin contrastarlo con alternativas, especialmente en espacios de alta dimensión y baja utilización efectiva.

La reproducibilidad exige disciplina desde el diseño experimental. Varias de las visualizaciones generadas durante el trabajo –la selección de parches de ejemplo, el muestreo de subconjuntos para el cálculo de activaciones medias– se implementaron inicialmente sin semilla fija y, en algún caso, mezclando muestras de entrenamiento y de test. Aunque no invalida las conclusiones cuantitativas del trabajo, que sí se apoyan en el conjunto de test completo y consistente, sí ha exigido documentar con precisión el alcance real de cada figura para evitar sobreinterpretar ejemplos ilustrativos como si fueran evidencia sistemática.

Un resultado negativo bien caracterizado es un resultado útil. El colapso del código a $m = 1024$ bajo *MSE* y *MAE* podría presentarse como una limitación del trabajo, pero su caracterización detallada –identificación de la época de colapso, coincidencia exacta de las métricas del checkpoint final entre ambas pérdidas por convergencia al mismo código nulo, contraste sistemático con el comportamiento de *SSIM* en los mismos ejes de análisis– constituye en sí misma una contribución: no solo se documenta que el modelo falla en ciertas condiciones, sino por qué y bajo qué mecanismo concreto lo hace.

6.3 Relación con las competencias de la titulación

El trabajo se enmarca en el Grado en Ciencia e Ingeniería de Datos de la Universidad de Coruña, y ha permitido poner en práctica de forma integrada competencias transversales de la titulación y competencias específicas de su mención.

Desde el punto de vista de la **ingeniería de datos**, el trabajo ha requerido diseñar un pipeline completo de procesamiento –desde el preprocesado del dataset **MIDOG** y la extracción de parches hasta la generación sistemática de resultados evaluables para seis configuraciones experimentales–, gestionar volúmenes de datos y resultados intermedios de tamaño considerable, y automatizar la evaluación reproducible de múltiples modelos bajo un protocolo común.

Desde el punto de vista de la **ciencia de datos** y el aprendizaje automático, se han aplicado y comprendido en profundidad técnicas de aprendizaje de representaciones no supervisado (codificación dispersa, arquitecturas desenrolladas tipo **LISTA**), métricas de evaluación de reconstrucción y de discriminabilidad (Fisher, ANOVA-F, información mutua), técnicas de reducción de dimensionalidad para visualización (**PCA**, t-SNE) y clasificación supervisada como herramienta de validación. El trabajo ha exigido además una interpretación estadística cuidadosa –distinguir entre significancia y relevancia práctica, entre correlación agregada y separabilidad visual, entre un resultado favorable puntual y un comportamiento estable– que es central a la formación en ciencia de datos.

El dominio de aplicación –imágenes histopatológicas y detección de figuras mitóticas– ha permitido además conectar estas competencias técnicas con un problema de relevancia clínica real, reforzando la importancia de evaluar los modelos no solo por sus métricas internas, sino por su potencial utilidad en la tarea final para la que están pensados.

6.4 Líneas futuras

De los resultados obtenidos se derivan varias líneas de trabajo futuro que no ha sido posible abordar dentro del alcance de este TFG.

- **Mitigar el colapso a dimensiones altas bajo MSE y MAE.** Dado que el mecanismo de reactivación de átomos muertos (`revival_every`, sección 5.2.4) estuvo desactivado en los experimentos reportados, una línea directa de continuación es evaluar si su activación –u otras estrategias de regularización explícita del diccionario, como una penalización de coherencia durante el entrenamiento– permite que **MSE** y **MAE** escalen a $m = 1024$ sin colapsar, cerrando la brecha observada frente a **SSIM**.
- **Explorar dimensiones intermedias del diccionario.** El trabajo ha comparado únicamente $m = 128$ y $m = 1024$; queda abierto si el punto de bifurcación observado es

abrupto o si existe una transición gradual en dimensiones intermedias, lo que permitiría caracterizar con mayor precisión el umbral de estabilidad de cada función de pérdida.

- **Profundizar en el análisis discriminativo mediante aprendizaje supervisado.** Una línea natural de continuación es entrenar clasificadores sobre el código disperso obtenido y compararlos con una referencia supervisada entrenada directamente sobre los parches, lo que permitiría cuantificar de forma directa el potencial discriminativo que este trabajo solo ha podido explorar de forma cualitativa. Esto abriría además la puerta a estudiar técnicas de *fine-tuning* del diccionario con supervisión parcial, o a combinar la representación dispersa no supervisada con un número reducido de anotaciones, en línea con el interés original del enfoque para escenarios con anotación escasa.
- **Extender la comparación a otras funciones de pérdida perceptuales.** Dado que la superioridad de *SSIM* parece residir en su naturaleza perceptual y no puntual, sería natural extender la comparación a otras familias de pérdidas con propiedades similares (por ejemplo, pérdidas basadas en características extraídas de redes preentrenadas), para determinar si la resistencia al colapso observada es una propiedad específica de *SSIM* o compartida por otras pérdidas perceptuales.
- **Validar la generalización a otros conjuntos de datos.** Todos los experimentos se han realizado sobre parches derivados del dataset *MIDOG*; una validación en otros conjuntos de histología *H&E*, o en dominios de imagen médica distintos, permitiría determinar si las conclusiones sobre la interacción entre función de pérdida y tamaño del diccionario son específicas de este dataset o generalizables.

Apéndices

Material adicional

ESTE apéndice recoge material complementario al cuerpo principal de la memoria: figuras, tablas y detalles adicionales que, por motivos de extensión o de nivel de detalle, no tienen cabida en el capítulo de experimentos y resultados, pero que pueden ser de interés para una lectura más exhaustiva del trabajo.

A.1 Detalle de los episodios de recuperación espontánea en $m = 1024$

Esta sección amplía la sección 5.2.1 con el detalle completo de los episodios de recuperación parcial y espontánea observados tras el colapso del diccionario en `mse_aug_1024` y `mae_aug_1024`. En ambos casos el mecanismo de revival está desactivado tras el calentamiento, por lo que estos episodios corresponden a inestabilidades intrínsecas del entrenamiento en un régimen degenerado, y no a reinicializaciones deliberadas.

mse_aug_1024. Tras el colapso mayor hacia las épocas 5319–5326 (repunte transitorio hasta 632 átomos vivos), se registran dos episodios aislados adicionales: épocas 17 337–17 364 (hasta 26 átomos vivos) y épocas 27 119–27 138 (hasta 312 átomos vivos, con un pico de $\text{rec} = 79\,346$ en la época 27 131). En todos los casos la trayectoria retorna en pocas épocas al estado colapsado (0 átomos vivos).

mae_aug_1024. Se observan cinco episodios: dos menores hacia las épocas 2305–2352 y 2567–2626 (máximo 138 átomos vivos); uno más severo hacia las épocas 15 673–15 727 (pico $\text{rec} = 9.02$, máximo 645 átomos vivos); y dos adicionales hacia las épocas 23 758–23 858 (máximo 233 vivos) y 25 632–25 660 (máximo 603 vivos, $\text{rec} = 6.54$). Como en el caso anterior, la trayectoria retorna en pocas decenas de épocas al suelo colapsado ($\text{rec} \approx 1.00$ – 1.01 , 0 átomos vivos).

Detalle de eventos de salto en mse_aug_1024 y mae_aug_1024 . Los eventos de salto de ambas configuraciones se concentran en los tramos de recuperación parcial descritos anteriormente en esta sección. A modo de ejemplo, hacia el final del entrenamiento de mse_aug_1024 los eventos registrados ya no reflejan reorganizaciones significativas, sino fluctuaciones de ruido numérico en torno al suelo colapsado: el evento de las épocas 29 973–29 974 registra un cambio de $rec = 1.436$ a $rec = 1.439$, una variación ínfima en términos absolutos que supera, sin embargo, el umbral relativo del 25% al estar calculada sobre valores ya muy próximos entre sí.

A.2 Reconstrucciones cualitativas complementarias

Esta sección amplía la sección 5.3.2 con las reconstrucciones cualitativas de mae_aug_128 , $ssim_aug_128$ y mae_aug_1024 , cuyo patrón general ya se resumió en el cuerpo principal.

mae_aug_128 . Reproduce el patrón cualitativo de mse_aug_128 (pérdida de textura fina, asimetría entre clases: $MAE_g = 0.2261$ mitótica frente a 0.1325 no mitótica en el checkpoint óptimo). La diferencia más notable está en los átomos individuales más activos, que presentan una estructura interna más rica —varias regiones de color diferenciadas en patrones concéntricos—, coherente con el mayor λ_{L1} de equilibrio de esta configuración (sección 5.2.2). En el checkpoint final se aprecia además un ligero oscurecimiento de las esquinas de la reconstrucción, coherente con el escaso peso de la máscara Gaussiana en la periferia. La figura A.1 muestra la muestra mitótica, y la figura A.2 la muestra no mitótica.

$ssim_aug_128$. Mantiene la asimetría entre clases ($SSIM_g = 0.6520$ mitótica frente a 0.7385 no mitótica en el checkpoint óptimo). Destaca por artefactos de color en las cuatro esquinas del parche, coherentes con el peso casi nulo de la máscara Gaussiana ahí, y por una jerarquía de átomos más desigual: el átomo dominante (átomo 81 en ambas muestras) es un disco casi uniforme con un coeficiente cercano al doble del segundo átomo más activo, lo que sugiere que el diccionario delega la luminancia de fondo en un único átomo casi constante. Las figuras A.3 y A.4 muestran ambas muestras de test.

mae_aug_1024 . Presenta el mismo patrón de colapso que mse_aug_1024 (sección 5.3.2), aunque con un checkpoint óptimo (época 1563) de calidad algo superior, apoyado en átomos de magnitud más moderada ($|z| \approx 27\text{--}42$), coherente con que en ese punto el diccionario aún conserva un número razonable de átomos vivos. En el checkpoint final, la reconstrucción es de nuevo un color uniforme idéntico para ambas clases. Las figuras A.5 y A.6 ilustran este patrón.

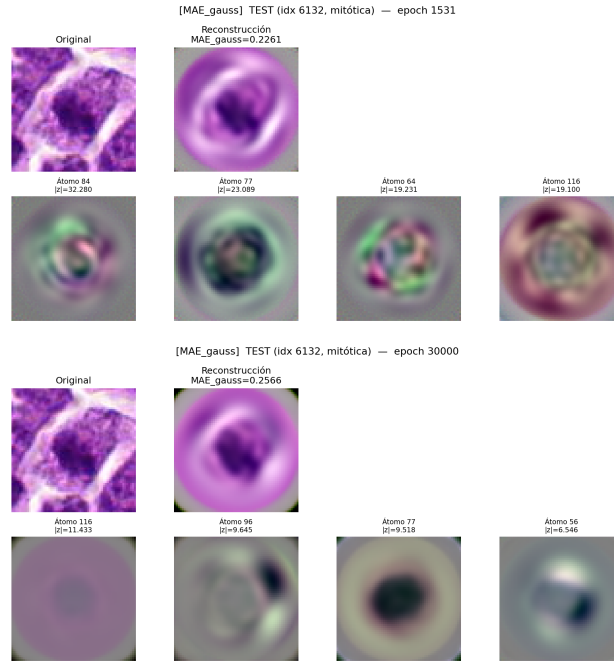


Figura A.1: Parche mitótico de test (idx 6132), mae_aug_128 , en el checkpoint óptimo (arriba, $\text{MAE}_g = 0.2261$) y el checkpoint final (abajo, $\text{MAE}_g = 0.2566$).

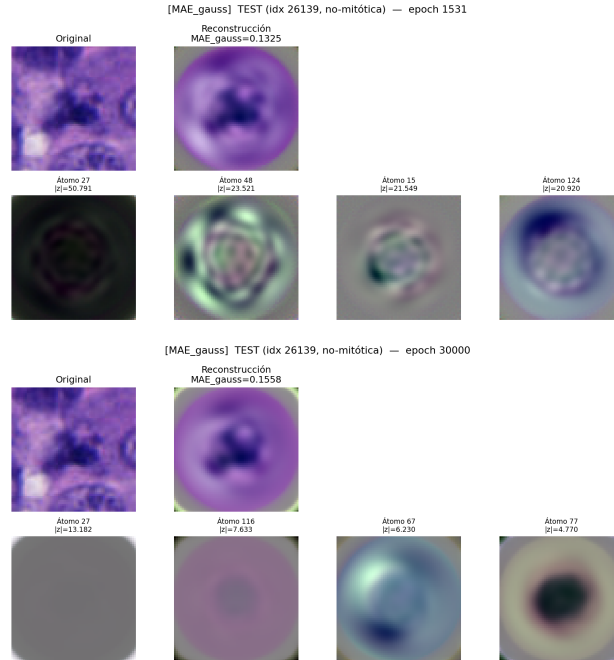


Figura A.2: Parche no mitótico de test (idx 26139), mae_aug_128 , en el checkpoint óptimo (arriba, $\text{MAE}_g = 0.1325$) y el checkpoint final (abajo, $\text{MAE}_g = 0.1558$).

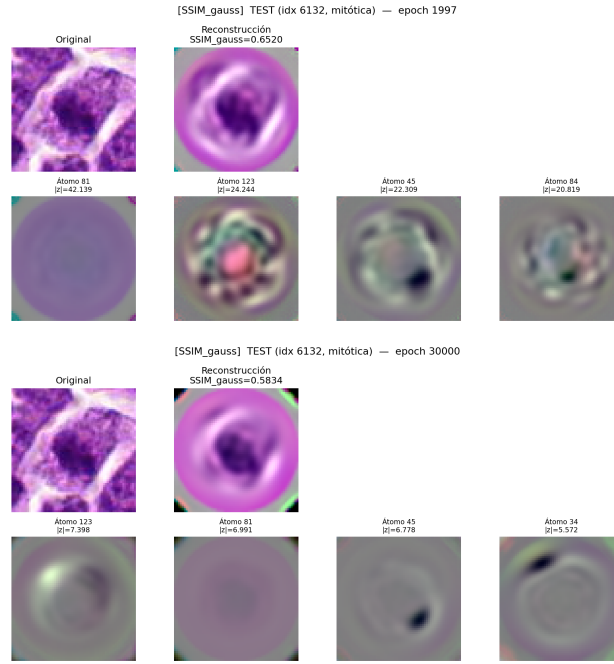


Figura A.3: Parche mitótico de test (idx 6132), $ssim_aug_128$, en el checkpoint óptimo (arriba, $SSIM_g = 0.6520$) y el checkpoint final (abajo, $SSIM_g = 0.5834$).

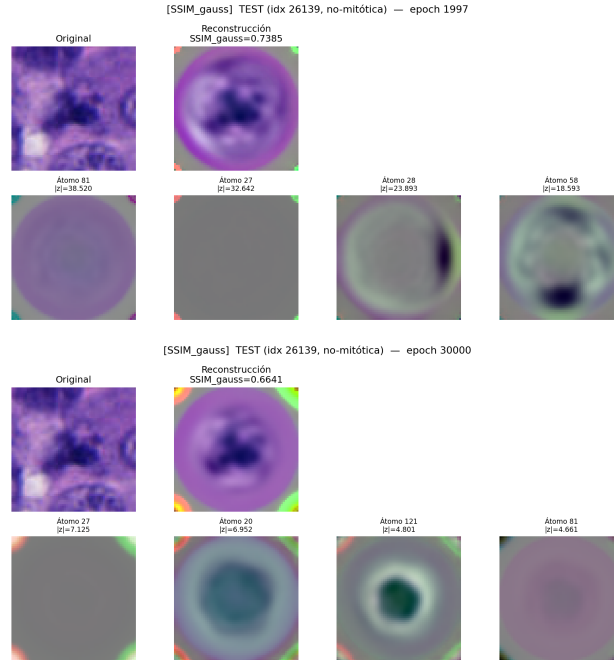


Figura A.4: Parche no mitótico de test (idx 26139), $ssim_aug_128$, en el checkpoint óptimo (arriba, $SSIM_g = 0.7385$) y el checkpoint final (abajo, $SSIM_g = 0.6641$).

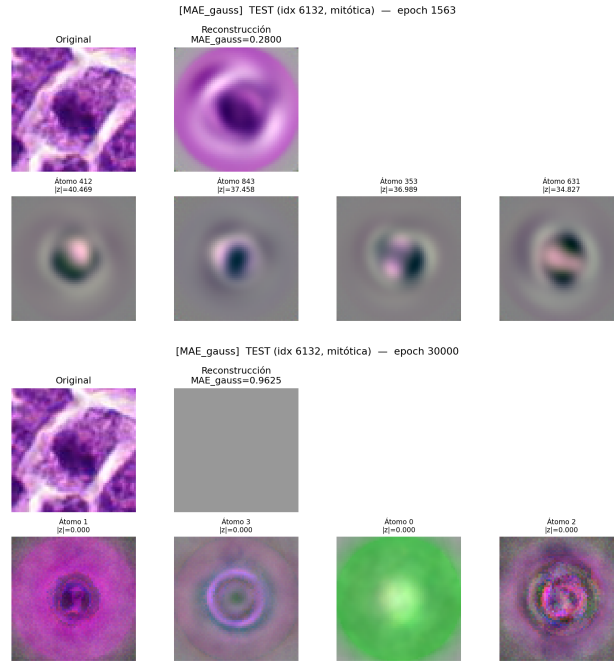


Figura A.5: Parche mitótico de test (idx 6132), mae_aug_1024 , en el checkpoint óptimo (arriba, $MAE_g = 0.2800$) y el checkpoint final (abajo, $MAE_g = 0.9625$).

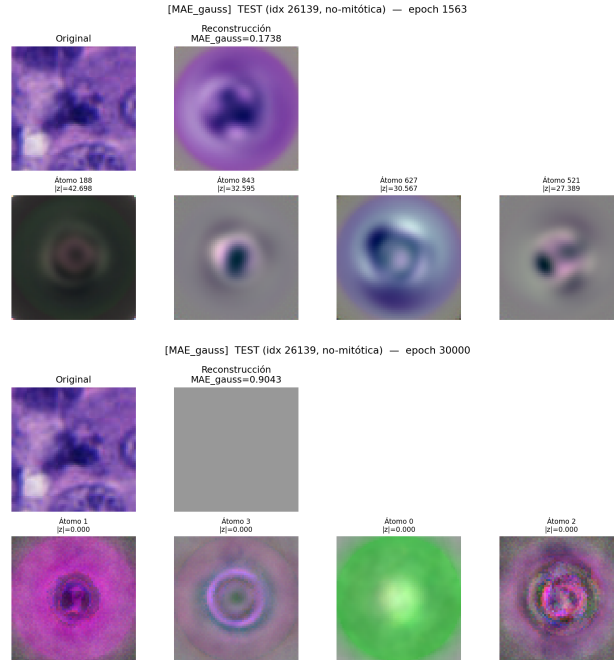


Figura A.6: Parche no mitótico de test (idx 26139), mae_aug_1024 , en el checkpoint óptimo (arriba, $MAE_g = 0.1738$) y el checkpoint final (abajo, $MAE_g = 0.9043$).

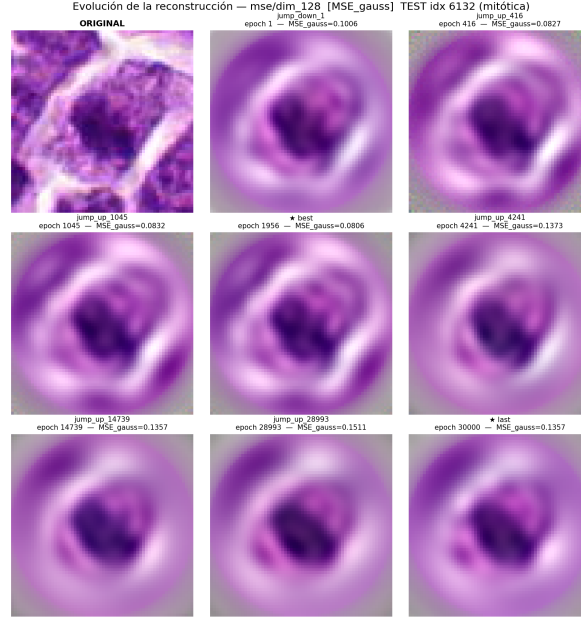


Figura A.7: Trayectoria de reconstrucción para `mse_aug_128`. El plateau alcanzado hacia la época 416 se mantiene hasta el checkpoint óptimo (época 1956); tras el salto de la época 4241 el error empeora a $MSE_g = 0.1373$ y no se recupera.

A.3 Trayectorias complementarias de reconstrucción

mse_aug_128. A diferencia de `mae_aug_128` y `ssim_aug_128`, esta configuración sí registra eventos de salto (sección 5.2.5), lo que permite reconstruir su trayectoria completa más allá de la simple comparación entre el checkpoint óptimo y el checkpoint final (sección 5.3.3): un primer plateau se alcanza ya hacia la época 416 y se sostiene hasta el checkpoint óptimo (época 1956, $MSE_g = 0.0806$ sobre la muestra mitótica de test); tras el salto de la época 4241, el error empeora a $MSE_g = 0.1373$ y no vuelve a recuperar el nivel previo en el resto del entrenamiento. La figura A.7 muestra la trayectoria completa.

mae_aug_128 y ssim_aug_128. Al no registrar ningún evento de salto, solo se dispone del checkpoint óptimo y el checkpoint final, con una degradación moderada y monótona entre ambos (MAE_g : $0.2261 \rightarrow 0.2566$; $SSIM_g$: $0.6520 \rightarrow 0.5834$). Las trayectorias correspondientes se muestran en las figuras A.8 y A.9.

mae_aug_1024. Reproduce el patrón de `mse_aug_1024` (sección 5.3.3): solo el checkpoint óptimo (época 1563) produce una reconstrucción no trivial, tal y como muestra la figura A.10.

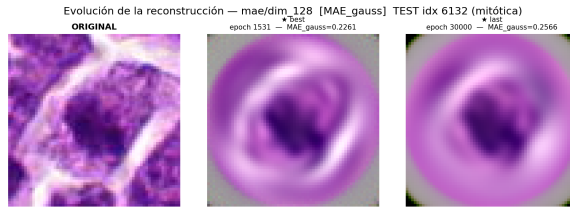


Figura A.8: Trayectoria de reconstrucción para mae_aug_128.

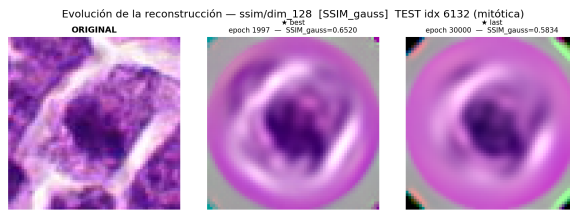


Figura A.9: Trayectoria de reconstrucción para ssim_aug_128.

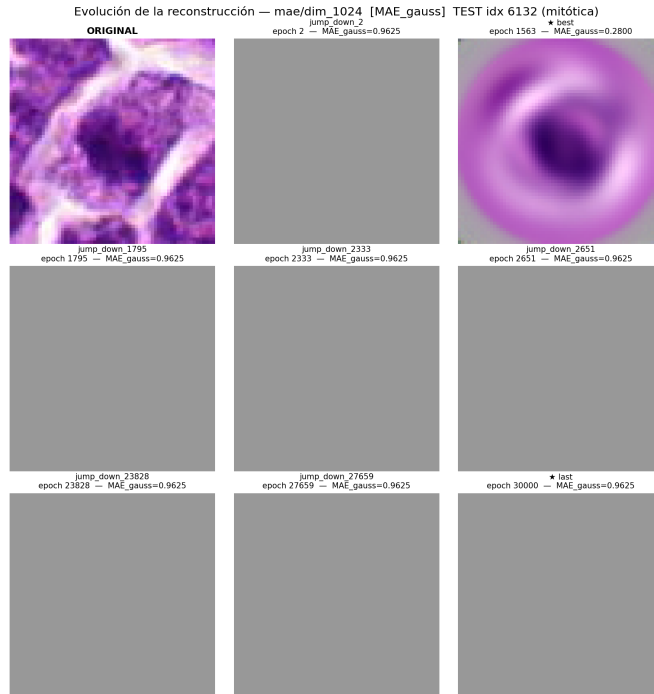


Figura A.10: Trayectoria de reconstrucción para mae_aug_1024.

Lista de acrónimos

CNN red neuronal convolucional. 11

FCN Fully Convolutional Network. 11

FISTA Fast Iterative Shrinkage-Thresholding Algorithm. 5–7, 16–18, 21, 23

H&E Hematoxilina-Eosina. 1–8, 10, 13, 18, 21, 22, 24, 30, 31, 33, 34, 74, 78

ICA Análisis de Componentes Independientes. 20

ISTA Iterative Shrinkage-Thresholding Algorithm. 4–7, 15–18, 21, 23, 34, 35

LISTA Learned Iterative Shrinkage-Thresholding Algorithm. 1, 3–7, 12, 14, 17, 18, 20–24, 27, 30–32, 34, 36, 48, 72, 74, 77

MAE Mean Absolute Error. 1, 4, 19, 22, 24, 25, 37, 39–41, 45, 48, 50, 52, 54, 56, 58, 60, 61, 67, 71, 72, 74–77

MIDOG Mitosis Domain Generalization Challenge. 1, 2, 5, 6, 8, 10, 11, 13, 14, 18, 21, 24, 30, 31, 34, 39, 48, 72, 77, 78

MSE Mean Squared Error. 1, 4, 13, 14, 19, 22, 24, 25, 37, 39–41, 45, 48, 50, 52, 54, 56, 58, 60, 61, 67, 71, 72, 74–77

PCA Análisis de Componentes Principales. 20, 77

SGD Stochastic Gradient Descent. 37

SSIM Structural Similarity Index. 1, 4, 22, 24, 37–41, 45, 48, 50, 52–54, 58, 60, 61, 67, 72, 74–78

VARPA Grupo de Visión Artificial e Reconhecimento de Padrões. 24, 28, 48

WSI Whole Slide Image. 6–9, 31

Bibliografía

- [1] C. Elston and I. Ellis, “Pathological prognostic factors in breast cancer. I. the value of histological grade in breast cancer,” *Histopathology*, vol. 19, no. 5, 1991.
- [2] M. Veta, P. J. van Diest, S. M. Willems *et al.*, “Assessment of algorithms for mitosis detection in breast cancer histopathology images,” *Medical Image Analysis*, vol. 20, no. 1, 2015.
- [3] M. Aubreville, C. A. Bertram, T. A. Donovan *et al.*, “Mitosis domain generalization in histopathology images – The MIDOG 2022 challenge,” *Medical Image Analysis*, vol. 84, 2023.
- [4] Y. Bengio, A. Courville, and P. Vincent, “Representation learning: A review and new perspectives,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, 2013.
- [5] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [6] B. A. Olshausen and D. J. Field, “Sparse coding with an overcomplete basis set: A strategy employed by V1?” *Vision Research*, vol. 37, no. 23, 1997.
- [7] L. Jiao, Y. Yang, F. Liu, S. Yang, and B. Hou, “The new generation brain-inspired sparse learning: A comprehensive survey,” *IEEE Transactions on Artificial Intelligence*, vol. 3, no. 6, 2022.
- [8] K. Gregor and Y. LeCun, “Learning fast approximations of sparse coding,” in *Proceedings of the 27th International Conference on Machine Learning*, 2010.
- [9] M. Amgad, H. Elfandy *et al.*, “Structured crowdsourcing enables convolutional segmentation of histology images,” *Bioinformatics*, vol. 35, no. 18, pp. 3461–3467, 2019.
- [10] N. Kumar, R. Verma, D. Anand *et al.*, “A multi-organ nucleus segmentation challenge,” *IEEE Transactions on Medical Imaging*, vol. 39, no. 5, pp. 1380–1391, 2020.

- [11] M. Peikari, S. Salama, S. Nofech-Mozes, and A. L. Martel, "Automatic cellularity assessment from post-chemotherapy histology slides," *Cytometry Part A*, vol. 91, no. 11, pp. 1078–1087, 2017.
- [12] G. Litjens, T. Kooi, B. E. Bejnordi *et al.*, "A survey on deep learning in medical image analysis," *Medical Image Analysis*, vol. 42, pp. 60–88, 2017.
- [13] D. Tellez, G. Litjens, P. Bándi *et al.*, "Quantifying the effects of data augmentation and stain color normalization in convolutional neural networks for computational pathology," *Medical Image Analysis*, vol. 58, p. 101544, 2019.
- [14] M. Veta, Y. J. Heng, N. Stathonikos *et al.*, "Predicting breast tumor proliferation from whole-slide images: the TUPAC16 challenge," *Medical Image Analysis*, vol. 54, pp. 111–121, 2019.
- [15] C. A. Bertram, M. Aubreville, C. Marzahl *et al.*, "A large-scale dataset for mitotic figure assessment on whole slide images of canine breast cancer," *Scientific Data*, vol. 7, no. 1, pp. 1–10, 2020.
- [16] D. C. Cireşan, A. Giusti, L. M. Gambardella, and J. Schmidhuber, "Mitosis detection in breast cancer histology images with deep neural networks," in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2013*, ser. Lecture Notes in Computer Science, vol. 8150. Springer, 2013, pp. 411–418.
- [17] H. Chen, Q. Dou, X. Wang, J. Qin, and P.-A. Heng, "Mitosis detection in breast cancer histology images via deep cascaded networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016.
- [18] C. Li, X. Wang, W. Liu, and L. J. Latecki, "DeepMitosis: Mitosis detection via deep detection, verification and segmentation networks," *Medical Image Analysis*, vol. 45, pp. 121–133, 2018.
- [19] M. Sebai, X. Wang, and T. Wang, "MaskMitosis: a deep learning framework for fully supervised, weakly supervised, and unsupervised mitosis detection in histopathology images," *Medical & Biological Engineering & Computing*, vol. 58, no. 7, pp. 1603–1623, 2020.
- [20] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *International Conference on Machine Learning*, 2020.

- [21] A. Bardes, J. Ponce, and Y. LeCun, “VICReg: Variance-invariance-covariance regularization for self-supervised learning,” in *International Conference on Learning Representations*, 2022.
- [22] O. Ciga, T. Xu, and A. L. Martel, “Self supervised contrastive learning for digital histopathology,” *Machine Learning with Applications*, vol. 7, p. 100198, 2022.
- [23] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019.